



# 机器学习概论

讲解人： 乜鹏

时 间： 2024年3月31日



# 第一章 数据挖掘概论

# 目 录

01

数据时代

---

02

数据挖掘概念

---

03

数据挖掘的案例

---

04

数据挖掘的应用

---

## 1.1 数据时代

### > ONE SECOND

会有60张照片上传到网络

### > ONE MINUTE

会有60小时视频上传到视频网站youtube

### > ONE DAY

在facebook有40亿的信息扩散

每个智能手机用户平均会安装65个应用

有2940亿封邮件发出，相当于美国两年的纸质信件数量

发出的社区帖子达200万个，相当于《时代》杂志770年的文字量

互联网产生的全部内容可以刻满1.68亿张DVD

## 1.1 数据时代

### 马云在2017年中国国际大数据产业博览会的“机器智能” 高峰对话会上演讲

- 在大数据时代，未来30年将重新定义“变革”。
- 在大数据时代，人类获得数据能力远远超过大家想象，我们对世界的认识要提升到新的高度。

## 1.1 数据时代

### 马云在2017年中国国际大数据产业博览会的“机器智能”高峰对话会上演讲

- 你们知道购买最大的内衣罩杯的消费者是在哪几个省？我这儿都有。最小的是哪个省？



## 1.1 数据时代

### 数据挖掘发展的动力——需要

- 数据爆炸问题--数据采集工具和成熟的数据库技术使得大量的数据被收集，存储在数据库数据仓库或其他信息库中以待分析。
- 我们拥有丰富的数据，但是缺乏有用的信息

## 1.1 数据时代

### 数据挖掘的提出

- 数大批成熟的业务信息系统投入运行
- 信息系统多年运行，积累了海量的数据



## 1.1 数据时代

### 数据挖掘的提出



## 1.2 数据挖掘概念

### 什么是数据挖掘

- 数据挖掘是从数据中，发现其有用的信息，从而帮助我们做出决策（广义角度）

## 1.2 数据挖掘概念

### 什么是数据挖掘

- 数据挖掘是从大量的、不完全的、有噪声的、模糊的、随机的实际应用数据中，提取隐含在其中的、人们事先不知道的、但又是潜在有用的信息和知识，寻找其规律的技术，结合统计学、机器学习和人工智能技术的综合的过程（技术角度）

## 1.2 数据挖掘概念

### 数据挖掘技术

- 统计学

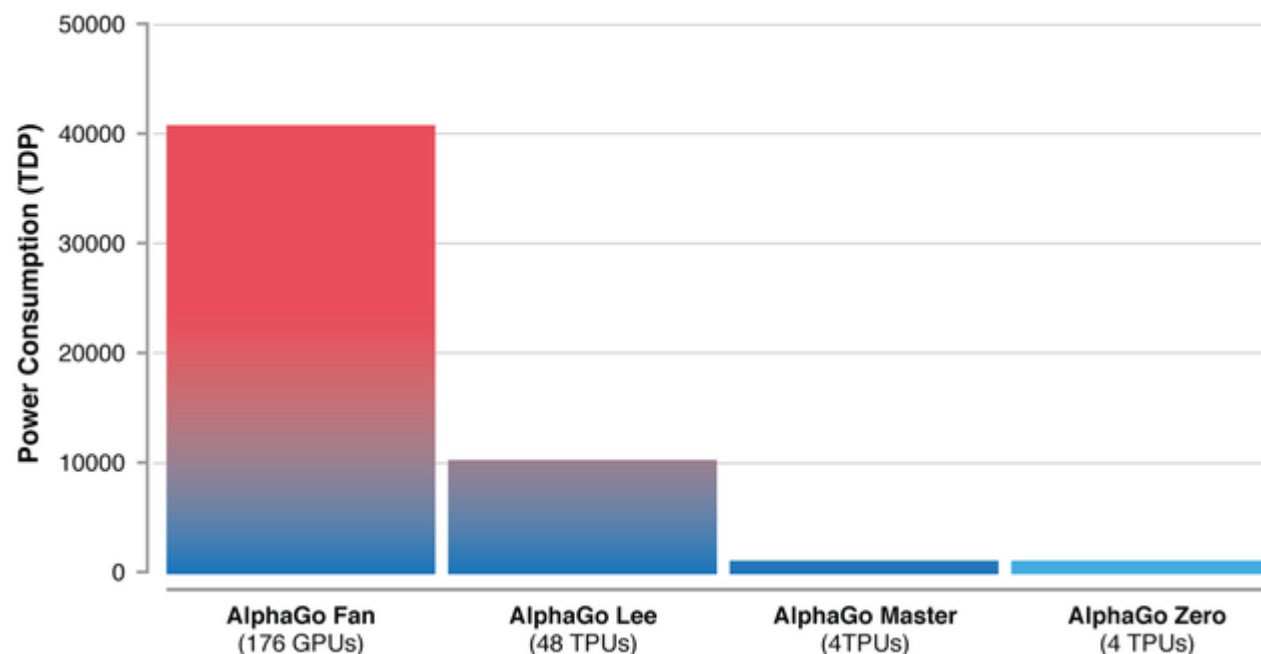
- 机器学习

- 统计学是关于认识客观现象总体数量特征和数量关系的科学。它是通过搜集、整理、分析统计资料，认识客观现象数量规律性的方法论科学。

## 1.2 数据挖掘概念

### 机器学习

- 人工智能棋手alphago先后战胜了两位顶尖围棋高手
- 九段李世乭以总比分1：4落败和九段棋手柯洁0：3落败
- AlphaGo Fan
- AlphaGo Lee
- AlphaGo Master
- AlphaGo Zero



## 1.2 数据挖掘概念

### 数据挖掘要做什么

- 数据挖掘的两大基本目标是预测和描述数据
- 其中预测的计算机建模及实现过程通常被称为：
- 监督学习(supervised learning)：从标记的训练数据来推断一个功能的机器学习任务。

## 1.2 数据挖掘概念

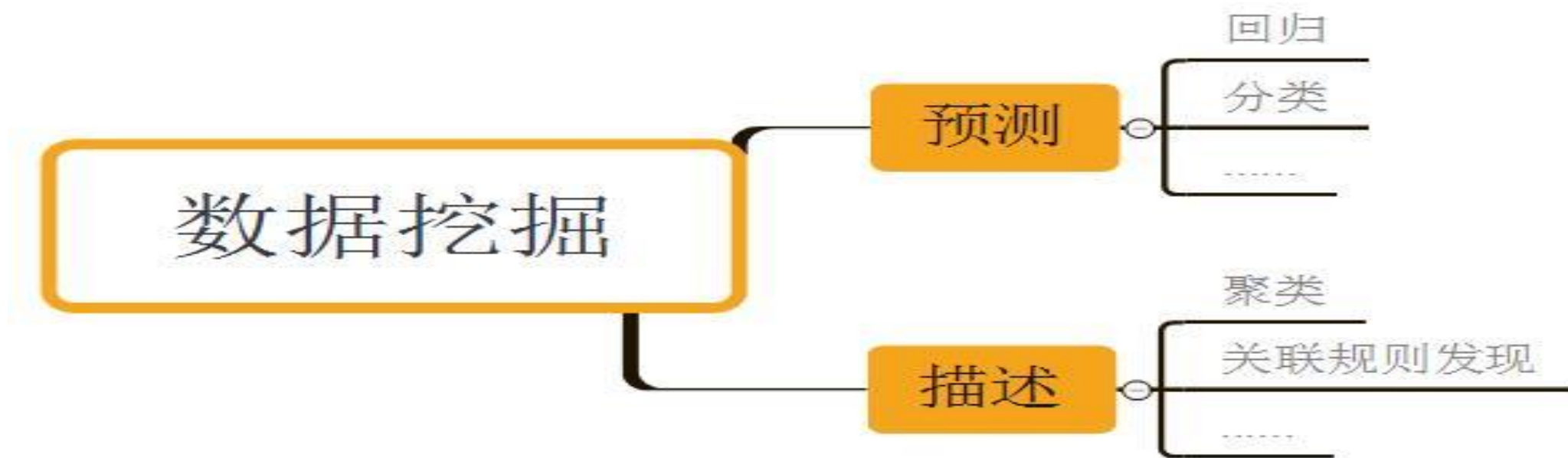
### 数据挖掘要做什么

- 数据挖掘的两大基本目标是预测和描述数据
- 描述的则通常被称为：
- 无监督学习(unsupervised learning)：根据类别未知(没有被标记)的训练样本解决模式识别中的各种问题。  
往更细分，数据挖掘的目标可以划分为以下这些：

## 1.2 数据挖掘概念

### 数据挖掘要做什么

■ 数据挖掘的两大基本目标是预测和描述数据





## 1.2 数据挖掘概念

### 预测与描述

■ 监督学习主要包括：

分类 - 将样本划分到几个预定义类之一

回归 - 将样本映射到一个真实值预测变量上

■ 无监督学习主要包括：

聚类 - 将样本划分为不同类(无预定义类)

关联规则发现 - 发现数据集中不同特征的相关性。

## 1.3 数据挖掘的案例

### 例子

- 垃圾电子邮件的分类
- 思考：人是如何进行分类的呢？



## 1.3 数据挖掘的案例

### 例子

- 股价走势预测
- 思考：你是如何进行价格预测的呢？

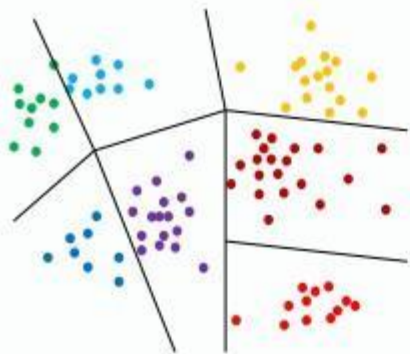


## 1.3 数据挖掘的案例

### 例子

- 客户群体的划分，大数据杀熟
- 思考：你是如何进行区别对待的呢？

#### Clustering: unsupervised learning



flat clustering

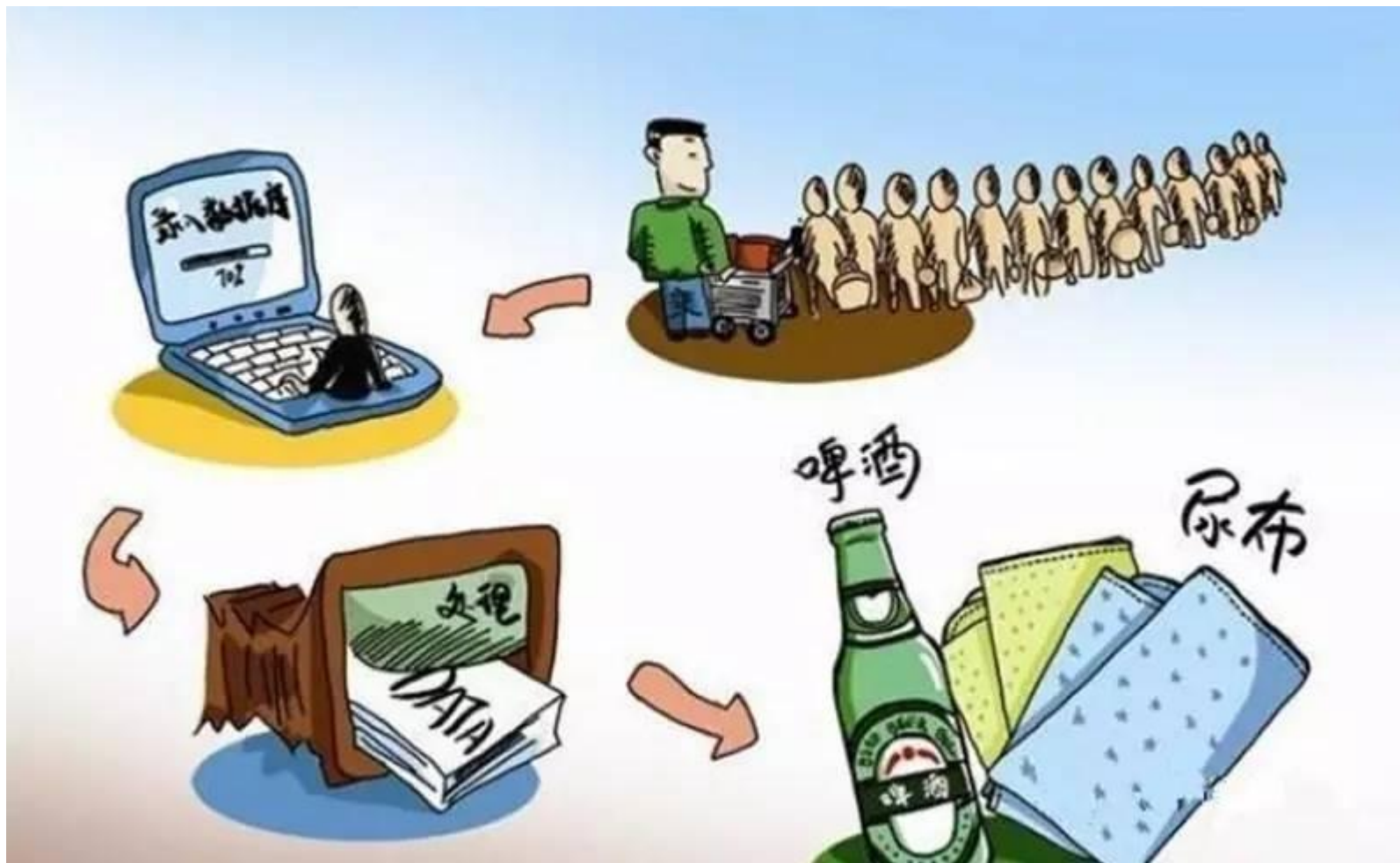


hierarchical clustering

## 1.3 数据挖掘的案例

### 例子

- 推荐商品, JD
- 思考: 你有过销售经验吗?



## 1.4 数据挖掘应用

### 数据挖掘的价值

- 沃尔玛基于每个月亿万的网络购物数据，并结合社交网络上有关产品的评分，开发机器学习语义搜索引擎北极星，方便浏览，在线购物者因此增加10%-15%，增加销售十多亿美元

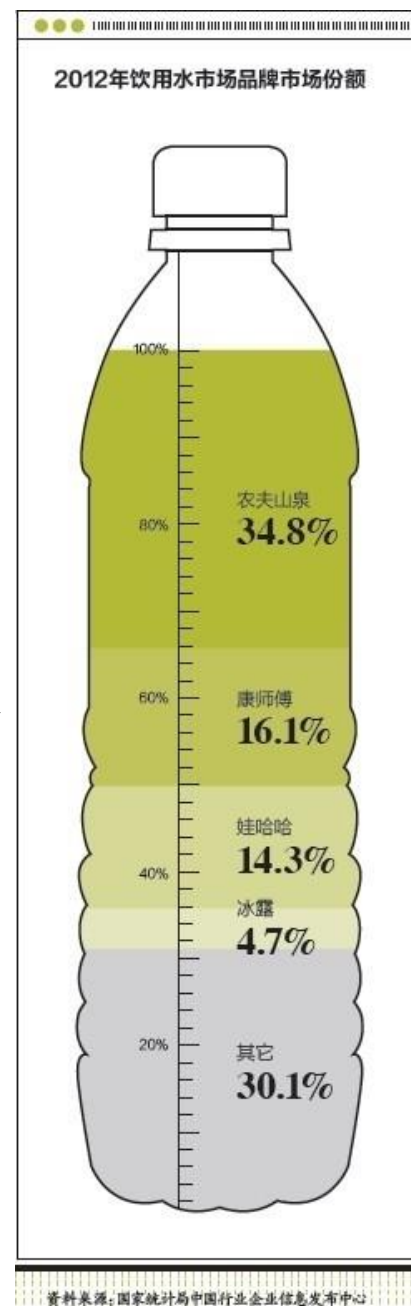




## 1.4 数据挖掘应用

### 数据挖掘的价值

- 农夫山泉用大数据卖矿泉水
- 在强大的数据分析能力做支持后，农夫山泉近年以30%-40%的年增长率，在饮用水方面快速超越了原先的三甲：娃哈哈、乐百氏和可口可乐。



## 1.4 数据挖掘应用

### 数据挖掘的价值

- 百合网的大数据
- 百合网提出的心灵匹配算法（可以看到两个人心灵的契合程度）



The screenshot displays the Baihe.com website interface. At the top, there's a navigation bar with the Baihe.com logo and links for '免费注册' (Free Registration) and '登录' (Login). Below the navigation bar, there are four small images illustrating different types of relationships: '夫妻相' (Spouse-like), '一见钟情' (Love at first sight), '一见钟情' (Love at first sight), and '一见钟情' (Love at first sight).

In the center, there's a circular diagram representing the 'Heart Matching Test' (心灵匹配测试). The diagram features a central image of a couple, surrounded by six colored circles, each representing a different personality type: '哲学家型' (Philosopher type), '挑战者型' (Challenger type), '将军型' (General type), '主人型' (Master type), '表演者型' (Performer type), and '作家型' (Writer type).

To the right of the diagram, the text reads: '心灵匹配测试' (Heart Matching Test), '美国著名婚恋心理学家独家授权' (Exclusive authorization from the famous American marriage and love psychologist), '数千万用户称赞的爱情测试' (Love test praised by tens of millions of users), '23个心理维度全方位解析' (Comprehensive analysis from 23 psychological dimensions), and '帮你找到幸福指数更高的理想伴侣' (Help you find your ideal partner with a higher happiness index).

At the bottom right corner, there is a small '广告' (Advertisement) label.



## 1.4 数据挖掘应用

### 数据挖掘的其他应用领域

- 金融领域
- 医疗领域
- 交通领域
- 地理信息领域



## 第二章 Pandas

# 目 录

01

Pandas快速入门

---

02

Pandas案例分析

---

## 2.1 Pandas快速入门

### 统计学

- 统计学是关于认识客观现象总体数量特征和数量关系的科学
- 统计学的一个案例：子弹中的统计学

## 2.1 Pandas快速入门

### 大数据与统计学

- 回想刚才例子，统计需基于大量的数据进行验证
- 同样的，数据也可以基于统计进行分析

## 2.1 Pandas快速入门

### 统计学在数据挖掘中的应用

- 文本统计与文学作品鉴真
- 文本统计设计到的指标词频率、平均句长、平均词长

## 2.1Pandas快速入门

### 统计学常用指标

- 平均数
- 百分比
- 频率
- 倍数

## 2.1 Pandas快速入门

### 统计分析常用模块Pandas简介

- 官网链接: <http://pandas.pydata.org/>
- 简介: Pandas是python的一个数据分析包, 最初由AQR Capital Management于2008年4月开发, 并于2009年底开源出来, 目前由专注于Python数据包开发的PyData开发team继续开发和维护, 属于PyData项目的一部分。Pandas最初被作为金融数据分析工具而开发出来, 因此, pandas为时间序列分析提供了很好的支持。



## 2.1 Pandas快速入门

### 强大的Pandas

#### 基本功能

- 开发pandas时提出的需求
- 具备按轴自动或显式数据对齐功能的数据结构
- 集成时间序列功能
- 既能处理时间序列数据也能处理非时间序列数据的数据结构

## 2.1 Pandas快速入门

### 强大的Pandas

#### 基本功能

- 数学运算和约简（比如对某个轴求和）可以根据不同的元数据（轴编号）执行
- 灵活处理缺失数据
- 合并及其他出现在常见数据库（例如基于SQL的）中的关系型运算

## 2.1 Pandas快速入门

### 强大的Pandas

#### 数据结构

- Series(一维)
- DataFrame(二维)
- Panel(三维)

## 2.1 Pandas快速入门

### 数据结构Series

- Series是一种类似于一维数组的对象，它由一组数据（各种NumPy数据类型）以及一组与之相关的数据标签（即索引）组成
- Series的字符串表现形式为：索引在左边，值在右边

## 2.1 Pandas快速入门

### 三方面来了解Series

- 创建
- 读写
- 运算

## 2.1 Pandas快速入门

### 数据结构Series的创建

```
>>> ser1 = Series(range(4))
```

```
>>> ser2 = Series(range(4), index = ["a", "b", "c", "d"])
```

```
>>> sdata = {'Ohio': 35000, 'Texas': 71000, 'Oregon':  
16000, 'Utah': 5000}
```

```
>>> ser3 = Series(sdata)
```

- 注：传递的data是一个dict字典类型对象，并且传递了index参数，那么对应的值将从字典中取出。否则，index的值将由字典对象里的key值进行构造

## 2.1 Pandas快速入门

### Series的index和values

```
>>> ser2.index
```

```
>>> ser2.values
```

```
>>> ser2[["a", "c"]]
```

## 2.1 Pandas快速入门

### 数据结构Series的读写

下标访问和切片

```
>>>a = pd.Series([11, 22, 33, 44, 55])
```

```
>>>a[1:3]
```

```
>>>a[1]=2
```



## 2.1 Pandas快速入门

### 数据结构Series的运算

Series间的计算

```
>>>a = pd.Series([1, 2, 3, 4])
```

```
>>>b = pd.Series([1, 2, 1, 2])
```

```
>>>print(a + b)
```

```
>>>print(a * 2)
```

```
>>>print(a >= 3)
```

```
>>>print(a[a >= 3])
```

## 2.1 Pandas快速入门

### 数据结构Series的运算

Series函数的使用

```
>>>a = pd.Series([1, 2, 3, 4, 5])
```

平均值

```
>>>print(a.mean())
```

练习：了解一下Series还有哪些函数吧

## 2.1 Pandas快速入门

### 数据结构DataFrame

- DataFrame是一个表格型的数据结构，它含有一组有序的列，每列可以是不同的值类型（数值、字符串、布尔值等）
- DataFrame既有行索引也有列索引，它可以被看做由Series组成的字典（共用同一个索引）

## 2.1 Pandas快速入门

### 几方面来了解DataFrame

- 创建
- 索引
- apply方法
- 算术运算
- 缺失值处理

## 2.1 Pandas快速入门

### 数据结构DataFrame的创建

二维的ndarray

```
>>>import pandas as pd
```

```
>>>df1=pd.DataFrame(np.arange(9).reshape(3,3),column  
s=list('bcd'),index=['b','s','g'])
```

## 2.1 Pandas快速入门

### 数据结构DataFrame的创建

外部导入

```
>>> import pandas as pd
```

```
>>> df = pd.read_csv('./data/titanic.csv')
```

## 2.1 Pandas快速入门

### 数据结构DataFrame的创建

字典导入

```
>>>import pandas as pd
```

```
>>>data = {'country':['aaa','bbb','ccc'],  
'population':[10,12,14]}
```

```
>>>df_data = pd.DataFrame(data)
```

## 2.1 Pandas快速入门

### 数据结构DataFrame的创建

使用Series导入

```
>>> d = {'one': pd.Series([1., 2., 3.], index=['a', 'b', 'c']),  
'two': pd.Series([1., 2., 3., 4.], index=['a', 'b', 'c', 'd'])}  
>>> df = pd.DataFrame(d)  
>>> pd.DataFrame(d, index=['d', 'b', 'a'])
```



## 2.1Pandas快速入门

### 数据结构DataFrame的创建

| 类型                | 说明                                                  |
|-------------------|-----------------------------------------------------|
| 二维ndarray         | 数据矩阵，还可以传入行标和列标。                                    |
| 由数组、列表或元组组成的字典    | 每个序列会变成DataFrame的一列，所有序列的长度必须相同。                    |
| NumPy的结构化/记录数组    | 类似于“由数组组成的字典”                                       |
| 由Series组成的字典      | 每个Series会组成一列。如果没有显示指定索引，则各Series的索引会被合并成结果的行索引。    |
| 由字典组成的字典          | 各内层字典会成为一列。键会被合并成结果的行索引，跟“由Series组成的字典”的情况一样。       |
| 字典或Series的列表      | 各项将会成为DataFrame的一行。字典键或Series索引的并集将会成为DataFrame的列标。 |
| 由列表或元组组成的列表       | 类似于“二维ndarray”                                      |
| 另一个DataFrame      | 该DataFrame的索引将会被沿用，除非显示指定了其他索引。                     |
| NumPy的MaskedArray | 类似于“二维ndarray”的情况，只是掩码值在结果DataFrame会变成NA/缺失值。       |

## 2.1 Pandas快速入门

### 数据结构DataFrame的查看

```
>>> df.head(6)
```

```
>>> df.info()
```

```
>>> df.index
```

```
>>> df.columns
```

```
>>> df.dtypes
```

```
>>> df.values
```

## 2.1 Pandas快速入门

### 数据结构的索引对象

- pandas的索引对象负责管理轴标签和其他元数据（比如轴名称等）。构建Series或DataFrame时，所用到的任何数组或其他序列的标签都可以被转换成一个Index。

- 指定索引

```
>>> df = df.set_index('Name')
```

## 2.1Pandas快速入门

### 数据结构的索引对象

#### ■pandas中主要的index对象

| 类型            | 说明                                      |
|---------------|-----------------------------------------|
| index         | 最泛化的Index对象，将轴标签为一个由Python对象组成的NumPy数组。 |
| Int64Index    | 针对整数的特殊Index                            |
| MultIndex     | “层次化”索引对象，表示单个轴上的多层索引。可以看做由园数组组成的数组。    |
| DatetimeIndex | 存储纳秒级时间戳                                |
| PeriodIndex   | 针对Period数据的特殊Index                      |

## 2.1Pandas快速入门

### 数据结构的索引对象

#### ■Index的方法和属性 I

| 类型           | 说明                               |
|--------------|----------------------------------|
| append       | append 连接另一个Index对象，产生一个新的Index。 |
| diff         | 计算差集，并得到一个Index。                 |
| intersection | 计算交集                             |
| union        | 计算并集                             |
| isin         | 计算一个指示各值是否包含在参数集合中的布尔型数组         |
| delete       | 删除索引处的元素，并得到新的Index。             |

## 2.1Pandas快速入门

### 数据结构的索引对象

#### ■Index的方法和属性 II

| 类型           | 说明                      |
|--------------|-------------------------|
| drop         | 删除传入的值，并得到新的索引。         |
| insert       | 将元素插入到索引i处，并得到新的Index。  |
| is_monotonic | 当各元素均大于等于前一个元素时，返回True。 |
| is_unique    | 当Index没有重复值时，返回True。    |
| unique       | 计算Index中唯一值得数组          |

## 2.1 Pandas快速入门

### 索引操作

■ loc用label来去定位, iloc用position来去定位

```
>>> df.iloc[0:5,1:3]
```

```
>>> df = df.set_index('Name')
```

```
>>> df.loc['Heikkinen, Miss. Laina']
```

```
>>> df.loc['Heikkinen, Miss. Laina':'Allen, Mr. William Henry']
```

## 2.1 Pandas快速入门

### 重新索引

- 创建一个适应新索引的新对象，该Series的reindex将会根据新索引进行重排。如果某个索引值当前不存在，就引入缺失值



## 2.1 Pandas快速入门

### 重新索引

#### ■reindex函数的参数

| 类型         | 说明                                                                |
|------------|-------------------------------------------------------------------|
| index      | 用作索引的新序列。既可以是Index实例，也可以是其它序列型的Python数据结构。Index会被完全使用，就像没有任何复制一样。 |
| method     | 插值填充方式，ffill或bfill。                                               |
| fill_value | 在重新索引过程中，需要引入缺失值时使用的替代值。                                          |
| limit      | 前向或后向填充时的最大填充量                                                    |
| level      | 在MultiIndex的指定级别上匹配简单索引，否则选取其子集。                                  |
| copy       | 默认为True，无论如何都复制。如果为False，则新旧相等就不复制。                               |

## 2.1 Pandas快速入门

### 重新索引

reindex操作

```
>>> s1 = pd.Series(['a','b','c','d','e'],index=[2,1,3,5,4])
```

```
>>> s2 = s1.reindex([1,2,3,4,5,6],fill_value=0)
```

```
>>> s3 = s2.reindex(range(10),method='bfill')
```

## 2.1 Pandas快速入门

### groupby操作

```
>>> df = pd.DataFrame({'key': ['A', 'B', 'C', 'A', 'B', 'C', 'A', 'B', 'C'],  
                        'data': [0, 5, 10, 5, 10, 15, 10, 15, 20]})
```

```
>>> df.groupby('key').sum()
```

```
>>> df = pd.read_csv('./data/titanic.csv')
```

```
>>> df.groupby('Sex')['Age'].mean()
```

## 2.1 Pandas快速入门

### 函数应用和映射

- DataFrame的apply方法 (index)
- 对象的applymap方法 (因为Series有一个应用于元素级的map方法)

## 2.1 Pandas快速入门

### DataFrame的apply方法

```
>>> def plus(df,n):  
>>>     df['c'] = (df['a']+df['b'])  
>>>     df['d'] = (df['a']+df['b']) * n  
>>>     return df  
  
>>> list1 = [[1,3],[7,8],[4,5]]  
>>> df1 = pd.DataFrame(list1,columns=['a','b'])  
>>> df1 = df1.apply(plus,axis=1,args=(2,))  
>>> print(df1)
```

## 2.1 Pandas快速入门

### DataFrame的applymap方法

- 对象的applymap方法（因为Series有一个应用于元素级的map方法）
- apply对一个列进行整体运算
- applymap对一个DataFrame中的每个元素进行转换
- map对一个Series中的每个元素进行转换

## 2.1 Pandas快速入门

### 排序和排名

- 对行或列索引进行排序
- 对于DataFrame，根据任意一个轴上的索引进行排序
- 可以指定升序降序
- 按值排序
- 对于DataFrame，可以指定按值排序的列
- rank函数

## 2.1 Pandas快速入门

### 排序和排名

```
>>> df = pd.DataFrame([1, 2, 3, 4, 5], index=[10, 52, 24, 158, 112], columns=['S'])
```

```
>>> df.sort_index(inplace=True)
```

■ 对于DataFrame, 可以指定按值排序的列

```
>>> frame = pd.DataFrame({'b':[4, 7, -3, 2], 'a':[0, 1, 0, 1]})
```

```
>>> frame.sort_values(by='b')
```



## 2.1 Pandas快速入门

### 算术运算

- 对不同的索引对象进行算术运算
- 对齐操作会同时发生在行和列上，把2个对象相加会得到一个新的对象，其索引为原来2个对象的索引的并集：

```
>>> df1 =
```

```
pd.DataFrame(np.arange(9).reshape(3,3),columns=list('bcd'),index=['b','s','g'])
```

## 2.1 Pandas快速入门

### 算术运算

```
>>> df2 =
```

```
pd.DataFrame(np.arange(12).reshape(4,3),columns=list('c  
de'),index=['b','s','g','t'])
```

```
>>> df1 + df2
```

```
>>> df3 = df1.add(df2,fill_value='0')
```

## 2.1Pandas快速入门

### 汇总和计算描述统计

#### ■常用描述和汇总统计函数

| 类型             | 说明                         |
|----------------|----------------------------|
| count          | 非NA值的数量                    |
| describe       | 针对Series或各DataFrame列计算汇总统计 |
| min, max       | 计算最小值和最大值                  |
| argmin, argmax | 计算能够获取到最小值和最大值的索引位置（整数）    |
| idxmin, idxmax | 计算能够获取到最小值和最大值的索引值         |
| sum            | 值的总和                       |
| mean           | 值的平均数                      |
| median         | 值的算术中位数                    |
| mad            | 根据平均值计算平均绝对离差              |

## 2.1Pandas快速入门

### 汇总和计算描述统计

#### ■唯一值以及成员资格

| 类型           | 说明                                   |
|--------------|--------------------------------------|
| is_in        | 计算一个表示 “Series各值是否包含于传入的值序列中” 的布尔型数组 |
| unique       | 计算Series中的唯一值数组，按发现的顺序返回。            |
| value_counts | 返回一个Series，其索引为唯一值，其值为频率，按计数值降序排列。   |

## 2.1 Pandas快速入门

### 缺失数据处理

- NaN (Not a Number) 表示浮点数和非浮点数组中的缺失数据
- None也被当作NaN处理

## 2.1 Pandas快速入门

### 处理缺失数据 滤除缺失数据方法

- `dropna()`

- `drop()`

- `how`参数控制行为, `axis`参数选择轴, `thresh`参数控制留下的数量

## 2.1 Pandas快速入门

### 处理缺失数据 滤除缺失数据

```
>>> ser = pd.Series([4.5, 7.2, -5.3, 3.6], index=['d', 'b', 'a', 'c'])
```

```
>>> ser.drop('c')
```

```
>>> from numpy import NaN
```

```
>>> se1 = pd.Series([4, NaN, 8, NaN, 5])
```

```
>>> se1.dropna()
```

```
>>> se1[se1.notnull()]
```

## 2.1 Pandas快速入门

### 处理缺失数据 滤除缺失数据

```
>>> df = DataFrame(np.arange(9).reshape(3,3), index=['a','c','d'],  
columns=['oh','te','ca'])  
  
>>> df.drop('a')  
  
>>> df.drop(['oh','te'],axis=1)  
  
>>> df1=pd.DataFrame([[1,2,3],[NaN,NaN,2],[NaN,NaN,NaN],[8,8,NaN]])  
  
>>> df1.dropna()  
  
>>> df1.dropna(how='all')
```



## 2.1 Pandas快速入门

### 处理缺失数据 填充缺失数据

- fillna

- inplace参数控制返回新对象还是就地修改

## 2.1 Pandas快速入门

### 处理缺失数据 填充缺失数据

```
>>> from numpy import nan as NaN
```

```
>>> df1 = pd.DataFrame([[1,2,3],[NaN,NaN,2],[NaN,NaN,NaN],[8,8,NaN]])
```

```
>>> df1.fillna(100)
```

```
>>> df1.fillna({0:10,1:20,2:30})
```

■ 传入inplace=True直接修改原对象

```
>>> df1.fillna(0,inplace=True)
```

## 2.1 Pandas快速入门

### 处理缺失数据 填充缺失数据

```
>>> df2=pd.DataFrame(np.random.randint(0,10,(5,5)))
```

```
>>> df2.iloc[1:4,3]=NaN
```

```
>>> df2.iloc[2:4,4]=NaN
```

```
>>> df2.fillna(method='ffill')#用前面的值来填充
```

```
>>> df2.fillna(method='bfill',limit=2) #传入limit="    "限制填充个数:
```

```
>>> df2.fillna(method="ffill",limit=1,axis=1) #传入axis="    "修改填充方向:
```



# **第三章 数据挖掘与机器学习**

# 目 录

01

机器学习概述

---

02

机器学习的框架

---

03

模型的评判

---

04

过拟合问题

---

## 数据挖掘与机器学习

### 机器学习

- 机器学习无处不在
- 百度搜索
- 微信朋友圈，微博的相片分类功能

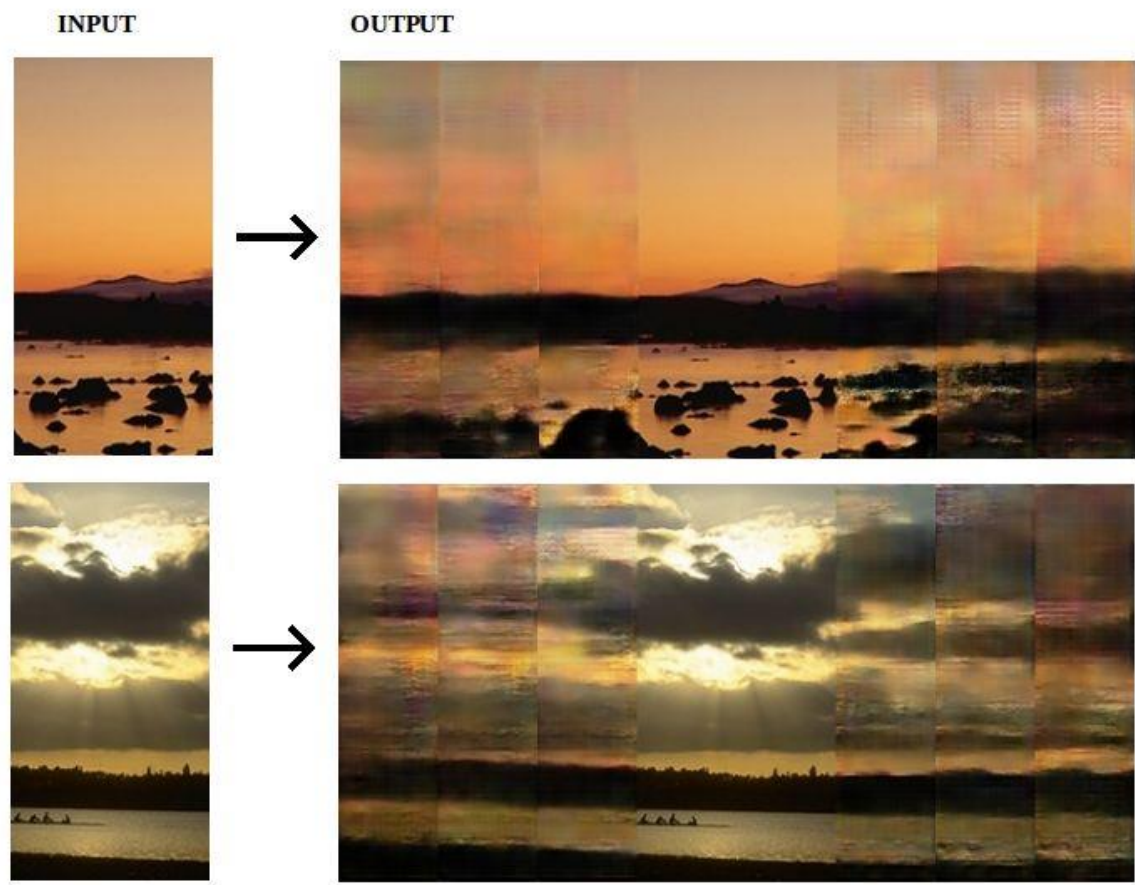
# 数据挖掘与机器学习

## 机器学习



## 数据挖掘与机器学习

### 通过例子来体会一下机器学习





## 数据挖掘与机器学习

### 通过例子来体会一下机器学习

数据从何而来

City



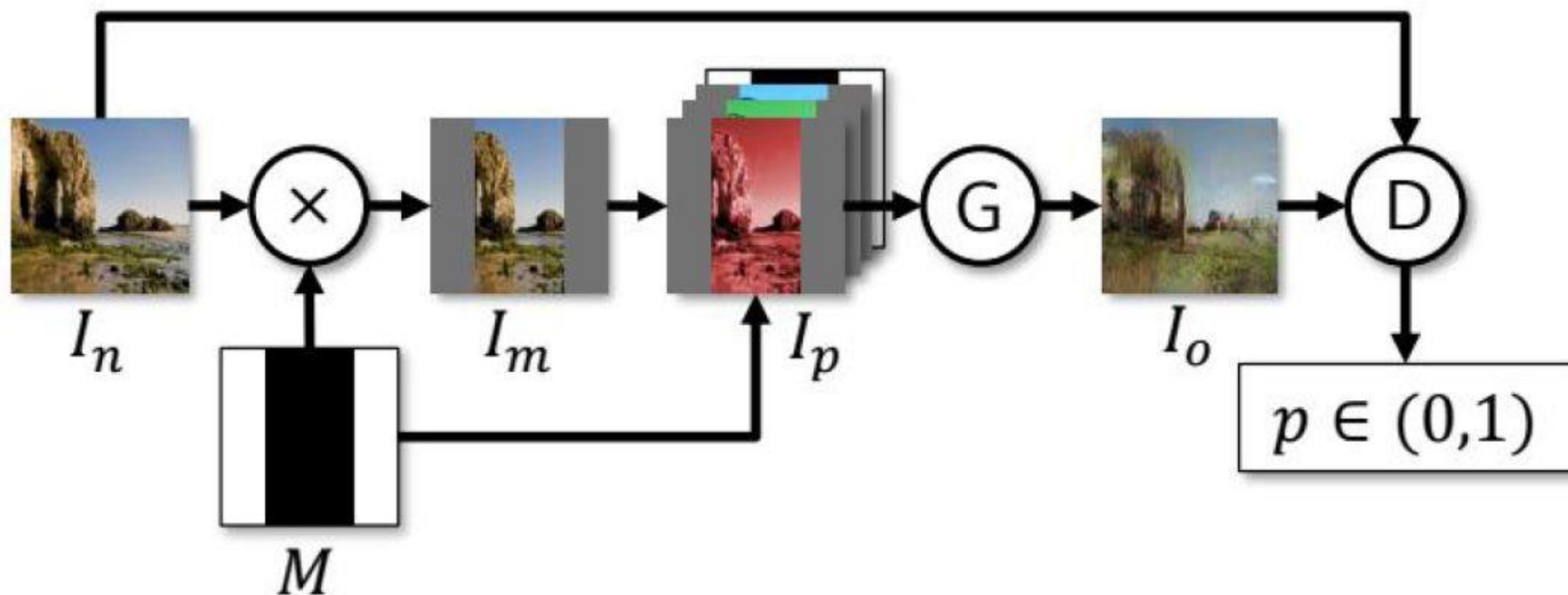
Places365



## 数据挖掘与机器学习

### 通过例子来体会一下机器学习

实现补全



### 什么是机器学习

- 机器学习的英文名称叫Machine Learning，简称ML，该领域主要研究的是如何使计算机能够模拟人类的学习行为从而获得新的知识。
- 简单来说，机器学习就是让计算机从大量的数据中学习相关的规律和逻辑，然后利用学习来的规律来预测以后的未知事物

## 数据挖掘与机器学习

### 机器学习如何学习？

- 思考人类学习过程
- 我们是否能将人类学习过程应用于机器呢？
- 实际上我们已经这么做了

## 数据挖掘与机器学习

### 人类学习的步骤

- 选择知识
- 选择学习方法
- 学习或记忆
- 运用
- 评测学习效果
- 知识保存脑海中

## 数据挖掘与机器学习

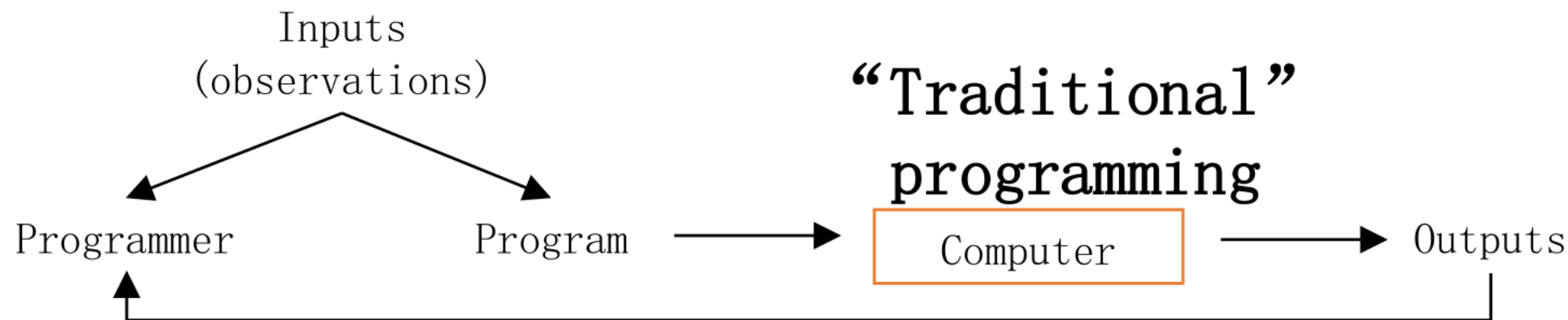
### 学习的步骤

- 数据的加载(第一步)
- 选择模型(第二步)
- 模型的训练(第三步)
- 模型的预测(第四步)
- 模型的评测(第五步)
- 模型的保存(第六步)

## 数据挖掘与机器学习

### 传统的程序模型与机器学习模型

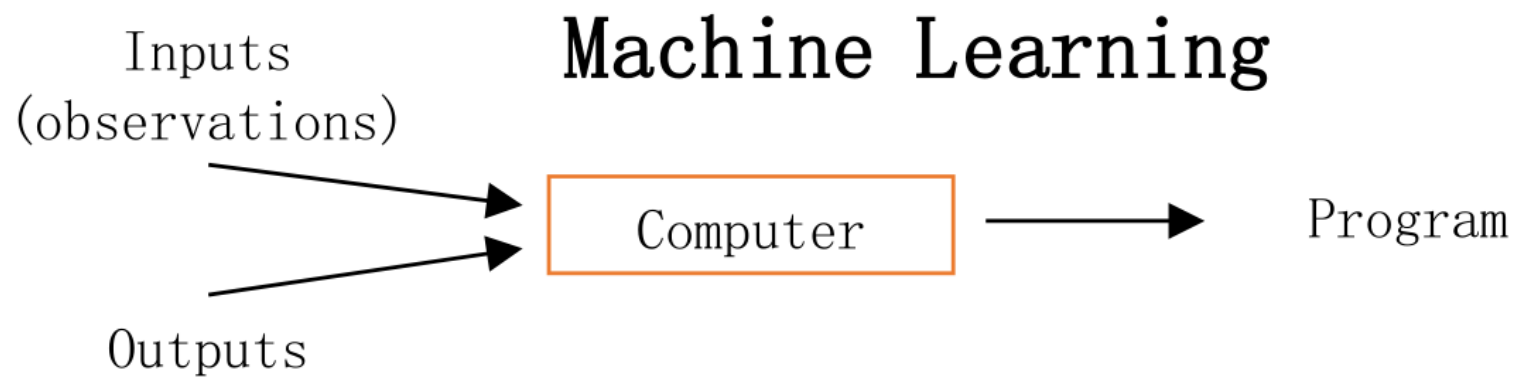
- 思考传统的程序模型的不足
- 传统计算机程序模型：



## 数据挖掘与机器学习

### 传统的程序模型与机器学习模型

#### ■ 机器学习模型：





## 数据挖掘与机器学习

### 机器学习库sklearn

- sklearn是机器学习中一个常用的python第三方模块，  
网址：<http://scikit-learn.org/stable/index.html>
- 里面对一些常用的机器学习方法进行了封装，不需要都实现所有的算法，只需要简单的调用sklearn里的模块就可以实现机器学习任务。
- 行业占有率高，spark等采用相同框架

## 数据挖掘与机器学习

### sklearn库的框架

- `train_x, train_y, test_x, test_y = getData()`
- `model1 = somemodel()`
- `model.fit(train_x, train_y)`
- `predictions = model.predict(test_x)`
- `score = score_function(test_y, predictions)`
- `joblib.dump(knn, 'filename.pkl')`

### 第一步使用sklearn导入数据

- 模块自带数据集，参考

<http://sklearn.apachecn.org/cn/0.19.0/datasets/index.html#datasets>

- 了解sklearn自带的数据集有哪些，数据集有哪些属性

## 数据挖掘与机器学习

### 第一步使用sklearn导入数据

#导入数据集模块

```
>>> from sklearn import datasets
```

#分别加载iris和digits数据集

```
>>> iris = datasets.load_iris()
```

```
>>> digits = datasets.load_digits()
```

## 数据挖掘与机器学习

### 第一步使用sklearn导入数据

#使用.data() 和.target()方法熟悉导入的数据结构

```
>>> print(iris.data)
```

```
>>> print(iris.target)
```

## 数据挖掘与机器学习

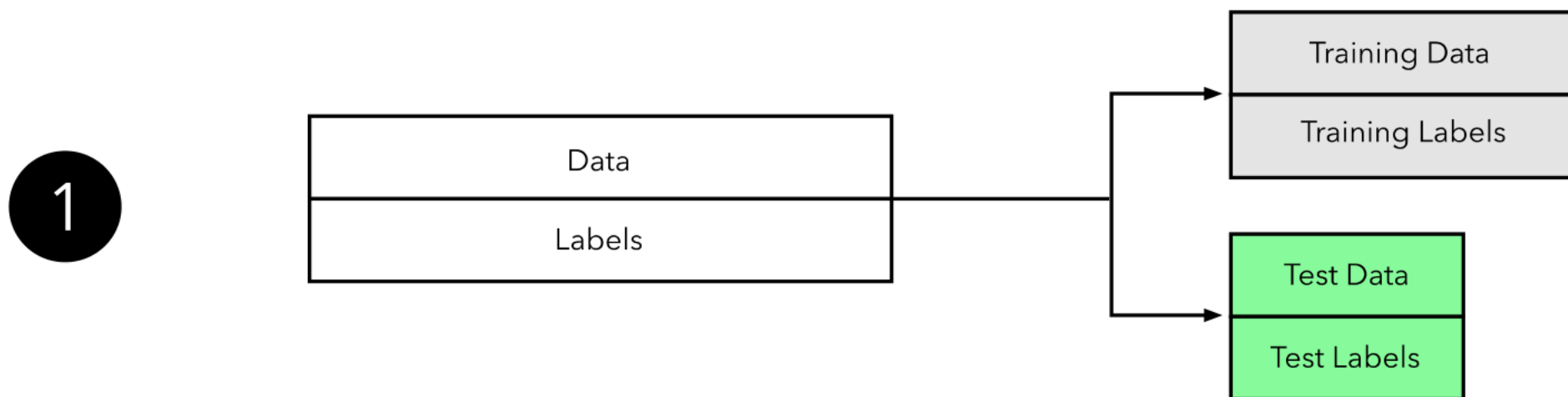
### 使用sklearn分割数据

机器学习是从数据的属性中学习经验，并将它们应用到新数据的过程。这就是为什么机器学习中评估算法的普遍实践是把数据分割成训练集（我们从中学习数据的属性）和测试集（我们测试这些性质）

## 数据挖掘与机器学习

### 使用sklearn分割数据

■测试的数据从何而来：



## 数据挖掘与机器学习

### 使用sklearn分割数据

```
>>> from sklearn.model_selection import  
train_test_split  
>>> X_train,X_test,y_train,y_test=train_test_split(iris.  
data,iris.target,test_size=0.4,random_state=0)
```



### 第二步使用sklearn模型的选择

学会加载模型，对于不同类型的数据选择不同的方法  
(既模型) 进行学习，关于各种不同的机器学习模型区别，我们会在下一章节讨论

### 第二步使用sklearn模型的选择

```
>>> from sklearn import svm  
>>> svc = svm.SVC()
```

### 第三步使用sklearn模型的训练

训练：我们使用scikit-learn模块实现机器学习训练，此处我们利用了训练数据集训练了“transformer”模型，向fit()方法提供输入训练数据集后即可训练模型

## 数据挖掘与机器学习

### 第三步使用sklearn模型的训练

```
>>> svc.fit(X_train, y_train)
```

### 第四步使用sklearn进行模型的预测

预测：预测既使用模型来预测未知的数据，上一步中我们的模型训练完成，我们就可以使用我们的模型进行预测了，快看看效果如何吧

### 第四步使用sklearn进行模型预测

```
> > > print(svc.predict([[5.84,4.4,6.9,2.5]]))
```

### 第五步机器学习评测的指标

通常从算法类型选择指标：

- 分类指标
- 回归指标
- 聚类指标

### 第五步机器学习评测的指标

- 分类常用的指标:
- 准确率( accuracy):对于给定的测试数据集, 分类器正确分类的样本数与总样本数之比
- AUC



### 第五步机器学习评测的指标

■ 回归常用的指标:

■ 均方误差 (mean\_squared\_error) : 是反映估计量与被估计量之间差异程度的一种度量

$$MSE(y, \hat{y}) = \frac{1}{n_{sample}} \sum_{i=0}^{n_{sample}-1} (y_i - \hat{y}_i)^2$$

■ 方均根差 RMSE(Root Mean Square Error)

### 第五步机器学习评测的指标

机器学习库sklearn中，我们使用metrics方法实现：

```
>>> import numpy as np
>>> from sklearn.metrics import accuracy_score
>>> y_pred = [0, 2, 1, 3]
>>> y_true = [0, 1, 2, 3]
>>> accuracy_score(y_true, y_pred)
```

### 第五步机器学习评测方法：交叉验证 (Cross validation)

- 为什么要使用交叉验证？
- 将原始数据分成K个子集（一般是均分），将每个子集数据分别做一次测试集（testing test），其余的K-1组子集数据作为训练集（training test），这样会得到K个模型，用这K个模型最终的验证集的分类指标的平均数作为此K-CV下分类器的性能指标

### 第五步机器学习评测方法：交叉验证 (Cross validation)

- K一般大于等于2，实际操作时一般从3开始取。交叉验证重复k次，每次选择一个子集作为测试集，并将k次的平均交叉验证识别正确率作为结果。
- 优点：所有的样本都被作为了训练集和测试集，每个样本都被验证一次。10-folder通常被使用。

### 第五步机器学习评测方法：交叉验证 (Cross validation)



### 第五步机器学习评测方法：交叉验证 (Cross validation)

机器学习库sklearn中，我们使用cross\_val\_score方法实现：

```
>>> from sklearn.model_selection import  
cross_val_score  
>>> scores = cross_val_score(knn, iris.data,  
iris.target, cv=5)
```

了解scoring参数及

[http://sklearn.apachecn.org/cn/0.19.0/modules/model\\_evaluation.html#scoring-parameter](http://sklearn.apachecn.org/cn/0.19.0/modules/model_evaluation.html#scoring-parameter)

### 第六步机器学习:模型的保存

这时候我们训练完成一个完美的模型，我们是那么喜欢这个模型，让我们保存它，以后他就可以长时间存在啦

### 第六步机器学习:模型的保存

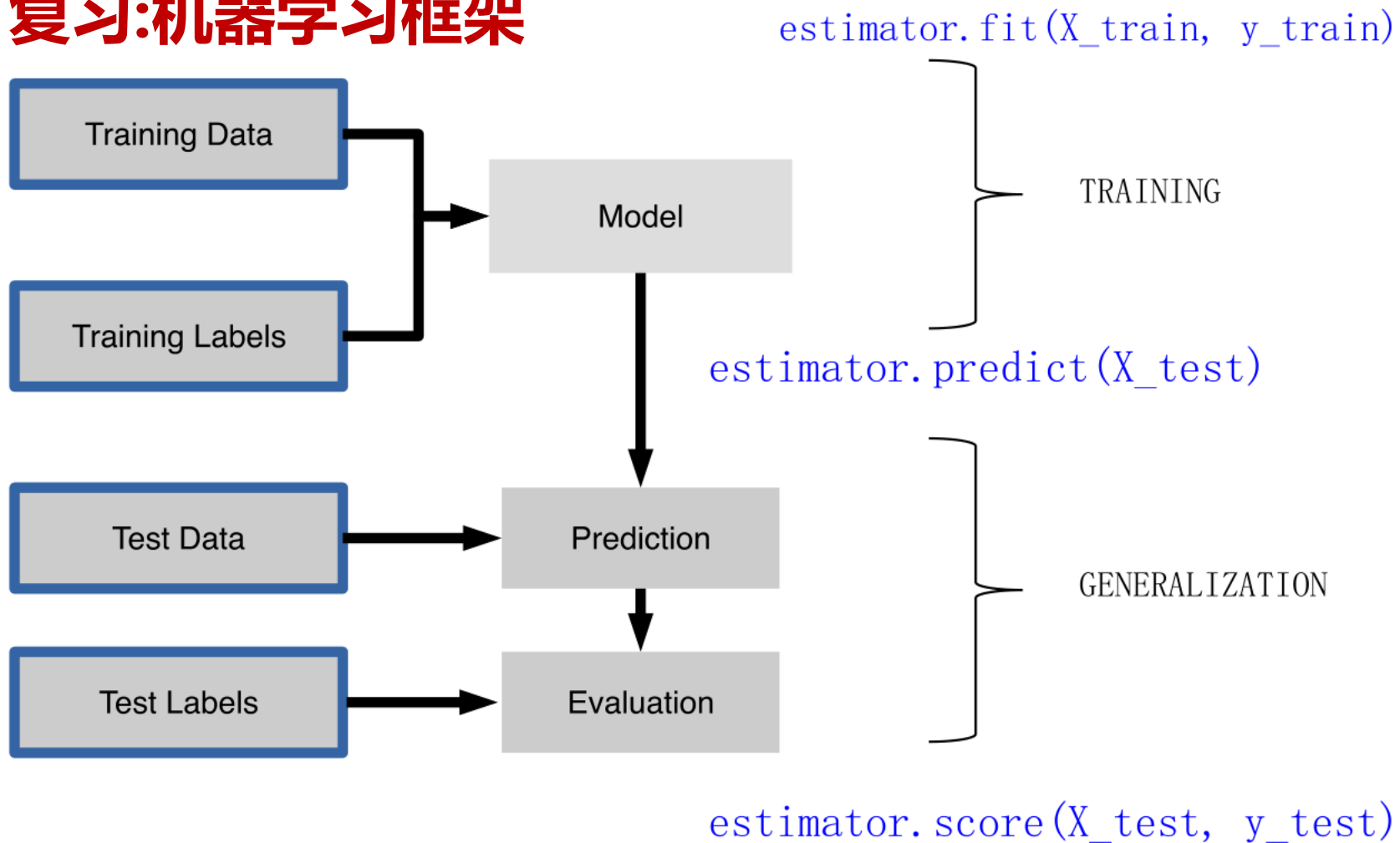
机器学习库sklearn中，我们使用joblib方法实现：

```
>>> from sklearn.externals import joblib  
>>> joblib.dump(knn, 'filename.pkl')  
>>> knn1 = joblib.load('filename.pkl')  
#测试读取后的Model  
>>> print(knn1.score(X_test, y_test))
```



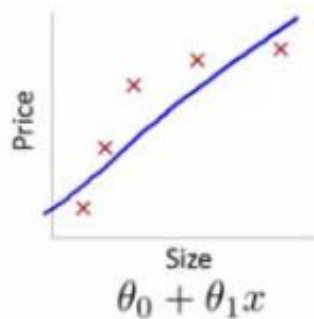
## 数据挖掘与机器学习

### 复习:机器学习框架

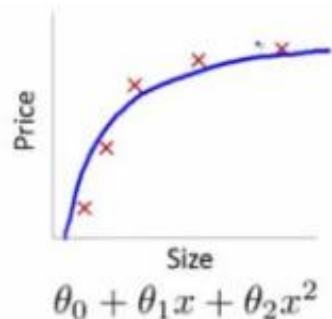


### 机器学习存在的问题过拟合

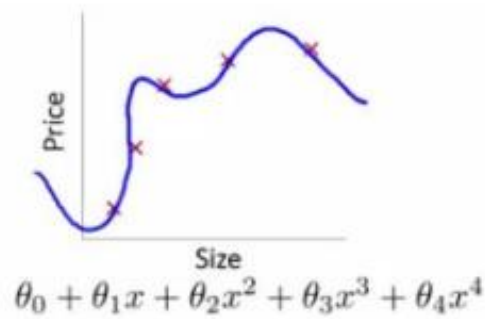
- 过拟合是指为了得到一致假设而使假设变得过度严格。  
避免过拟合是机器学习设计中的一个核心任务。



High bias  
(underfit)



"Just right"



High variance  
(overfit)

### 拟合问题的原因

- 1.使用的模型比较复杂，学习能力过强
- 2.有噪声存在
- 3.数据量有限
- 过拟合：做的太过好以至于偏离了原本，泛化能力差
- 欠拟合：泛化能力强，但过于泛化

## 数据挖掘与机器学习

### 拟合问题的解决

- 寻找参数的最优：超参数优化器
- 使用sklearn中的学习曲线



# 第四章 分类算法

# 目 录

01

分类算法概述

---

02

概率模型

---

03

朴素贝叶斯算法

---

04

向量空间模型

---

# 目 录

05

K邻近分类器

---

06

多问题的分类

---



# 分类算法的概述



## 引言

### 机器学习算法的最普通类型是什么

- 监督学习算法-就是我们教计算机如何做事情
- 无监督学习算法-在非监督学习中，我们将让计算机自己学习

## 引言

### 监督学习可分为

- 分类
- 回归

## 引言

### 分类方法的定义

- 分类分析的是根据已知类别的训练集数据，建立分类模型，并利用该分类模型预测未知类别数据对象所属的类别。

## 引言

### 分类方法的应用

- 模式识别 (Pattern Recognition) , 就是通过计算机用数学技术方法来研究模式的自动处理和判读。模式识别的目标往往是识别, 即分析出待测试的样本所属的模式类别

## 引言

### 分类方法的应用

- 预测，从利用历史数据记录中自动推导出对给定数据的推广描述，从而能对未来数据进行类预测

## 引言

### 现实应用案例用

- 行为分析
- 物品识别、图像检测
- 电子邮件的分类（垃圾邮件和非垃圾邮件等）
- 新闻稿件的分类、手写数字识别、个性化营销中的客户群分类、图像/视频的场景分类等

## 引言

### 分类器

- 分类的实现方法是创建一个分类器（分类函数或模型），该分类器能把待分类的数据映射到给定的类别中。
- 创建分类的过程与机器学习的一般过程一致

## 引言

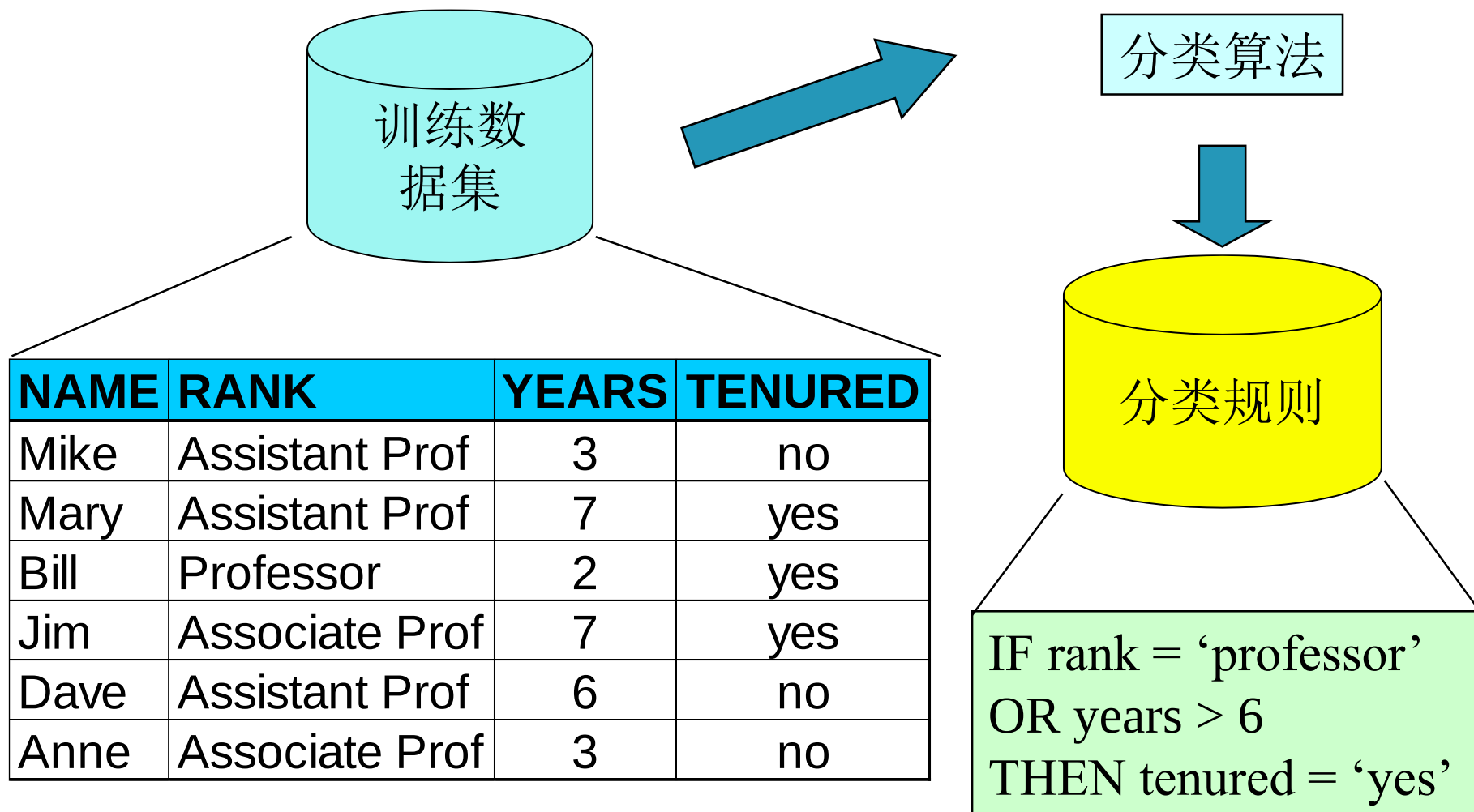
### 分类器的构建

- 回顾构建一个机器学习框架的基本步骤：
  - (1) 数据的加载。
  - (2) 选择模型。
  - (3) 模型的训练。
  - (4) 模型的预测。
  - (5) 模型的评测。
  - (6) 模型的保存。



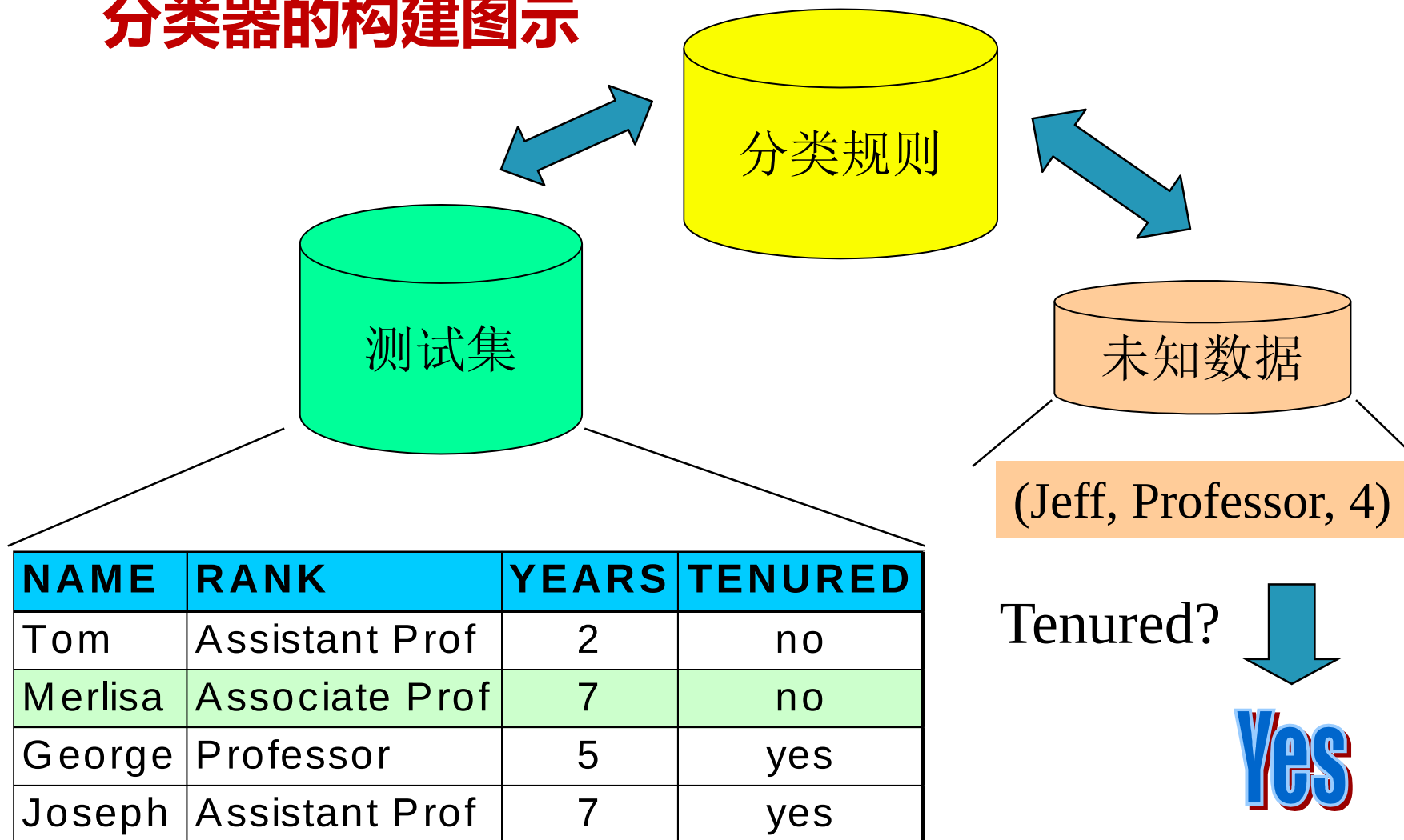
## 引言

### 分类器的构建图示



## 引言

### 分类器的构建图示



## 引言

### 分类器的构建标准

- 使用下列标准比较分类和预测方法
- 预测的准确率：模型正确预测新数据的类编号的能力
- 速度：产生和使用模型的计算花销
- 健壮性：给定噪声数据或有空缺值的数据，模型正确预测的能力
- 可伸缩性：对大量数据，有效的构建模型的能力
- 可解释性：学习模型提供的理解和洞察的层次biao

## 概率模型

### 贝叶斯简介

- 贝叶斯（约1701-1761），英国数学家
- 贝叶斯方法源于他生前解决逆概的一篇文章



## 概率模型

### 贝叶斯要解决的问题

- 正向概率：假设袋子里有 $N$ 个白球， $M$ 个黑球，随机摸一个，摸出黑球的概率有多大
- 逆向概率：如果事先不知道袋子里黑白球的比例，随机摸出几个球，根据这些球的颜色，可以推测袋子里面的黑白球比例

## 概率模型

### 一个例子



男生：60%  
女生：40%

## 概率模型

### 一个例子

- 男生总是穿长裤，女生则一半穿长裤一半穿裙子
- 正向概率：随机选取一个学生，穿长裤的概率和穿裙子的概率是多大
- 逆向概率：迎面走来一个穿长裤的学生，无法确定该学生的性别，请问该学生是女生的概率有多大

## 概率模型

### 一个例子

- 假设学校里面人的总数是U

- 穿长裤的男生:  $U * P(\text{Boy}) * P(\text{Pants}|\text{Boy})$

$P(\text{Boy})$ 是男生的概率 = 60%

$P(\text{Pants}|\text{Boy})$ 是条件概率, 即在Boy的条件下, 穿长裤的概率是多大, 这里是100%

- 穿长裤的女生:  $U * P(\text{Girl}) * P(\text{Pants}|\text{Girl})$



## 概率模型

### 一个例子

■ 求解：穿长裤的总数：

$$U * P(\text{Boy}) * P(\text{Pants}|\text{Boy}) + U * P(\text{Girl}) * P(\text{Pants}|\text{Girl})$$

■ 穿长裤的人为女生的概率：

$$P(\text{Girl}|\text{Pants})$$

$$= U * P(\text{Girl}) * P(\text{Pants}|\text{Girl}) / \text{穿长裤的总数}$$

$$= U * P(\text{Girl}) * P(\text{Pants}|\text{Girl}) / [U * P(\text{Boy}) * P(\text{Pants}|\text{Boy}) + U * P(\text{Girl}) * P(\text{Pants}|\text{Girl})]$$

## 概率模型

### 一个例子

- 与总人数有关吗?
- 分子分母均包含总人数, 结果与总人数无关, 可以消去
- $$P(\text{Girl}|\text{Pants}) = \frac{P(\text{Girl}) * P(\text{Pants}|\text{Girl})}{[P(\text{Boy}) * P(\text{Pants}|\text{Boy}) + P(\text{Girl}) * P(\text{Pants}|\text{Girl})]}$$
- 分母就是  $P(\text{Pants})$

## 概率模型

### 贝叶斯公式

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

$$P(\text{类别}|\text{特征}) = \frac{P(\text{特征}|\text{类别})P(\text{类别})}{P(\text{特征})}$$

## 朴素贝叶斯

### 朴素贝叶斯分类器

- 我们介绍的第一个分类学习方法是朴素贝叶斯（Naive Bayes）模型, 它是一种基于概率的学习方法
- “朴素”指的是条件的独立性
- 我们一起通过一个例子来了解一下朴素贝叶斯分类算法

## 朴素贝叶斯

### 朴素贝叶斯案例

- 现在给我们的问题是，如果一对男女朋友，男生想女生求婚，男生的四个特点分别是不帅，性格不好，身高矮，不上进，请你判断一下女生是嫁还是不嫁？

## 朴素贝叶斯算法

### 朴素贝叶斯案例

| 帅？ | 性格好？ | 身高？ | 上进？ | 嫁与否 |
|----|------|-----|-----|-----|
| 帅  | 不好   | 矮   | 不上进 | 不嫁  |
| 不帅 | 好    | 矮   | 上进  | 不嫁  |
| 帅  | 好    | 矮   | 上进  | 嫁   |
| 不帅 | 好    | 高   | 上进  | 嫁   |
| 帅  | 不好   | 矮   | 上进  | 不嫁  |
| 不帅 | 不好   | 矮   | 不上进 | 不嫁  |
| 帅  | 好    | 高   | 不上进 | 嫁   |
| 不帅 | 好    | 高   | 上进  | 嫁   |
| 帅  | 好    | 高   | 上进  | 嫁   |
| 不帅 | 不好   | 高   | 上进  | 嫁   |
| 帅  | 好    | 矮   | 不上进 | 不嫁  |
| 帅  | 好    | 矮   | 不上进 | 不嫁  |

## 朴素贝叶斯算法

### 朴素贝叶斯案例

- 这是一个典型的分类问题
- 数学问题就是比较 $p(\text{嫁} | (\text{不帅、性格不好、身高矮、不上进}))$ 与 $p(\text{不嫁} | (\text{不帅、性格不好、身高矮、不上进}))$

## 朴素贝叶斯算法

### 朴素贝叶斯案例

■联系到我们的朴素贝叶斯公式：

$$p(\text{嫁} | \text{不帅、性格不好、身高矮、不上进}) = \frac{p(\text{不帅、性格不好、身高矮、不上进} | \text{嫁}) * p(\text{嫁})}{p(\text{不帅、性格不好、身高矮、不上进})}$$



## 朴素贝叶斯算法

### 朴素贝叶斯案例

- 求 $p(\text{嫁} | (\text{不帅、性格不好、身高矮、不上进}))$ , 这是我们不知道的, 但是通过朴素贝叶斯公式可以转化为好求的三个量
- $p(\text{不帅、性格不好、身高矮、不上进} | \text{嫁})$ 、 $p(\text{不帅、性格不好、身高矮、不上进})$ 、 $p(\text{嫁})$

## 朴素贝叶斯算法

### 朴素贝叶斯案例

- 其中 $p(\text{不帅、性格不好、身高矮、不上进}|\text{嫁}) = p(\text{不帅}|\text{嫁}) * p(\text{性格不好}|\text{嫁}) * p(\text{身高矮}|\text{嫁}) * p(\text{不上进}|\text{嫁})$
- 那么我就要分别统计后面几个概率，也就得到了左边的概率！

## 朴素贝叶斯算法

### 朴素贝叶斯案例

■我们将上面公式整理一下如下：

$$\begin{aligned} p(\text{嫁} | \text{不帅、性格不好、身高矮、不上进}) &= \frac{p(\text{不帅、性格不好、身高矮、不上进} | \text{嫁}) * p(\text{嫁})}{p(\text{不帅、性格不好、身高矮、不上进})} \\ &= \frac{p(\text{不帅} | \text{嫁}) * p(\text{性格不好} | \text{嫁}) * p(\text{身高矮} | \text{嫁}) * p(\text{不上进} | \text{嫁}) * p(\text{嫁})}{p(\text{不帅}) * p(\text{性格不好}) * p(\text{身高矮}) * p(\text{不上进})} \end{aligned}$$

## 朴素贝叶斯算法

### 朴素贝叶斯案例

- 下面我将一个一个的进行统计计算（在数据量很大的时候，中心极限定理，频率是等于概率的）
- $p(\text{嫁}) = ?$
- 首先我们整理训练数据中，嫁的样本数如下：则  $p(\text{嫁}) = 6/12$ （总样本数）  $= 1/2$

## 朴素贝叶斯算法

### 朴素贝叶斯案例

| 帅？ | 性格好？ | 身高？ | 上进？ | 嫁与否 |
|----|------|-----|-----|-----|
| 帅  | 好    | 矮   | 上进  | 嫁   |
| 不帅 | 好    | 高   | 上进  | 嫁   |
| 帅  | 好    | 高   | 不上进 | 嫁   |
| 不帅 | 好    | 中   | 上进  | 嫁   |
| 帅  | 好    | 中   | 上进  | 嫁   |
| 不帅 | 不好   | 高   | 上进  | 嫁   |

■ 则  $p(\text{嫁}) = 6/12$  (总样本数)  $= 1/2$

## 朴素贝叶斯算法

### 朴素贝叶斯案例

■  $p(\text{不帅}|\text{嫁})=?$  统计满足样本数如下:

| 帅? | 性格好? | 身高? | 上进? | 嫁与否 |
|----|------|-----|-----|-----|
| 不帅 | 好    | 高   | 上进  | 嫁   |
| 不帅 | 好    | 中   | 上进  | 嫁   |
| 不帅 | 不好   | 高   | 上进  | 嫁   |

■ 则  $p(\text{不帅}|\text{嫁}) = 3/6 = 1/2$  在嫁的条件下, 看不帅有多少

## 朴素贝叶斯算法

### 朴素贝叶斯案例

#### ■ 求出其他统计量的概论代入

$$\begin{aligned} p(\text{嫁} | \text{不帅、性格不好、身高矮、不上进}) &= \frac{p(\text{不帅、性格不好、身高矮、不上进} | \text{嫁}) * p(\text{嫁})}{p(\text{不帅、性格不好、身高矮、不上进})} \\ &= \frac{p(\text{不帅} | \text{嫁}) * p(\text{性格不好} | \text{嫁}) * p(\text{身高矮} | \text{嫁}) * p(\text{不上进} | \text{嫁}) * p(\text{嫁})}{p(\text{不帅}) * p(\text{性格不好}) * p(\text{身高矮}) * p(\text{不上进})} \\ &= (1/2 * 1/6 * 1/6 * 1/6 * 1/2) / (1/3 * 1/3 * 7/12 * 1/3) \end{aligned}$$

## 朴素贝叶斯

### 朴素贝叶斯算法

- 思考刚才的案例，我们做了哪些事情
- 1、找到一个已知分类的待分类项集合，这个集合叫做训练样本集



## 朴素贝叶斯

### 朴素贝叶斯算法

- 2、统计得到在各类别下各个特征属性的条件概率估计，  
即

$$P(a_1|y_1), P(a_2|y_1), \dots, P(a_m|y_1); P(a_1|y_2), P(a_2|y_2), \dots, P(a_m|y_2); \dots; P(a_1|y_n), P(a_2|y_n), \dots, P(a_m|y_n)$$

只计算  $P(a_1|y_1)$  ,  $P(a_2|y_1)$  .....的概率而不是联合概率

## 朴素贝叶斯

### 朴素贝叶斯分类器分类

- 3、假设各个特征属性是条件独立的，则根据贝叶斯定理有如下推导：

$$P(x|y_i)P(y_i) = P(a_1|y_i)P(a_2|y_i)\dots P(a_m|y_i)P(y_i) = P(y_i) \prod_{j=1}^m P(a_j|y_i)$$

## 朴素贝叶斯

### 朴素贝叶斯分类器分类

■分母对于所有类别为常数，我们只要将分子最大化皆可。

又因为各特征属性是条件独立的，所以有：

$$P(y_i|x) = \frac{P(x|y_i)P(y_i)}{P(x)}$$

## 朴素贝叶斯

### 朴素贝叶斯分类算法

- 1、设  $x = \{a_1, a_2, a_3, \dots, a_m\}$  为一个待分类项，而每个  $a$  为  $x$  的一个特征属性。
- 2、有类别集合  $C = \{y_1, y_2, \dots, y_n\}$ 。
- 3、计算  $P(y_1|x), P(y_2|x), \dots, P(y_n|x)$ 。
- 4、如果， $P(y_k|x) = \max\{P(y_1|x), P(y_2|x), \dots, P(y_n|x)\}$ ，则  $x$  属于  $y_k$ 。

## 概率模型

### 讨论 $P(a|y)$ 的估计

- 朴素贝叶斯分类的关键：计算条件概率 $P(a|y)$ ，当特征属性为离散值时，使用频率即可用来估计 $P(a|y)$
- 下面重点讨论特征属性是连续值的情况。

## 概率模型

### 讨论 $P(a|y)$ 的估计

- 当特征属性为连续值时，通常假定其值服从高斯分布（也称正态分布）。即：

$$g(x, \eta, \sigma) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x - \eta)^2}{2\sigma^2}}$$

而

$$P(a_k|y_i) = g(a_k, \eta_{y_i}, \sigma_{y_i})$$

## 概率模型

### 讨论 $P(a|y)$ 的估计

- 计算出训练样本各类别中特征项划分的各均值和标准差，即可得到需要的估计值 $P(a|y)$
- 均值与标准差的计算在此不再赘述。

## 概率模型

### 讨论 $P(a|y)=0$ 的估计

- 另一个需要讨论的问题就是当 $P(a|y)=0$ 怎么办？
- 当某个类别下某个特征项划分没有出现时，就是产生这种现象，这会令分类器质量大大降低



## 概率模型

### 讨论 $P(a|y)=0$ 的估计

- 为了解决这个问题，我们引入Laplace校准，它的思想非常简单，就是对没类别下所有划分的计数加1，这样如果训练样本集数量充分大时，并不会对结果产生影响，并且解决了上述频率为0的尴尬局面

## 概率模型

### 高斯和多项式朴素贝叶斯

- 数据集类型的不同，数据的分布也不同
- 什么样的数据适合使用高斯？什么样的数据适合使用多项式贝叶斯？
- 了解多项式分布和高斯分布

## 朴素贝叶斯算法

### 朴素贝叶斯案例

- 加载数据

- `from sklearn import datasets`

- `iris = datasets.load_iris()`

- 导入模型

- `>>> from sklearn.naive_bayes import GaussianNB`

- `>>> gnb = GaussianNB()`

## 朴素贝叶斯算法

### 朴素贝叶斯案例

- 训练模型+预测数据

- `y_pred = gnb.fit(iris.data, iris.target).predict(iris.data)`

- 输出

- `print("Number of mislabeled points out of a total %d points :  
%d"% (iris.data.shape[0],(iris.target != y_pred).sum()))`

## 向量空间模型

### 向量空间模型

- 空间中具有大小和方向的量叫做空间向量
- 我们可以想象我们我们所分析的数据的每一个属性视为一个向量维度，我们输入的数据其实是某个高维向量空间中的一个点

## 向量空间模型

### 向量空间模型

- 很多基于向量空间的分类器在分类决策时用到距离的概念。
- 空间距离计算方法：欧氏距离 (Euclidean distance)

## 向量空间模型

### 向量空间模型

- 在二维和三维空间中的欧式距离的就是两点之间的距离，  
二维的公式是

$$d = \sqrt{(x1-x2)^2 + (y1-y2)^2}$$

## 向量空间模型

### 向量空间模型

■ 三维的公式是

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2 + (z_1 - z_2)^2}$$

■ 推广到n维空间



## 向量空间模型

### 向量空间模型进行分类

- 空间向量如何进行分类?
- 在向量空间的分类中，我们必须要做的工作是定义类别之间的边界，从而得到分类的结果

## KNN算法

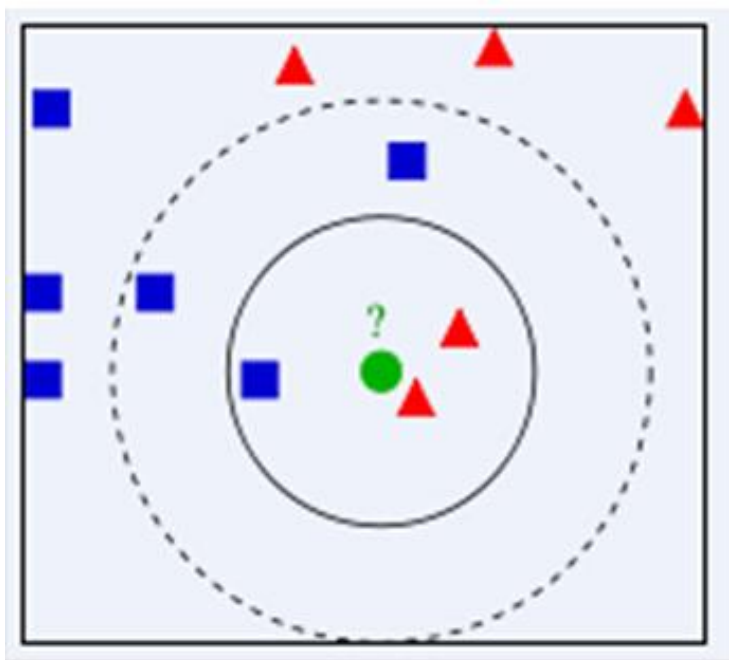
### KNN分类器算法

- K近邻 (K Nearest Neighbors, KNN) 算法, 又称为 KNN算法
- 思想是寻找与待分类的样本在特征空间中距离最近的K个已标记样本 (即K个近邻), 以这些样本的标记为参考, 通过投票等方式, 将占比例最高的类别标记赋给待标记样本

## KNN算法

### KNN分类器

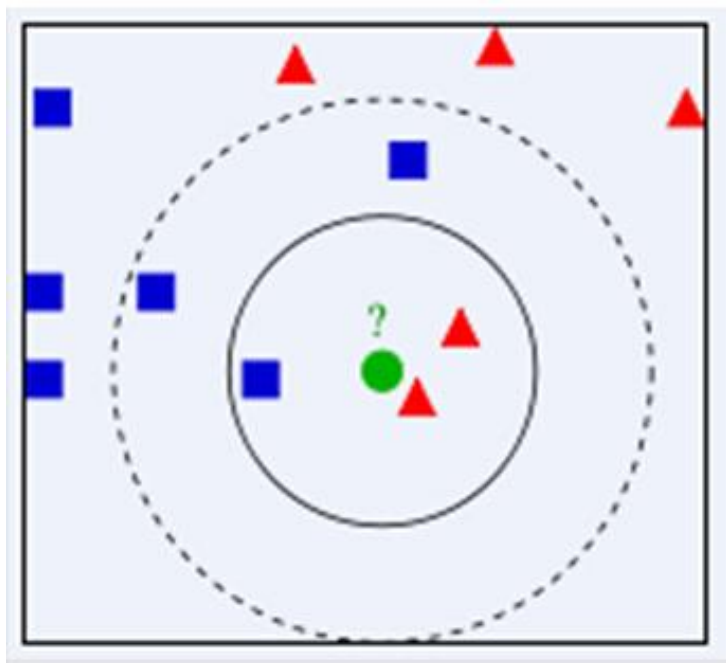
- K面通过一个简单的例子说明一下：如下图，绿色圆要被决定赋予哪个类，是红色三角形还是蓝色四方形？



## KNN算法

### KNN分类器

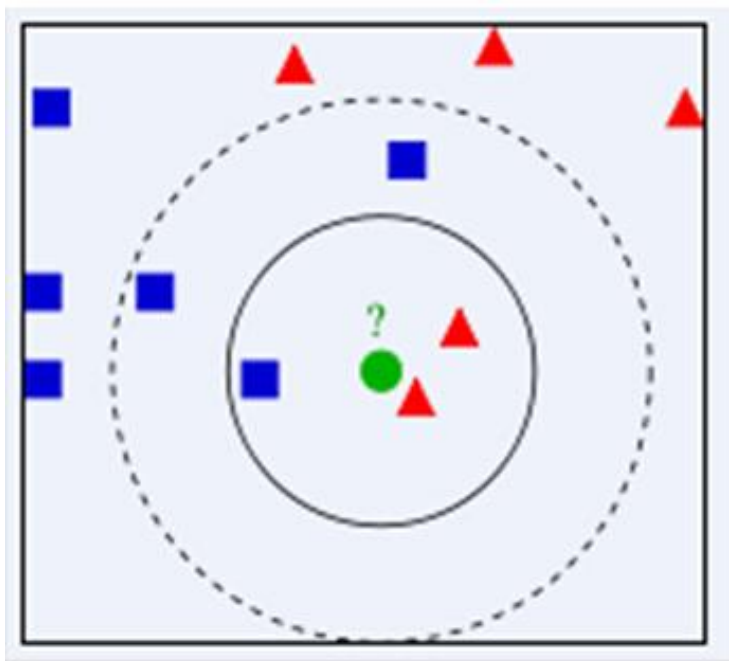
- 如果 $K=3$ ，由于红色三角形所占比例为 $2/3$ ，绿色圆将被赋予红色三角形那类



## KNN算法

### KNN分类器

- 如果 $K=5$ ，蓝色四方形比例为 $3/5$ ，因此绿色圆被赋予蓝色四方形类。



## KNN算法

### KNN分类器算法

- KNN算法需要确定K值、距离度量和分类决策规则
- K值过小时，只有少量的训练样本对预测起作用，容易发生过拟合，或者受含噪声训练数据的干扰导致错误
- K值过大，过多的训练样本对预测起作用，当不同类别样本数量不均衡时，结果偏向数量占优的样本

## KNN算法

### KNN分类器算法

- 距离度量在KNN中，通过计算对象间距离来作为各个对象之间的相似性指标，距离一般使用欧氏距离或曼哈顿距离：

$$\text{欧式距离: } d(x, y) = \sqrt{\sum_{k=1}^n (x_k - y_k)^2}, \quad \text{曼哈顿距离: } d(x, y) = \sqrt{\sum_{k=1}^n |x_k - y_k|}$$

## KNN算法

### KNN分类器算法

- 1) 计算测试数据与各个训练数据之间的距离;
- 2) 按照距离的递增关系进行排序;
- 3) 选取距离最小的K个点;
- 4) 确定前K个点所在类别的出现频率;
- 5) 返回前K个点中出现频率最高的类别作为测试数据的预测分类。



## KNN算法

### KNN案例

- 加载数据

- `from sklearn import datasets`

- `iris = datasets.load_iris()`

- 导入模型

- `>>> from sklearn.neighbors import KNeighborsClassifier`

- `>>> knn = KNeighborsClassifier()`

## ■ 多类问题的分类

### 多类分类和多标签分类

- 多类分类指一个样本属于且只属于多个类中的一个，一个样本只能属于一个类，不同类之间是互斥的
- 多标签分类(multilabel classification)又称，多标签学习、多标记学习，不同于多类分类，一个样本可以属于多个类别（或标签），不同类之间是有关联的。

## 多类问题的分类

### 多类分类

- 直接使用能直接处理多类问题的分类器

## 多类问题的分类

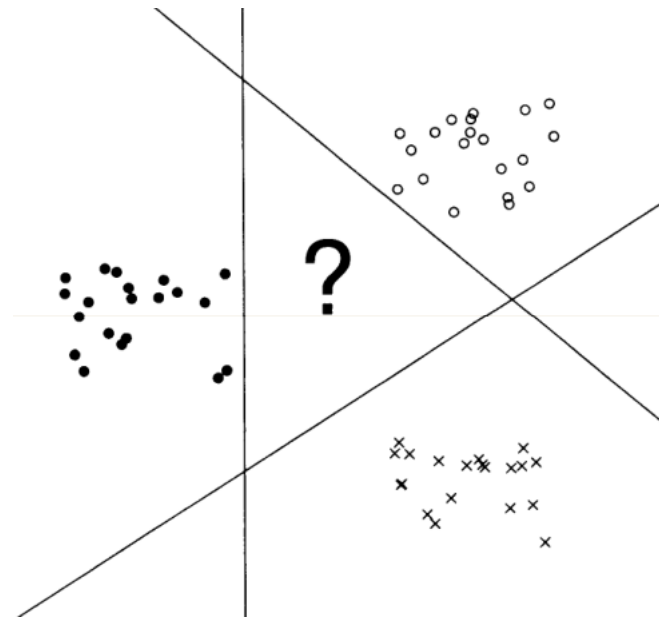
### 多标签分类

- 多标签分类实现，将多标记学习问题转化为“标记排序 (labelranking) 问题求解，既对所有类别排序然后选出排名最高的类

## 多类问题的分类

### 多标签分类器

- 一种方法是，我们可以使用一个直接的置信度指标来对类别排序，比如，类别归属的概率





# 第四章 回归预测

# 目 录

01

回归预测

---

02

线性回归

---

03

逻辑回归

---



## 回归预测



## 回归预测

### 引言

回归分析最早是由19世纪末期高尔顿发展的。1855年，他发表了一篇文章名为“遗传的身高向平均数方向的回归”，分析父母与其孩子之间身高的关系，发现父母的身高越高的其孩子也越高，反之则越矮。他把孩子跟父母身高这种现象拟合成一种线性关系

## 回归预测

### 引言

但是他还发现个有趣的现象，高个子的人生出来的儿子往往比他父亲矮一点更趋于父母的平均身高，矮个子的人生出来的儿子通常比他父亲高一点也趋向于平均身高。高尔顿选用了“回归”一词，把这一现象叫做“向平均数方向的回归”

## 回归预测

### 回归分析

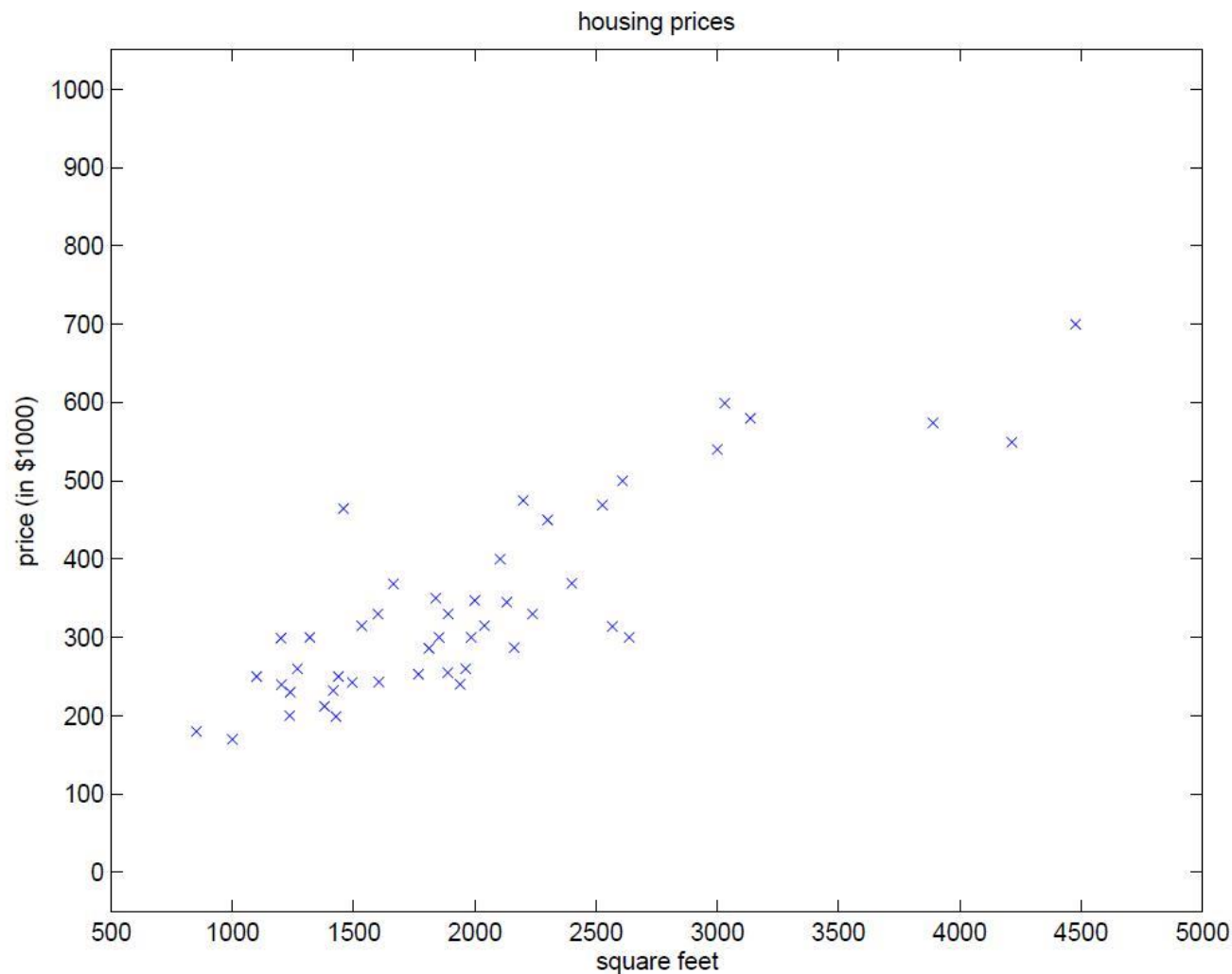
■ 回归分析(regression analysis)是确定两种或两种以上变量间相互依赖的定量关系的一种统计分析方法



## 回归预测

### 回归分析案例

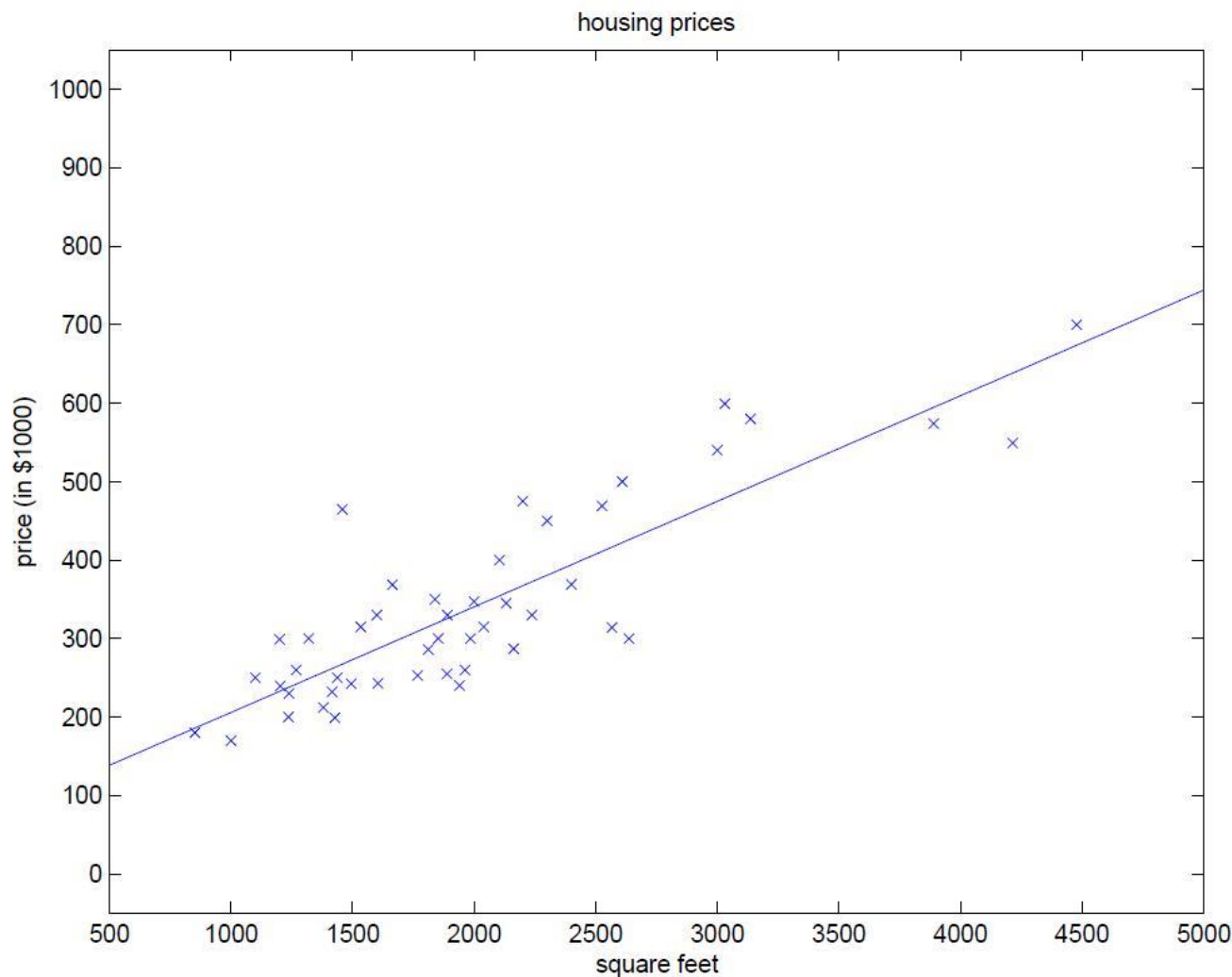
图为某地区的房屋面积(feet)与价格(\$)的一个数据集，在该数据集中，只有一个自变量面积(feet)，和一个因变量价格(\$)



## 回归预测

### 回归分析案例

利用该数据集，我们可以训练一个线性方程，无限逼近所有数据点，然后利用该方程与给定的某一自变量（本例中为面积），可以预测因变量（本例中为房价）



## 回归预测

### 其它应用

- 预测二手车价格的系统。该系统的输入是我们认为会影响车价的属性信息：品牌、年龄、发动机性能、里程以及其他信息。输出是车的价格。这种输出为数值的问题是回归问题

## 回归预测

### 其它应用

- 想要造一个焙炒咖啡的机器，影响咖啡品质的输入分别有温度、时间、咖啡豆种类等配置。我们针对不同的输入配置进行大量实验，并检测咖啡的品质，例如，根据消费者的满意度测量咖啡的品质(输出)



## 线性回归



## 线性回归

### 线性回归

- 线性回归(Linear Regression): 因变量 (Y) 和一个或多个自变量 (X) 之间建立一种线性方程关系
- Linear Regression线性回归它是最为人熟知的建模技术之一, 线性回归通常是人们在学习预测模型时首选的技术之一
- 在这种技术中, 因变量是连续的, 自变量可以是连续的也可以是离散的

## 线性回归

### 一元线性回归介绍

■一元线性回归包含一个自变量(X)和一个因变量(Y)

■一元线性回归方程：

$$E(y) = \beta_0 + \beta_1 x$$

■这个方程对应的图像是一条直线，称作回归线，其中， $\beta_0$ 是回归线的截距， $\beta_1$ 是回归线的斜率， $E(y)$ 是在一个给定x值下y的期望值（均值）

## 线性回归

### 一元线性回归举例

汽车卖家做电视广告数量  
与卖出的汽车数量，如图  
所示，来预测广告数量为  
“6” 时，卖出的汽车数量？

| <u>Number of<br/>TV Ads (<math>x</math>)</u> | <u>Number of<br/>Cars Sold (<math>y</math>)</u> |
|----------------------------------------------|-------------------------------------------------|
| 1                                            | 14                                              |
| 3                                            | 24                                              |
| 2                                            | 18                                              |
| 1                                            | 17                                              |
| 3                                            | 27                                              |

## 线性回归

### 一元线性回归举例

- 如何找出适合一元线性回归模型的最佳回归线？
- 介绍一种拟合方法：最小二乘法
- 最小二乘法，出发点是使实际测量数据 $y_i$ 与拟合直线上对应的估计值的差的平方和为最小，即：

$$\min \sum (y_i - \hat{y}_i)^2$$

## 线性回归

### 最小二乘法

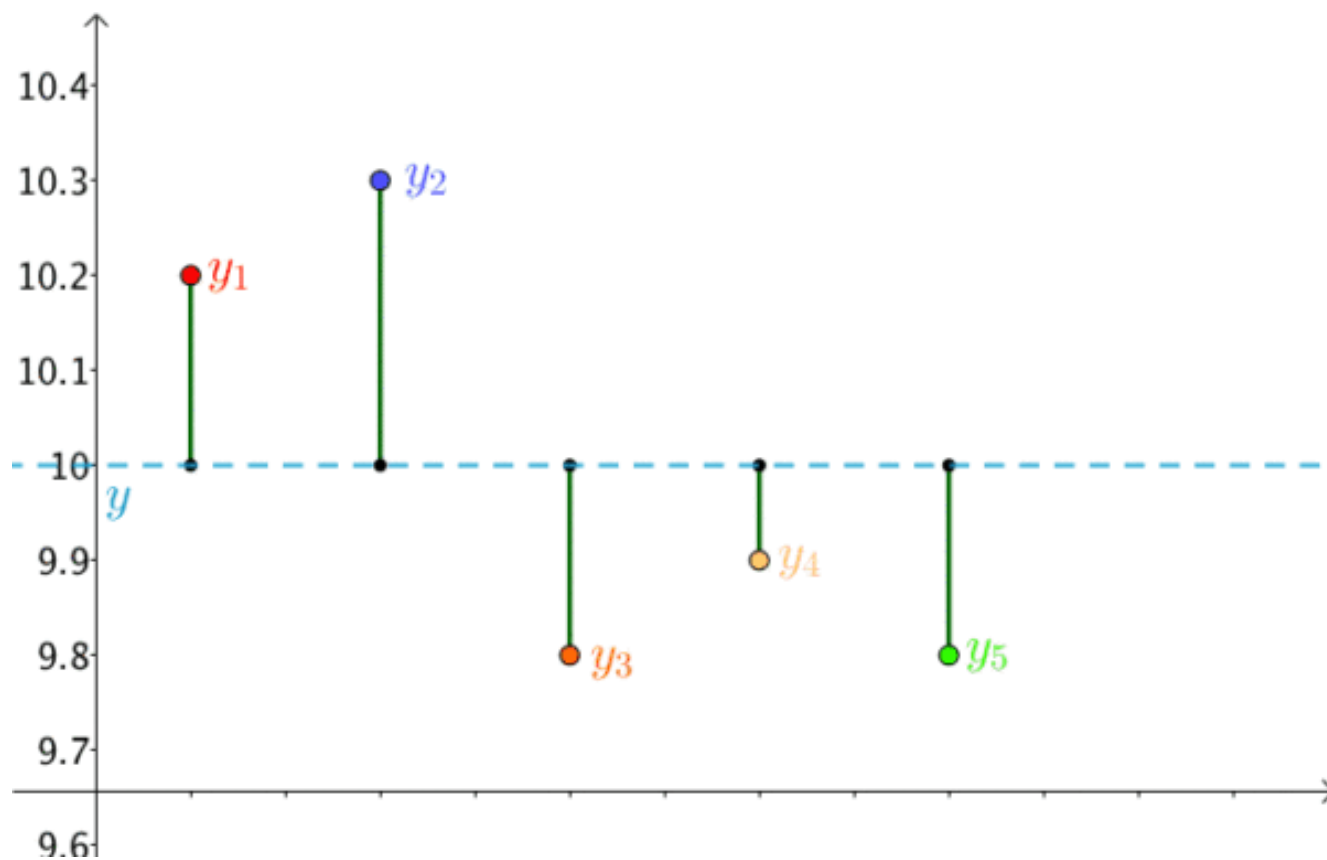
- 法国数学家，阿德里安-马里·勒让德（1752 - 1833）提出让总的误差的平方最小的 $y$  就是真值，这是基于，如果误差是随机的，应该围绕真值上下波动



知乎 @马同学

## 线性回归

### 最小二乘法



## 线性回归

### 一元线性回归举例

汽车卖家做电视广告数量  
与卖出的汽车数量，如图  
所示，来预测广告数量为  
“6” 时，卖出的汽车数量？

| <u>Number of<br/>TV Ads (<math>x</math>)</u> | <u>Number of<br/>Cars Sold (<math>y</math>)</u> |
|----------------------------------------------|-------------------------------------------------|
| 1                                            | 14                                              |
| 3                                            | 24                                              |
| 2                                            | 18                                              |
| 1                                            | 17                                              |
| 3                                            | 27                                              |

## 线性回归

### 一元线性回归举例

■ 假设我们的线性方程为：  $f(x) = ax + b$

■ 通过最小二乘法思想，总误差为：

$$S_{e^2} = \sum (f(x_i) - y_i)^2 = \sum (ax_i + b - y_i)^2$$

■ 根据多元微积分的知识

$$\begin{cases} \frac{\partial}{\partial a} S_{e^2} = 2 \sum (ax_i + b - y_i)x_i = 0 \\ \frac{\partial}{\partial b} S_{e^2} = 2 \sum (ax_i + b - y_i) = 0 \end{cases}$$



## 线性回归

### 一元线性回归算法实现

```
■ >>> from sklearn import datasets
```

```
■ >>> from sklearn.linear_model import LinearRegression
```

```
■ >>> data = datasets.load_boston()
```

```
■ >>> linear_model = LinearRegression()
```

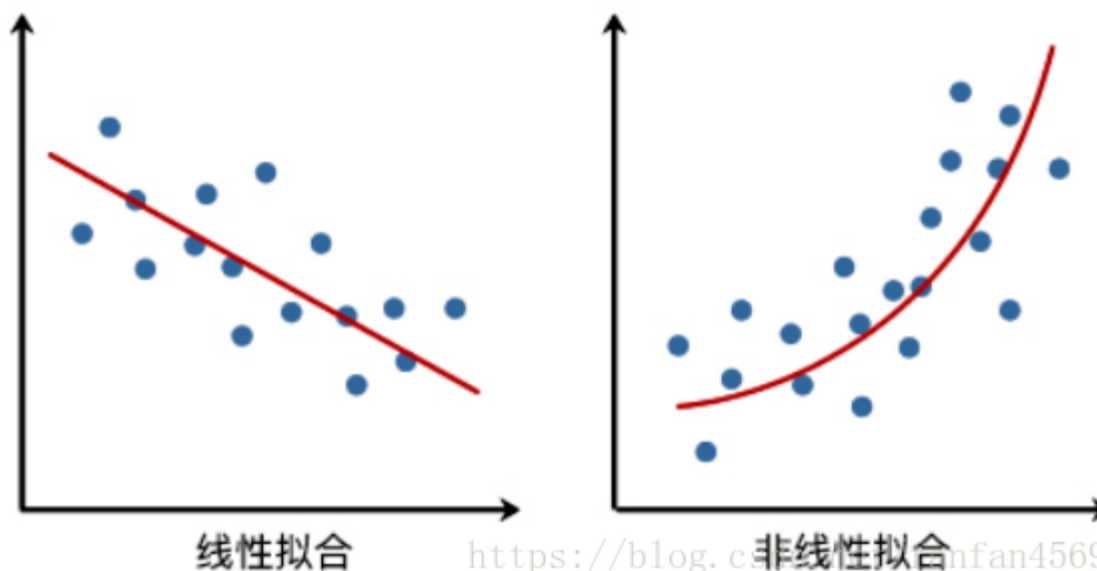
```
■ >>> linear_model.fit(data.data,data.target)
```

```
■ >>> liner_model. coef_
```

## 线性回归

### 多项式回归介绍

- 在线性回归中，我们通过建立自变量  $x$  的一次方程来拟合数据
- 而非线性回归中，则需要建立因变量和自变量之间的非线性关系



## 线性回归

### 多项式回归介绍

- 机器学习中一种常见的模式，是使用线性模型训练数据的非线性函数。这种方法保持了一般快速的线性方法的性能，同时允许它们适应更广泛的数据范围
- 在标准线性回归的情况下，你可能有一个类似于二维数据的模型：

$$\hat{y}(w, x) = w_0 + w_1 x_1 + w_2 x_2$$

## 线性回归

### 多项式回归

- 如果我们想把抛物面拟合成数据而不是平面，我们可以结合二阶多项式的特征，使模型看起来像这样：

$$\hat{y}(w, x) = w_0 + w_1x_1 + w_2x_2 + w_3x_1x_2 + w_4x_1^2 + w_5x_2^2$$

- 仔细观察这两个模型会发现，我们可以想象创造一个新的变量：

$$z = [x_1, x_2, x_1x_2, x_1^2, x_2^2]$$

## 线性回归

### 多项式回归介绍

- 有了这些重新标记的数据，我们可以将原多项式问题写成

$$\hat{y}(w, x) = w_0 + w_1 z_1 + w_2 z_2 + w_3 z_3 + w_4 z_4 + w_5 z_5$$

- 因此我们可以将原始数据转换后再使用线性回归训练构成多项式回归模型,其中PolynomialFeatures方法将数据进行转换

## 线性回归

### PolynomialFeatures方法实现

```
■ >>> from sklearn.preprocessing import  
PolynomialFeatures
```

```
■ >>> model = PolynomialFeatures(degree=2)
```

```
■ >>> data = datasets.load_boston()
```

```
■ >>> x_transformed=model.fit_transform(x_train)
```

## 线性回归

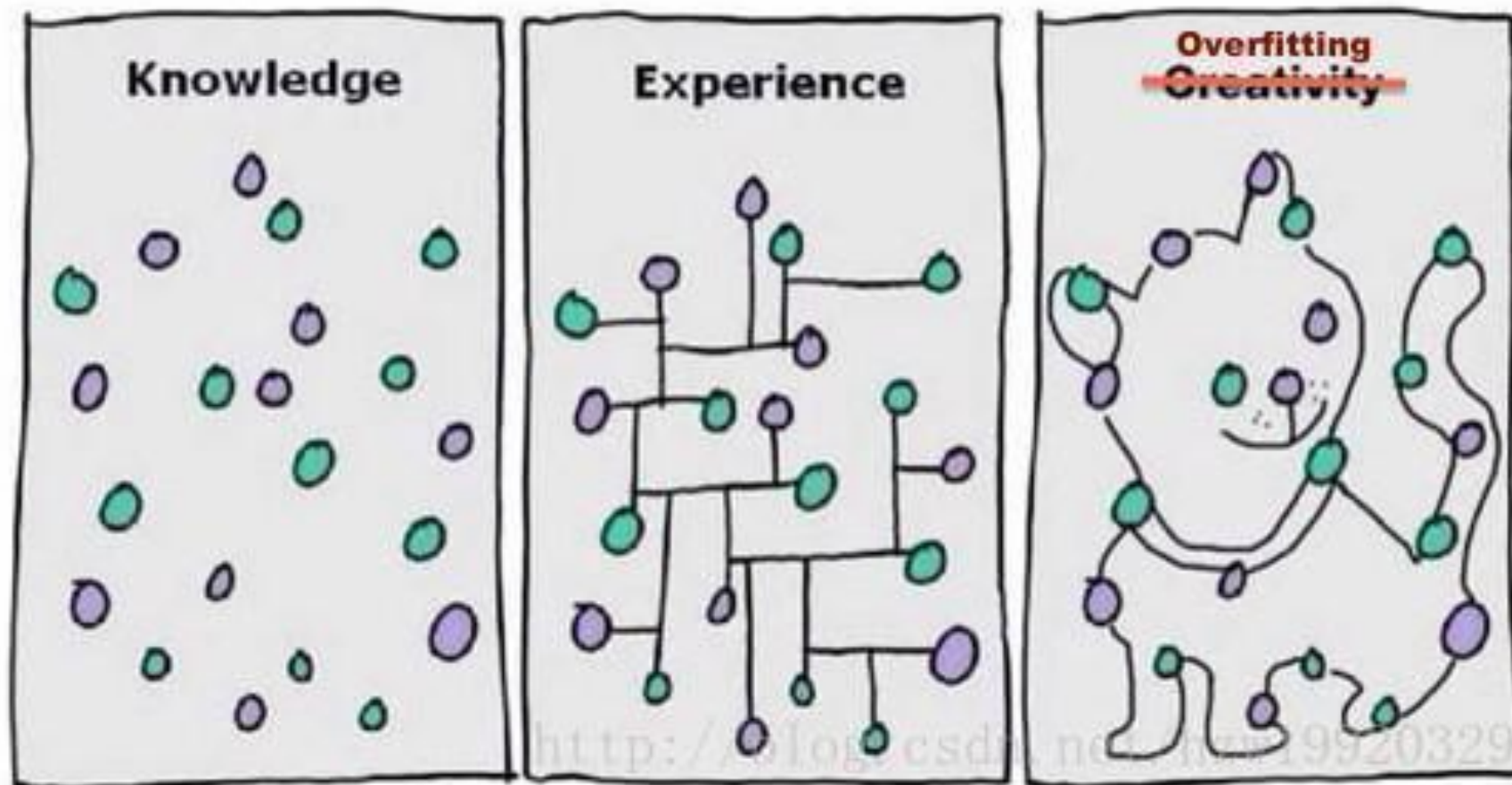
### 线性回归问题

- 过拟合：过拟合指的是在训练集上模型表现很好，但是在测试集上效果很差，即模型的泛化能力不行。过拟合是模型训练过程中参数拟合的问题，由于训练数据本身有采样误差，拟合模型参数时这些采样误差都拟合进去带来的问题

## 线性回归

### 线性回归问题

■ 过拟合:





## 线性回归

### Ridge回归和Lasso回归

- Ridge回归与Lasso回归的出现是为了解决线性回归出现的过拟合
- 线性回归的损失函数：

$$MSE = \min \sum (y_i - f(x_i))^2$$

## 线性回归

### Ridge回归和Lasso回归

■ Ridge回归的损失函数(L2范数惩罚项):

$$MSE + \alpha \sum_{i=1}^i x_i^2$$

■ Lasso回归的损失函数(L1范数惩罚项):

$$MSE + \alpha \sum_{i=1}^i |x_i|$$

## 线性回归

### Ridge回归和Lasso回归

- 为了防止参数 $x_i$ 过大，我们可以在目标函数后面加上约束项
- L1会趋向于产生少量的特征，而其他的特征都是0，而L2会选择更多的特征，这些特征都会接近于0
- 范式中，alpha是控制系数收缩量的复杂性参数：alpha的值越大，收缩量越大，这样系数对共线性的鲁棒性也更强

## 线性回归

### Ridge回归和Lasso回归实现

```
■ >>> from sklearn import linear_model
■ >>> reg = linear_model.Ridge (alpha = .5)
■ >>> reg.fit ([[0, 0], [0, 0], [1, 1]], [0, .1, 1])
■ >>> reg.coef_
```

## 线性回归

### 寻找Ridge最优参数alpha

- 参数优化器GridSearchCV，将参数model，param\_grid传入，GridSearchCV是为了寻找出model的alpha最优参数

- 参考：

[http://sklearn.apachecn.org/cn/0.19.0/modules/generated/sklearn.model\\_selection.GridSearchCV.html#sklearn.model\\_selection.GridSearchCV](http://sklearn.apachecn.org/cn/0.19.0/modules/generated/sklearn.model_selection.GridSearchCV.html#sklearn.model_selection.GridSearchCV)

## 逻辑回归

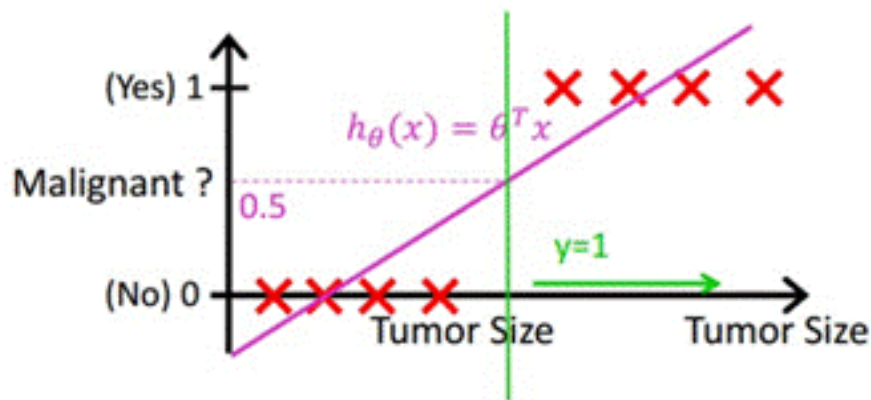
### 逻辑回归

- 逻辑回归算法是用来解决分类问题的
- 回归与分类的区别在于：
- 回归所预测的目标量的取值是连续的（例如房屋的价格）
- 而分类所预测的目标变量的取值是离散的（例如判断肿瘤大小是否为恶性）

## 逻辑回归

### 逻辑回归

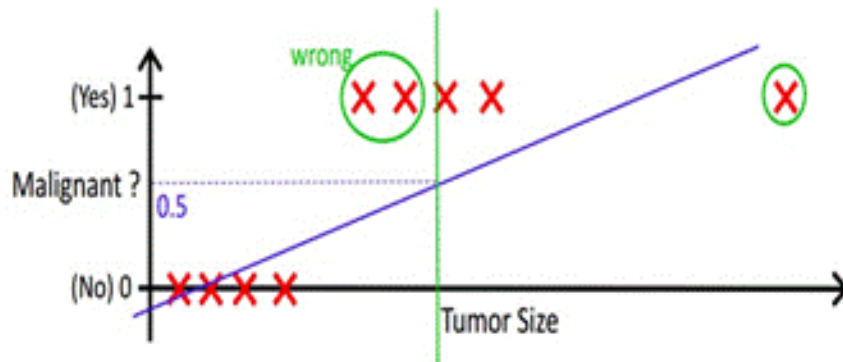
如图所示，X为数据点-肿瘤的大小，Y为观测值-是否为恶性肿瘤（0良性肿瘤，1恶性肿瘤）。通过构建线性回归模型，即可根据肿瘤大小，预测是否为恶性肿瘤， $h_{\theta}(x) \geq 0.5$ 为恶性， $h_{\theta}(x) < 0.5$ 为良性。



## 逻辑回归

### 逻辑回归

然而线性回归的鲁棒性(稳定和可靠)很差，如下图所示，因最右边噪点的存在，使回归模型在训练上表现很差，这主要是由于线性回归在整个实数域内敏感度一致，而分类范围，需要在 $[0,1]$ 。





## 逻辑回归

### 逻辑回归

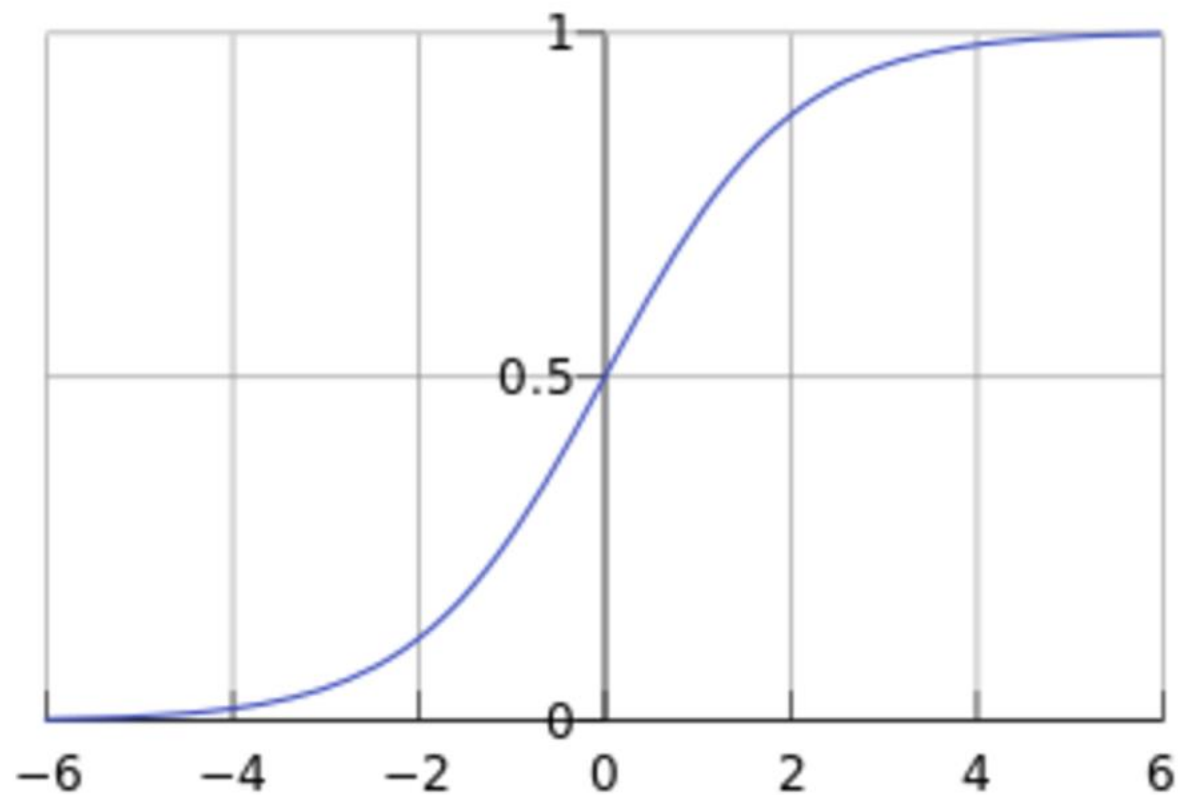
逻辑回归就是一种减小预测范围，将预测值限定为[0,1]间的一种回归模型，此时引入了一个sigmoid函数，这个函数的性质，非常好的满足了，函数的定义域x的输入是全体实数，而值域输出y总是[0,1]，以一种概率的形式表示. 并且当x=0的时，y=0.5，这是决策边界。

$$f(x) = \frac{1}{1 + e^{-x}}$$

## 逻辑回归

### Sigmoid 函数

■ Sigmoid 函数有个很漂亮的 “S” 形，如下图所示：



## 逻辑回归

### 逻辑回归

■对于线性回归的情况，方程如下：

$$\theta_0 + \theta_1 x_1 + \dots + \theta_n x_n = \sum_{i=1}^n \theta_i x_i = \theta^T x$$

■构造预测函数为：取值在[0,1]

$$h_{\theta}(x) = g(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}} = \frac{1}{1 + e^{-(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n)}}$$

## 逻辑回归

### 逻辑回归实现

```
■ >>> from sklearn.linear_model import LogisticRegression
■ >>> clf = LogisticRegression()
■ >>> clf.fit(train_feature,label)
■ >>> clf.predict(predict_feature)
```

## 逻辑回归

### 逻辑回归优点

- 逻辑回归的算法已经比较成熟，预测较为准确
- 模型求出的系数易于理解，便于解释，尤其在银行业，80%的预测是使用逻辑回归
- 结果是概率值
- 训练快。

## 逻辑回归

### 逻辑回归的不足

- 当然它也有缺点，分类较多的y都不是很适用；对于自变量的多重共线性比较敏感，所以需要利用因子分析或聚类分析来选择代表性的自变量；另外预测结果呈现S型，两端概率变化小，中间概率变化大比较敏感，导致很多区间的变量的变化对目标概率的影响没有区分度，无法确定阈值。



# 第五章 无监督学习

# 目 录

01

概述

---

02

聚类中的数据结构

---

03

基于划分的聚类方法

---

04

层次聚类方法

---





# 目 录



05

密度聚类方法

---

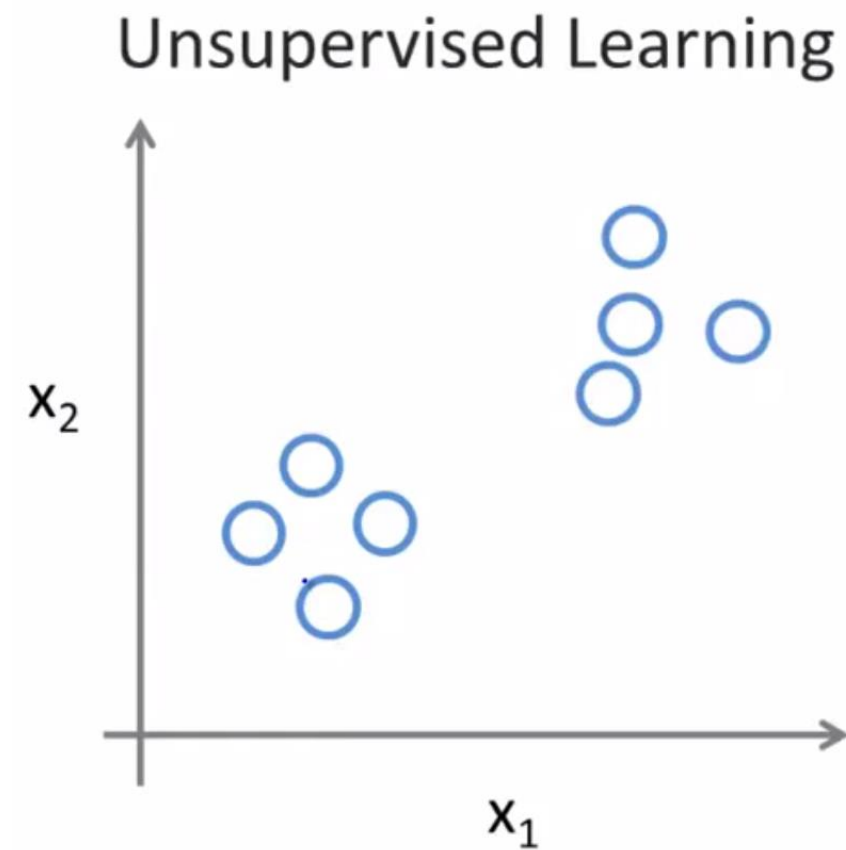
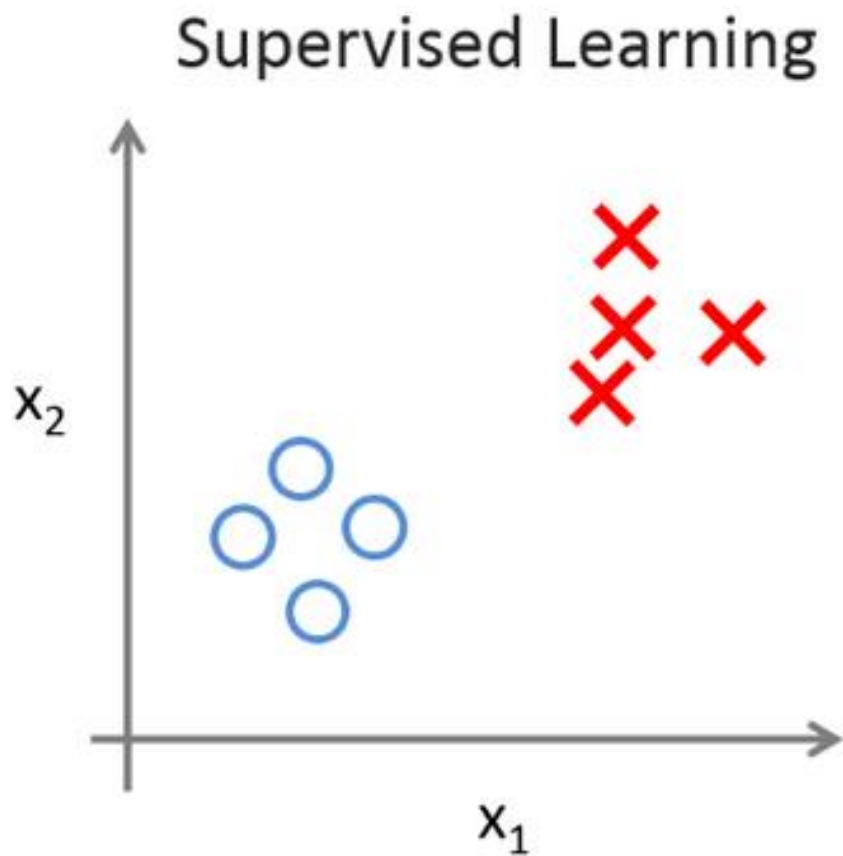
## 机器学习算法

### 无监督学习

- 无监督学习算法-在非监督学习中，我们将让计算机自己学习
- 对于监督学习，我们回想一下每个样本都已经被标明，既我们已经被告知了明确的答案
- 与监督学习不一样的数据类型
- 没有属性或标签这一概念

## 机器学习算法

### 无监督学习的数据



## 引言

### 聚类 (clustering) 的概念

- 聚类分析，将数据对象分组成为多个簇
- 同一簇中的对象彼此相似
- 不同簇中的对象彼此相异

## 引言

### 聚类算法的功能

- 作为一个独立的工具来获得数据分布的情况
- 作为其他算法（如：特征和分类）的预处理步骤

## 引言

### 聚类算法的应用场景

- 市场营销：帮市场分析人员从客户基本库中发现不同的客户群，从而可以对不同的客户群采用不同的营销策略
- 保险业：发现汽车保险中索赔率较高的客户群
- 城市规划：根据房子的类型、价值和地理位置对其进行分组
- 地震研究：将观测到的震中点沿板块断裂带进行聚类，得出地震高危区

## 引言

### 聚类算法的评判标准

- 可扩展性：大多数来自于机器学习和统计学领域的聚类算法在处理数百万条数据时能表现出高效率
- 处理不同数据类型的能力：数字型、二元类型、图像型等等
- 发现任意形状的能力：基于距离的聚类算法往往发现的是球形的聚类，其实现实的聚类是任意形状的

## 引言

### 聚类算法的评判标准

- 处理噪声数据的能力：对空缺值、离群点、数据噪声不敏感
- 对于输入数据的顺序不敏感：同一个数据集合，以不同的次序提交给同一个算法，应该产生相似的结果



## 引言

### 聚类算法的数据问题

- 已经了解了无监督学习的有监督学习的数据类型不同
- 聚类算法的数据是什么样子的呢？

## 引言

### 聚类算法的数据结构

- 聚类算法采用以下两种数据结构常见的数据矩阵：行代表不同的数据，列为数据的属性
- 相异度矩阵：存储n个对象两两之间的相似度

$$\begin{bmatrix} x_{11} & \cdots & x_{1f} & \cdots & x_{1p} \\ \cdots & \cdots & \cdots & \cdots & \cdots \\ x_{i1} & \cdots & x_{if} & \cdots & x_{ip} \\ \cdots & \cdots & \cdots & \cdots & \cdots \\ x_{n1} & \cdots & x_{nf} & \cdots & x_{np} \end{bmatrix}$$

$$\begin{bmatrix} 0 & & & & \\ d(2,1) & 0 & & & \\ d(3,1) & d(3,2) & 0 & & \\ \vdots & \vdots & \vdots & & \\ d(n,1) & d(n,2) & \cdots & \cdots & 0 \end{bmatrix}$$

## 聚类数据结构

### 聚类算法的距离计算

- 对象间的相似度是基于两个对象间的 $1/\text{距离}$ 来计算的
- 欧氏距离
- Manhattan距离

## 基于划分的聚类

### 主要聚类方式

- 基于划分的方法
- 基于层次的方法
- 基于密度的方法

## 基于划分的聚类

### 基于划分的方法

- 划分方法(Partitioning method): 将数据集中的数据对象分配到 $n$ 个簇中, 并且通过设定目标函数来驱使算法趋向于目标
- 每个组至少包含一个对象
- 每个对象必须属于且只属于一个组

## 基于划分的聚类

### k-means聚类方式

- K-Means (K均值) 算法是划分聚类算法中的一种，其中K表示簇的数量，means表示均值。算法通过预先设定的 K值及每个簇的初始质心对相似的数据点进行划分。并通过划分后的均值迭代优化获得最优的聚类结果

## 基于划分的聚类

### k-means聚类方式

- K-Means算法的伪代码如下：
- 1.从D中任意选择k个对象作为初始簇的质心
- 2.Repeat
- 3. 根据簇质心的值，将每个数据分配到最相似的簇
- 4. 计算簇均值，更新簇质心值
- 5.until簇中心不在发生变化

## 基于划分的聚类

### k-means聚类方式

- 关键词：K表示簇的数量，means表示均值，初始质心，划分，迭代
- 质点系质量分布的平均位置，是最能代表一个簇的点
- 算法的目标是什么？质心为什么需要改变？



## 基于划分的聚类

### 质心对样本的代表函数

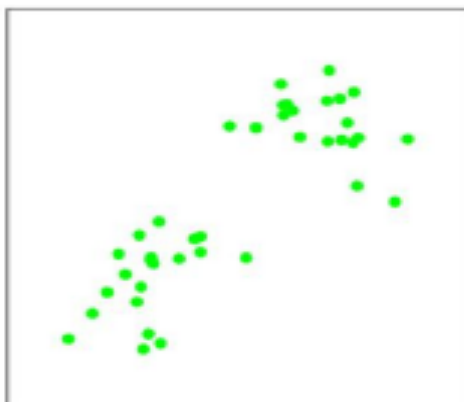
- 一个衡量质心对簇样本的代表程度的指标是 RSS（Residual Sum of Squares，残差平方和），即所有数据到其簇质心距离的平方和，公式为：

$$\text{RSS}_k = \sum_{x \in \omega_k} |\vec{x} - \vec{\mu}(\omega_k)|^2。$$

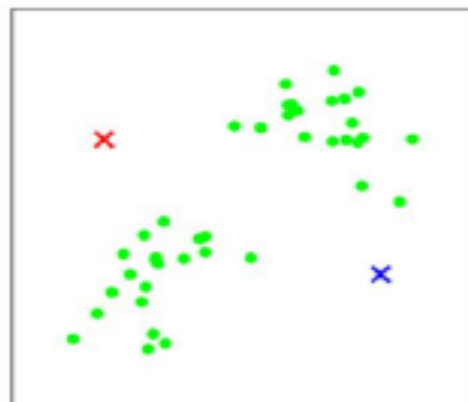
$$\text{RSS} = \sum_{k=1}^K \text{RSS}_k。$$

## 基于划分的聚类

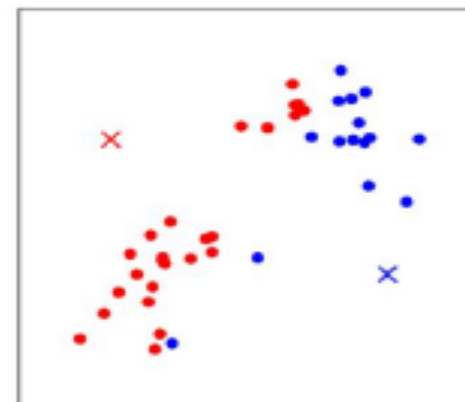
### k-means聚类执行过程



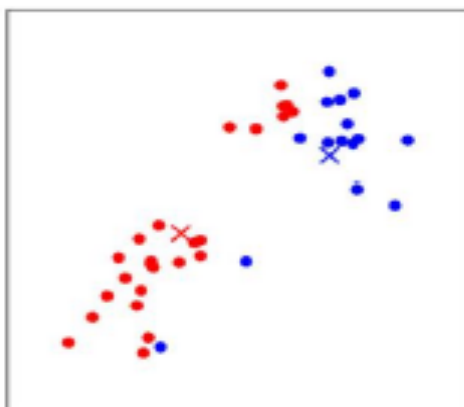
(a)



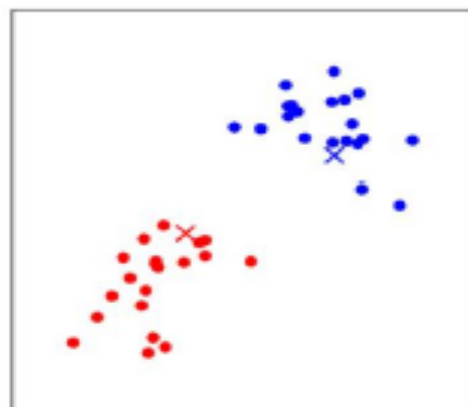
(b)



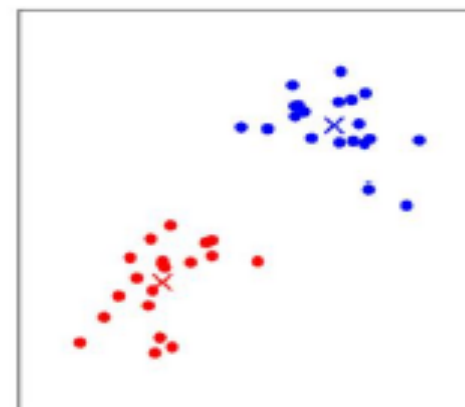
(c)



(d)



(e)



(f)

## 基于划分的聚类

### k-means聚类方式

#### ■ 一个动态网站:

<https://www.naftaliharris.com/blog/visualizing-k-means-clustering/>

## 基于划分的聚类

### **k-means聚类方式**

- 算法会得到我们想要的结果吗？
- 质心会趋于稳定吗？

## 基于划分的聚类

### k-means的结束

- 第一，在每次数据的重分配过程中，每个向量都分配给最近的质心，此时它对 RSS 计算的贡献值也会减小，因此 RSS 会单调减少
- 第二，在簇中心重计算过程中，由于新的质心向量使得每个 RSS 达到最小值
- 因此，此时 RSS 也会单调递减

## 基于划分的聚类

### k-means的结束

- 当迭代一个固定次数后停止
- 当RSS低于某个阈值低于某个阈值时停止
- 当质心不再改变后停止

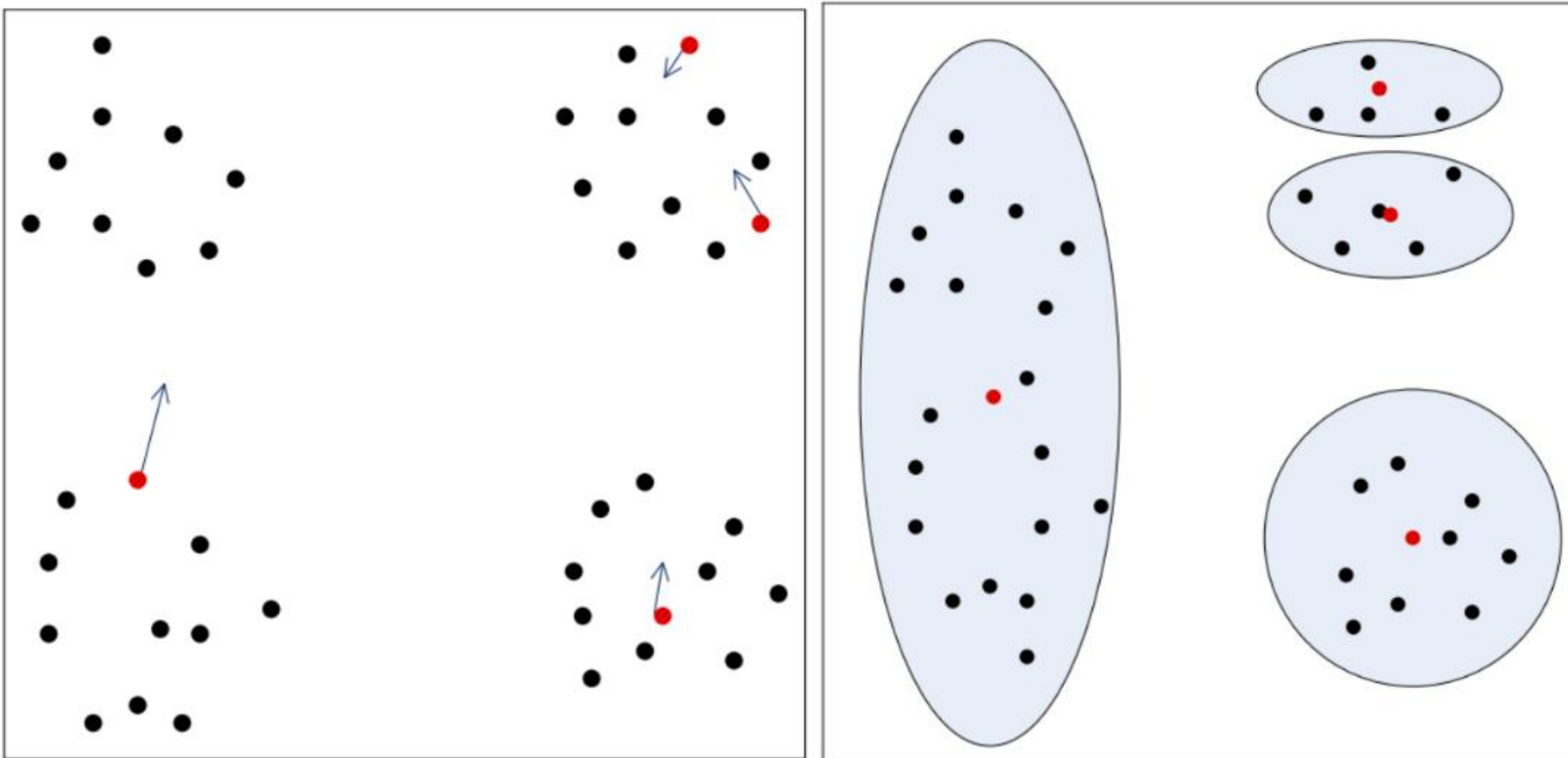
## 基于划分的聚类

### k-means的问题

- k-means算法是一个比较容易理解的算法
- 但是却也存在以下问题

## 基于划分的聚类

### 初始值对k-means聚类影响





## 基于划分的聚类

### k-means初始值选择

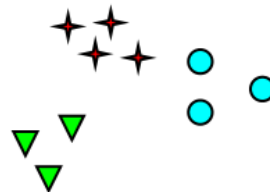
- 将全部数据随机地分为C簇，计算各个簇的质心，将这些质心作为每簇的初始点
- 从(c-1)聚类划分问题的解中产生 C 聚类划分问题的代表点。其具体做法是先从一类聚类的解找两聚类划分的代表点，再依次增加一个聚类代表点。对样本集首先看作一个聚类，计算其总均值，然后找与该均值相距最远的点

## 基于划分的聚类

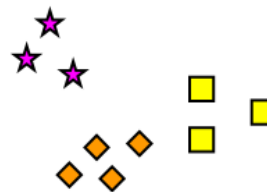
### k值对k-means聚类影响



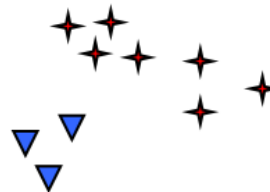
How many clusters?



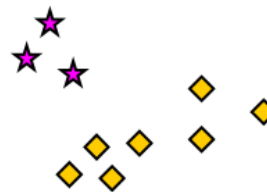
Six Clusters



Two Clusters



Four Clusters



## 基于划分的聚类

### **k-means算法k值选择**

- 依据实际业务来选择
- 从层次聚类中选择参考

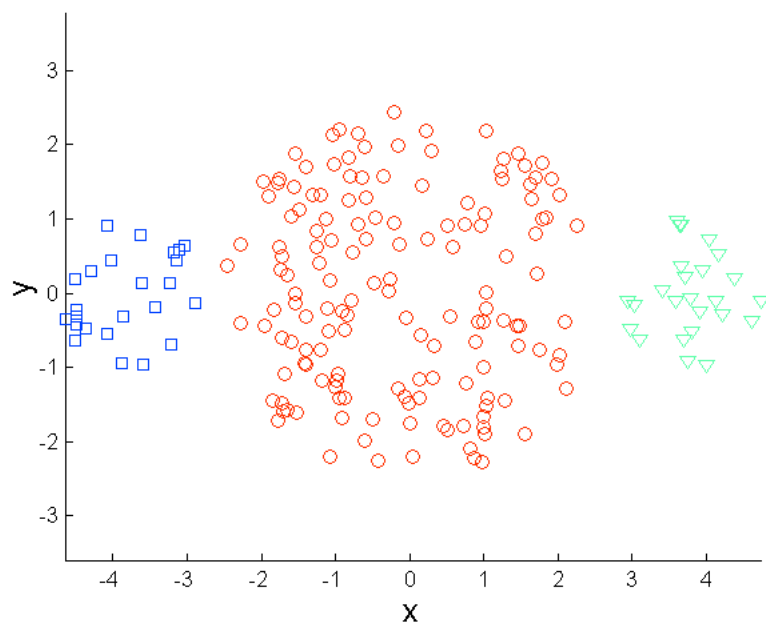
## K均值算法

### K均值算法运行注意问题

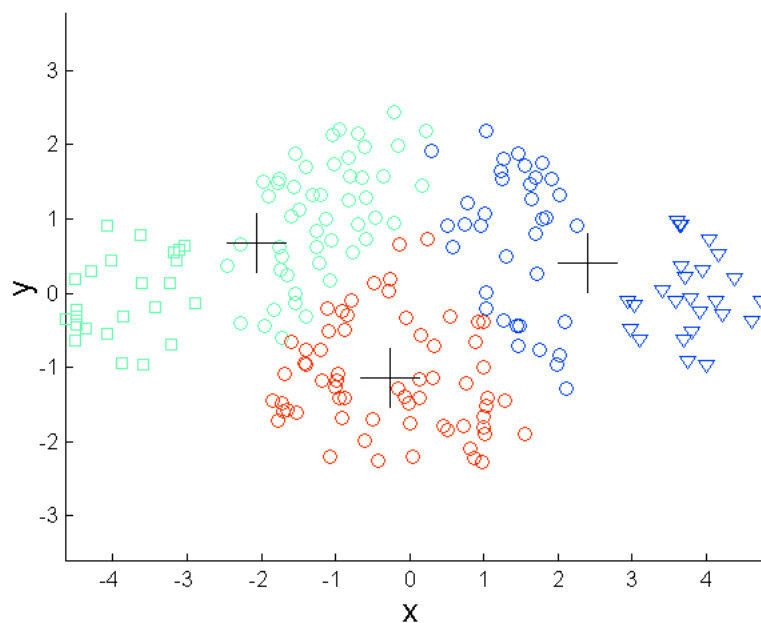
- 数据集如果有以下不同的簇类型，kmeans效果会不佳：
  - 簇大小相差大
  - 簇密度相差大
  - 非球型形状
- 数据如果存在离群点，kmeans会很受影响，因为使用算术平均来选取类中心

## K均值算法

### 簇大小相差大



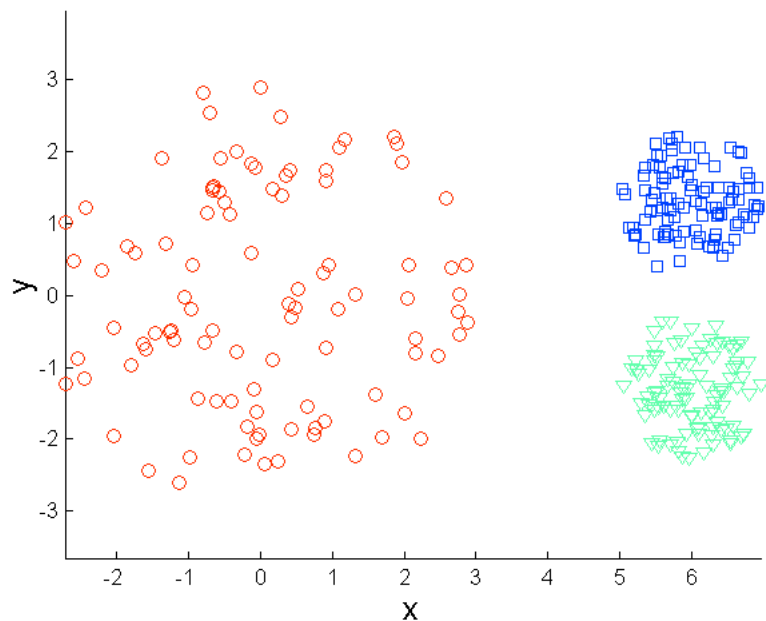
原始



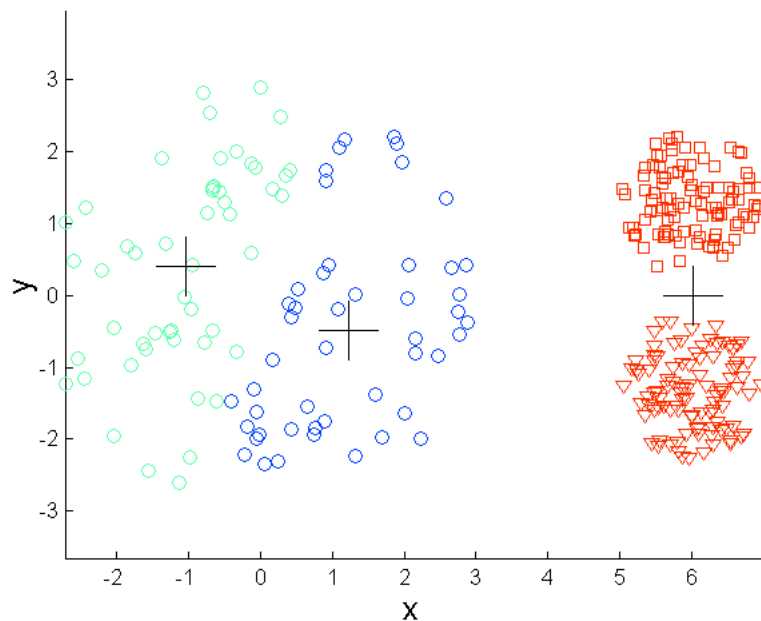
kmeans聚3类

## K均值算法

### 簇密度相差大



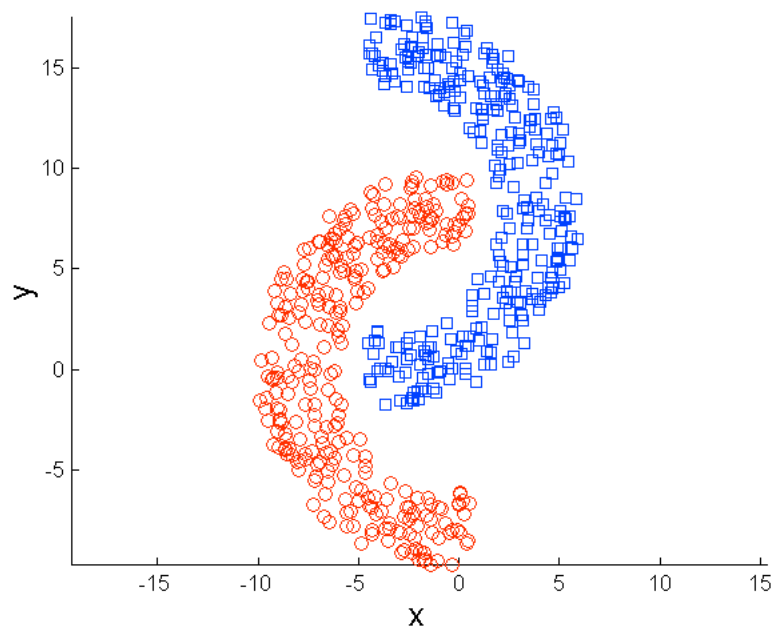
原始



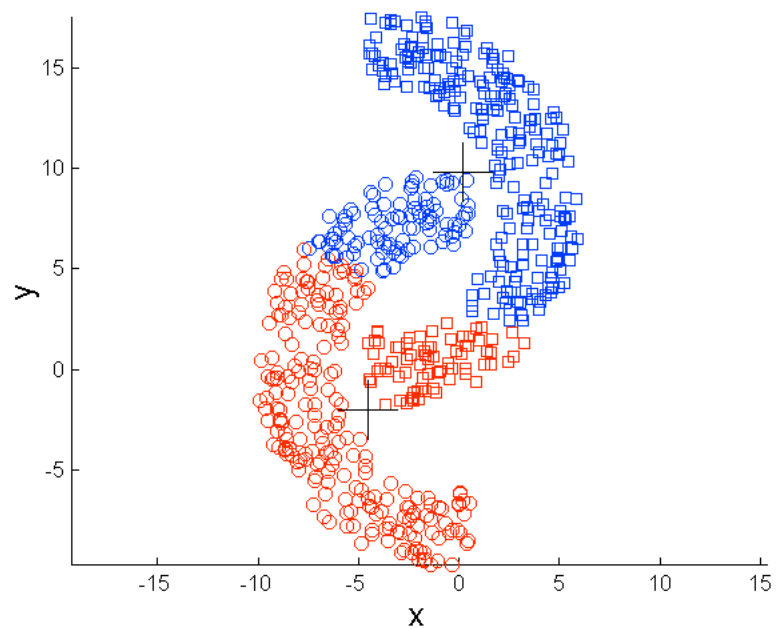
kmeans聚3类

## K均值算法

### 非球型形状



原始



kmeans聚2类

## K均值算法

### sklearn调用演示

```
>>> from sklearn.cluster import KMeans
>>> import numpy as np
>>> X = np.array([[1, 2], [1, 4], [1, 0],
...               [4, 2], [4, 4], [4, 0]])
>>> kmeans =
KMeans(n_clusters=2,random_state=0).fit(X)
>>> kmeans.labels
>>> kmeans.predict([[0, 0], [4, 4]])
>>> kmeans.cluster_centers_
```



## 层次聚类

### 引入层次聚类

- 基于划分聚类具有概念简单、速度快的优点，但是同时也有很多缺点：它们需要预先定义簇的数目

## 层次聚类

### 层次聚类

- 层次聚类 (hierarchical clustering) 是指对与给定的数据集对象，我们通过层次聚类算法获得一个具有层次结构的数据集合子集结合的过程
- 层次聚类分为两种：自底向上的凝聚法以及自顶向下的分裂法

## 层次聚类

### 凝聚层次聚类

- 凝聚法指的是初始时将每个样本点当做一个簇，所以原始簇的数量等于样本点的个数，然后依据某种准则合并这些初始的簇，直到达到某种条件或者达到设定的簇的数目
- 某种准则可以是相似度

## 层次聚类

### 分裂层次聚类

- 分裂法指的是初始时将所有的样本归为一个簇，然后依据某种准则进行逐渐的分裂，直到达到某种条件或者达到设定的簇的数目

## 层次聚类

### 凝聚层次聚类伪代码

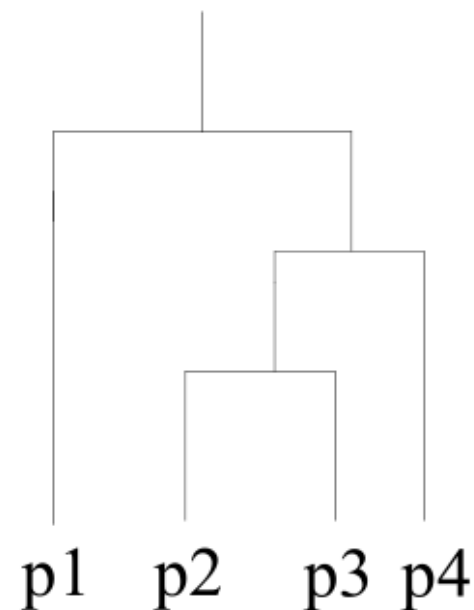
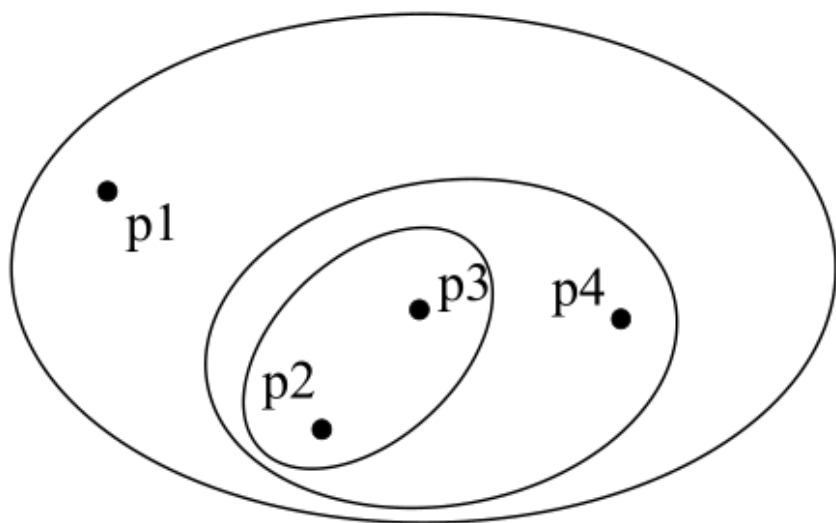
#### ■ 基本算法流程

1. 计算各数据间的相似度矩阵
2. 每个数据就是一个簇
3. Repeat
4.       合并两个最相似的簇形成新簇
5.       更新相似度矩阵
6. Until 只剩一个类簇

## 层次聚类

### 层次聚类运行过程

- 层次聚类不需要事先指定簇的数目，设定层次树高度就可以达到任意选择簇数量的目的



## 层次聚类

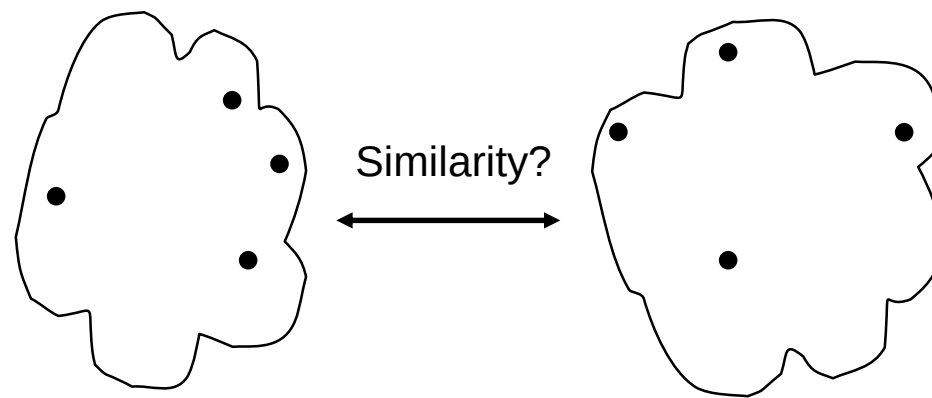
### 层次聚类问题

- 簇之间的相似度如何计算呢，即跟新相似度矩阵？
- 常用的策略

## 层次聚类：如何定义簇之间的距离

### 簇间相似度

- MIN(单连接)
- MAX(全连接)
- Group Average(组平均)
- Distance Between Centroids(质心距离)

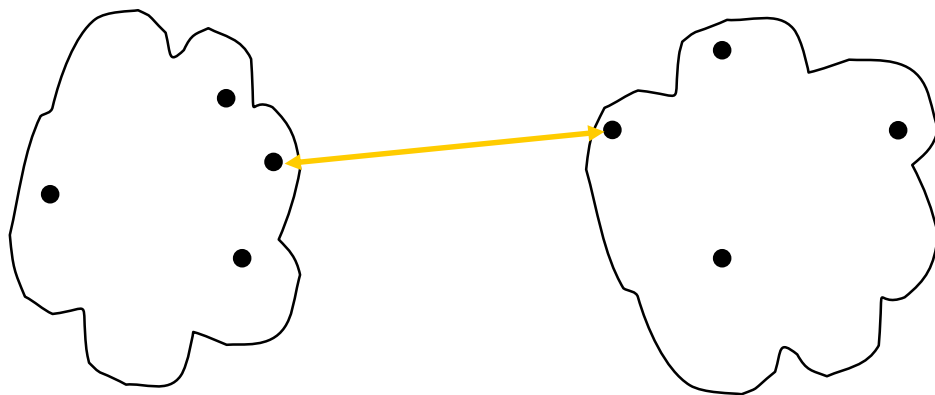




## 层次聚类：如何定义簇之间的距离

### 簇间相似度

- MIN(单连接)：单链接的计算方法是将两个簇的数据点中距离最近的两个数据点间的距离作为这两个簇的距离

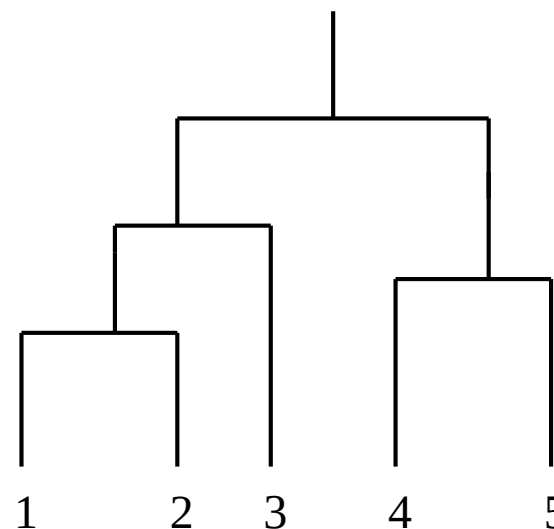


## 层次聚类：如何定义簇之间的距离

### 单链接的运算过程

■ MIN(单连接)：单链接的计算方法是将两个簇的数据点中距离最近的两个数据点间的距离作为这两个簇的距离

|    | I1   | I2   | I3   | I4   | I5   |
|----|------|------|------|------|------|
| I1 | 1.00 | 0.90 | 0.10 | 0.65 | 0.20 |
| I2 | 0.90 | 1.00 | 0.70 | 0.60 | 0.50 |
| I3 | 0.10 | 0.70 | 1.00 | 0.40 | 0.30 |
| I4 | 0.65 | 0.60 | 0.40 | 1.00 | 0.80 |
| I5 | 0.20 | 0.50 | 0.30 | 0.80 | 1.00 |



## 单链接，全链接，组平均，质心聚类

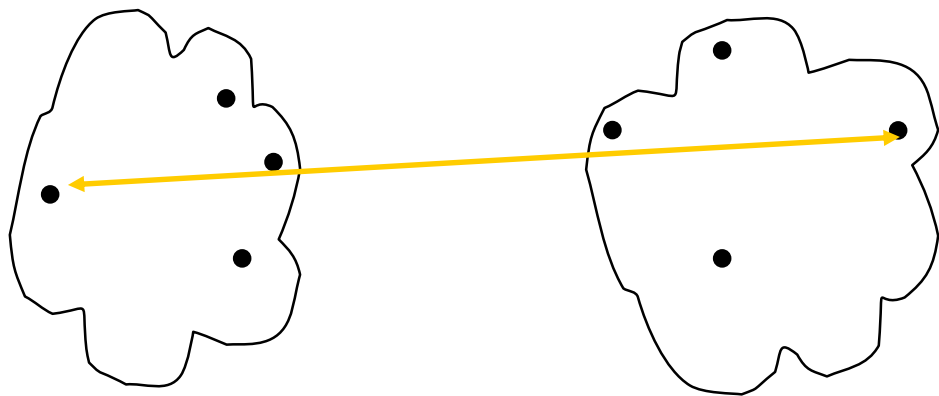
### 单链接聚类

- 这种方法容易受到极端值的影响。两个并非很相似的簇可能由于其中的某个极端的数据点距离较近而组合在一起
- 单连接产生非常分散的簇，擅长处理非球形簇
- 对离群点和噪声敏感

## 层次聚类：如何定义簇之间的距离

### 簇间相似度

- 全链接（Complete Linkage）的计算方法与单链接相反，将两个簇的数据点中距离最远的两个数据点间的距离作为这两个簇的距离

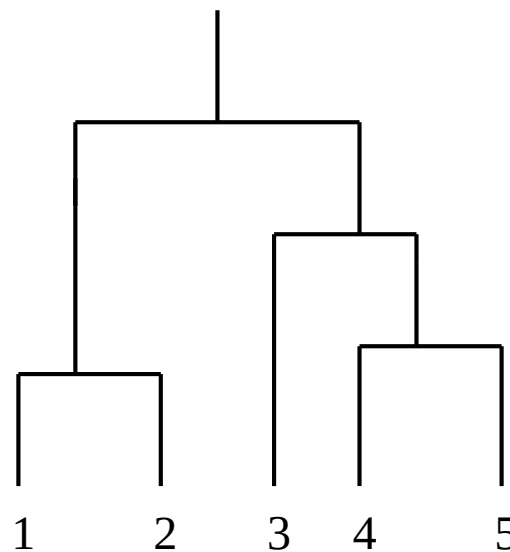


## 层次聚类：如何定义簇之间的距离

### 全链接的运算过程

- 全链接（Complete Linkage）的计算方法与单链接相反，将两个簇的数据点中距离最远的两个数据点间的距离作为这两个簇的距离

|    | I1   | I2   | I3   | I4   | I5   |
|----|------|------|------|------|------|
| I1 | 1.00 | 0.90 | 0.10 | 0.65 | 0.20 |
| I2 | 0.90 | 1.00 | 0.70 | 0.60 | 0.50 |
| I3 | 0.10 | 0.70 | 1.00 | 0.40 | 0.30 |
| I4 | 0.65 | 0.60 | 0.40 | 1.00 | 0.80 |
| I5 | 0.20 | 0.50 | 0.30 | 0.80 | 1.00 |



## 单链接，全链接，组平均，质心聚类

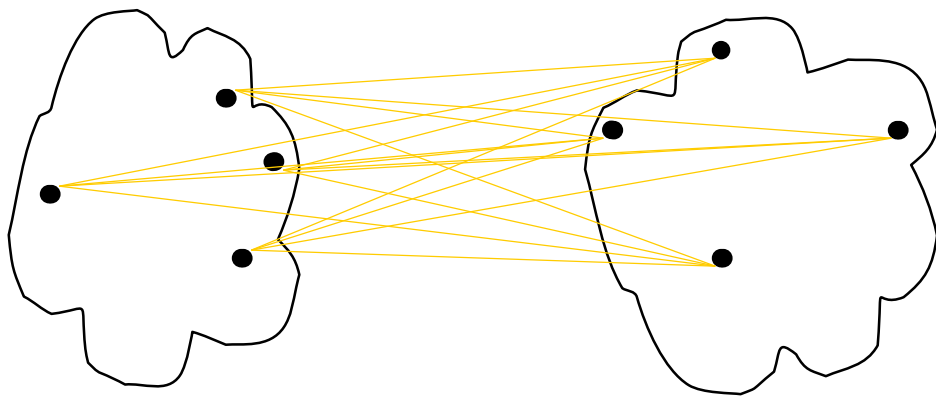
### 全链接聚类

- 对那些与总体结构不太一致的离群点给予了过多的关注。也就是说，全连接聚类并不能找出最直观的簇结构，容易打破大团
- 对离群点和噪声敏感

## 层次聚类：如何定义簇之间的距离

### 簇间相似度

- 组平均（Average Linkage）的计算方法是计算两个簇的数据点中的每个数据点与其他所有数据点的距离特点：



## 单链接，全链接，组平均，质心聚类

### 组平均链接聚类

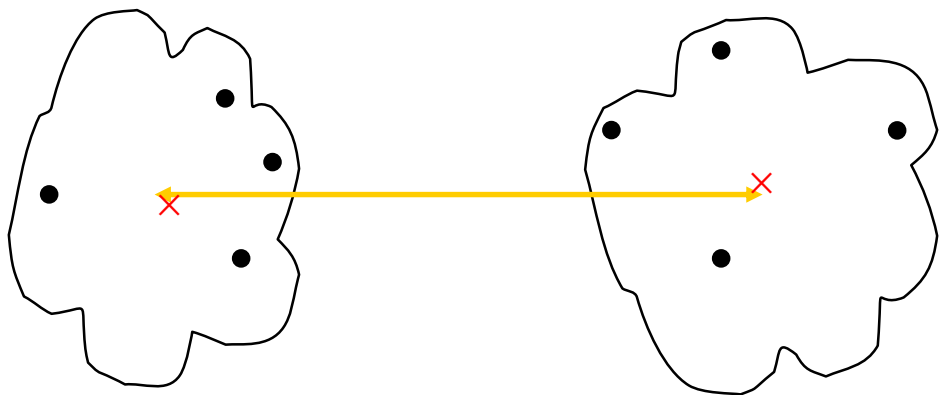
- 对单链接和全链接的权衡，方法计算量比较大，但结果比前两种方法合理
- 对噪声和离群点没那么敏感



## 层次聚类：如何定义簇之间的距离

### 簇间相似度

- 质心距离 (Distance Between Centroids) 的计算方法是将两个簇的质心距离作为这两个簇的距离



## 单链接，全链接，组平均，质心聚类

### 质心距离聚类

- 减少组平均方法计算量比较大问题
- 对噪声和离群点没那么敏感
- 偏向于球型簇

## 层次聚类

### 层次聚类特点

- 不需要输入聚类数目 $k$
- 一旦一个合并被执行，就不能修正
- 没有一个全局最优化的目标函数
- 计算量较大

## 层次聚类

### 代码实现层次聚类

```
>>> AgglomerativeClustering(n_clusters=2,  
affinity='euclidean', memory=None, connectivity=None,  
compute_full_tree='auto', linkage='ward',  
pooling_func=<function mean at 0x174b938>)
```

## 密度聚类

# 密度聚类 (Density-Based Clustering)

- 基于主要特点:
  - 发现任意形状的聚类
  - 处理噪音
  - 需要密度参数作为终止条件

## 基于划分的聚类

### DBSCAN聚类方式

#### ■ 一个动态网站:

<https://www.naftaliharris.com/blog/visualizing-dbscan-clustering/>

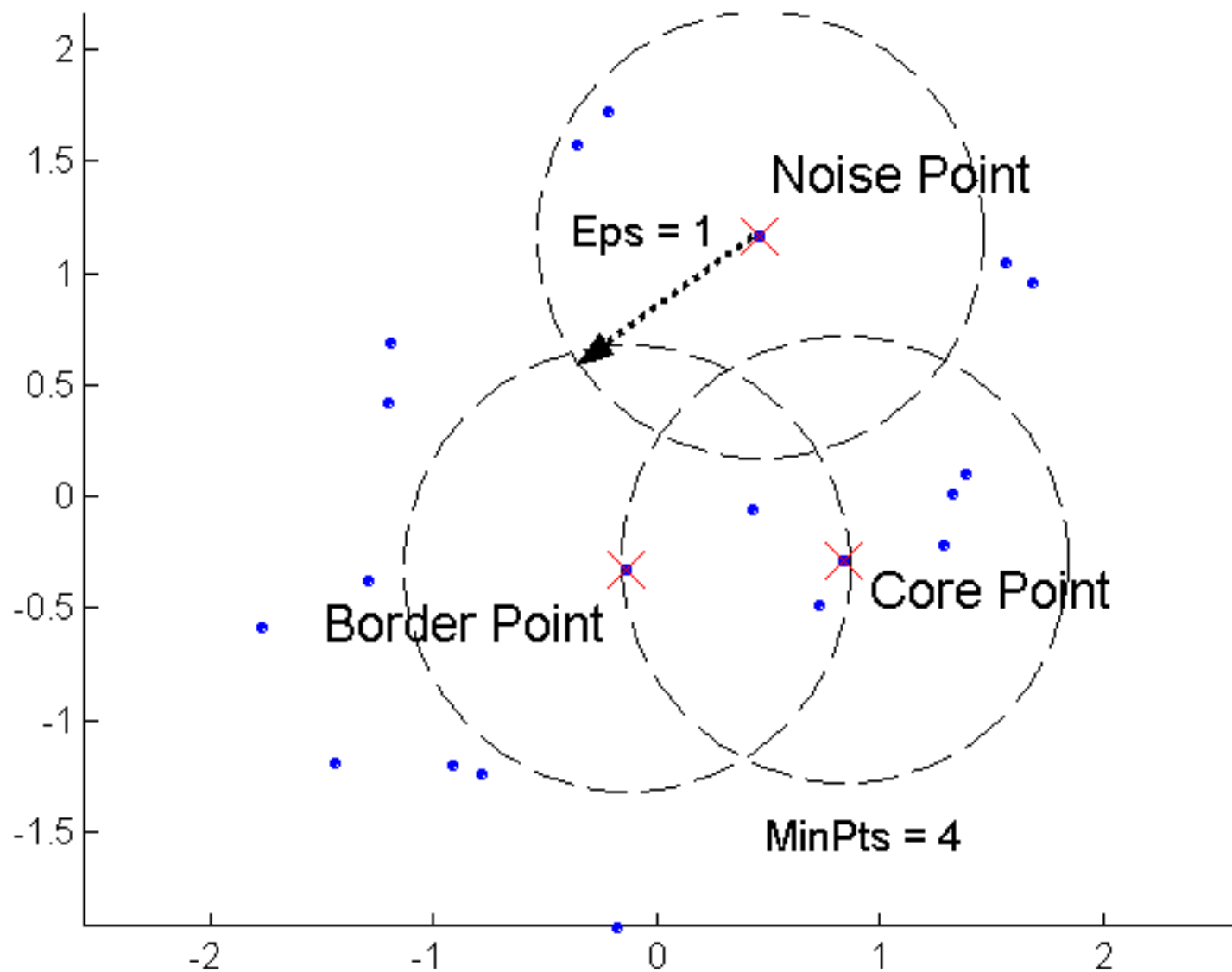
## 密度聚类

### DBSCAN

- 密度 = 特定半径内 (Eps) 数据点的数量
- 核心点: 当一个点 Eps 半径内 数据点的数量(包含自己)大于 (MinPts) 时, 它是一个核心点。
- 边界点: 当一个点 Eps 半径内 数据点的数量小于 (MinPts), 但包含至少一个核心点 时, 它是一个边界点。
- 噪声点: 不是核心点和边界点的其他点

## 密度聚类

### DBSCAN





## 密度聚类

### DBSCAN

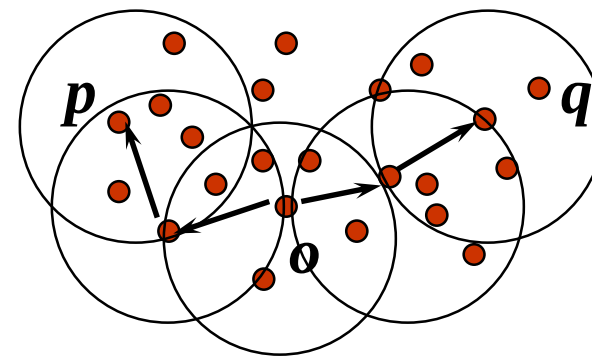
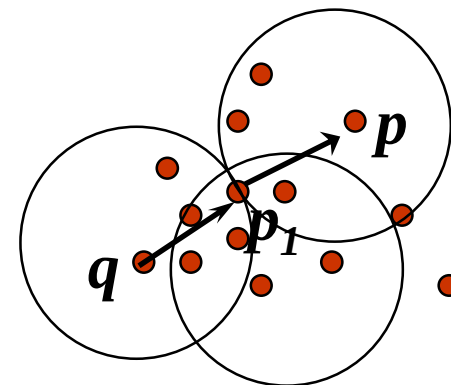
- $NEps(x) : \{y \text{ belongs to } D \mid \text{dist}(y,x) \leq Eps\}$
- 直接密度可达的: 点  $p$  关于  $Eps, MinPts$  是从点  $q$  直接密度可达的, 如果 :
  - 1)  $q$  达到核心点条件:
$$|NEps(q)| \geq MinPts$$
  - 2)  $p$  属于  $NEps(q)$

## 密度聚类

### DBSCAN

密度可达:

点  $p$  关于  $Eps$ ,  $MinPts$  是从  $q$  密度可达的: 如果 存在一个节点链  $p_1, \dots, p_n, p_1 = q, p_n = p$  使得  $p_{i+1}$  是从  $p_i$  直接密度可达的。(不是对称关系)

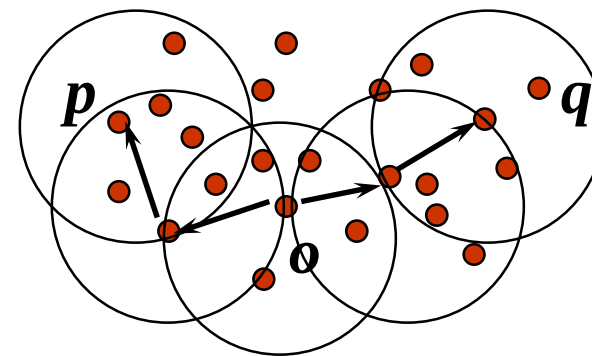
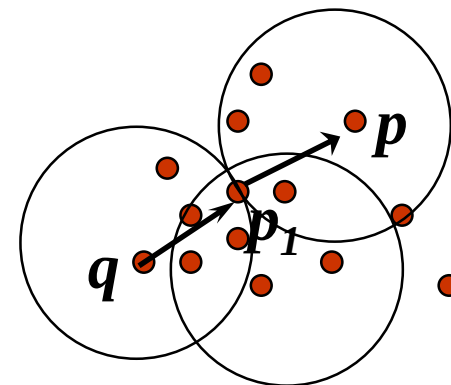


## 密度聚类

### DBSCAN

密度相连的:

点  $p$  关于  $Eps$ ,  $MinPts$  与点  $q$  是密度相连的: 如果存在点  $o$  使得,  $p$  和  $q$  都是关于  $Eps$ ,  $MinPts$  是从  $o$  密度可达的 (是对称的)



## 密度聚类

### DBSCAN

- 利用密度相连的特性来发现连通的稠密区域
- 任意两个足够靠近( $\epsilon$ 内)的核心点将放在一个簇内
- 任何与核心点足够靠近的边界点也会被放入与核心点的相同的簇

## 密度聚类

### DBSCAN算法

- (1) 检测数据库中尚未检查过的对象 $p$ ，如果 $p$ 未被处理(归为某个簇或者标记为噪声)，则检查其邻域，若包含的对象数 $\geq \text{minPts}$ ，建立新簇 $C$ ，将其中的所有点加入候选集 $N$ ；若 $\leq \text{minPts}$  则暂时标记为噪声
- (2) 对候选集 $N$  中所有尚未被处理的对象 $q$ ，检查其邻域，若至少包含 $\text{minPts}$ 个对象，则将这些对象加入 $N$ ；如果 $q$  未归入任何一个簇，则将 $q$  加入 $C$ ；
- (3) 重复步骤2)，继续检查 $N$  中未处理的对象，当前候选集 $N$ 为空；
- (4) 重复步骤1)~3)，直到所有对象都归入了某个簇或标记为噪声。

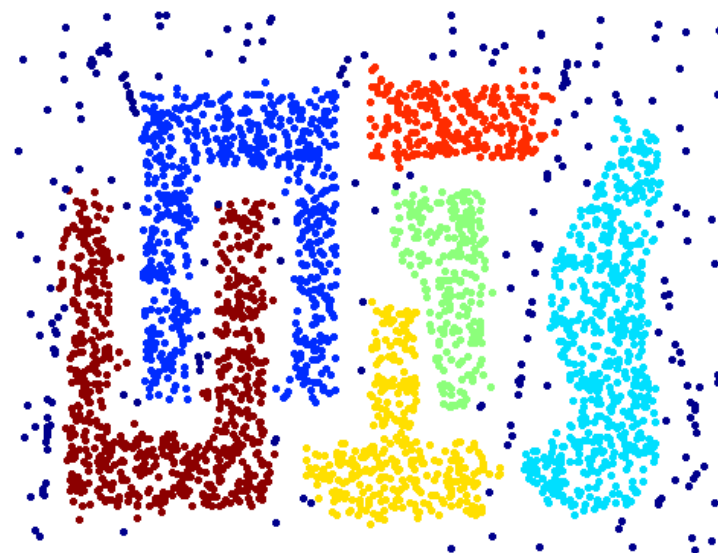
## 密度聚类

### DBSCAN优势

- 忽略噪声
- 发现任意形状的簇



原数据



聚类效果

## 密度聚类

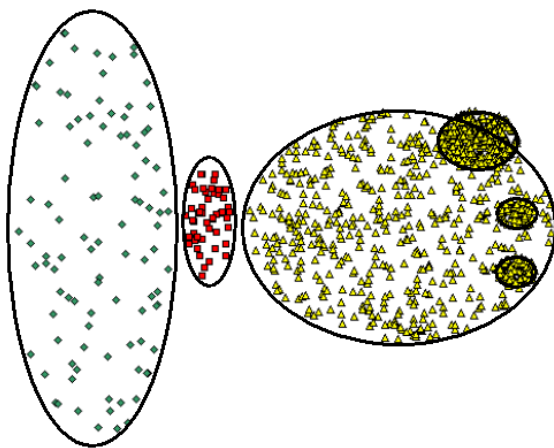
### DBSCAN劣势

■ 参数难以确定

■ 效果不佳:

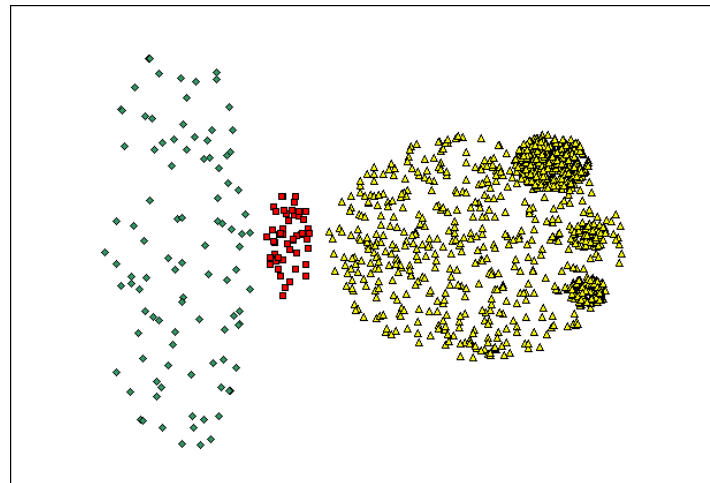
■ 密度不均

■ 高维数据

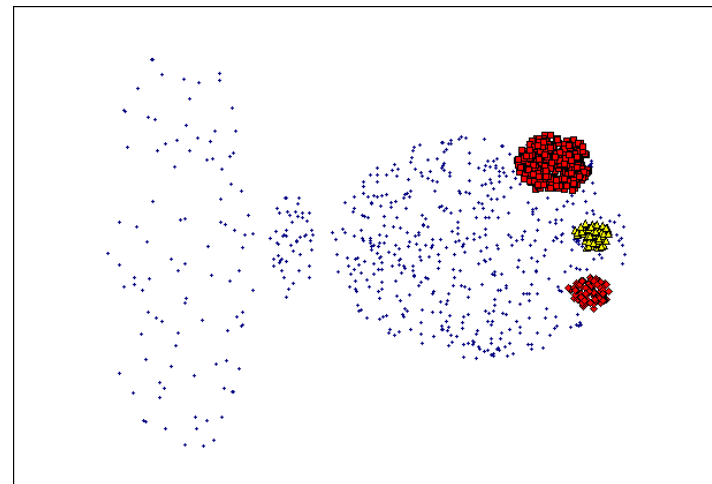


Original Points

- Varying densities
- High-dimensional data



(MinPts=4, Eps=9.75).



(MinPts=4, Eps=9.92)

## 聚类评测

### 聚类质量

- 聚类质量的评测是主观且困难的
- 外部方法：有监督的类标或者专家构建基准
- 内部方法：轮廓系数等等



## 聚类评测

### 轮廓系数

- 回想一下我们聚类的目标是什么呢？
- 测聚类结果好坏的一个方式是观察集群分离的离散程度，集群分离的离散程度的计算方式我们称为轮廓系数
- 轮廓系数的值是介于  $[-1, 1]$ ，越趋近于1代表内聚度和分离度都相对较优。将所有点的轮廓系数求平均，就是该聚类结果总的轮廓系数

## 聚类评测

### 轮廓系数

```
>>> from sklearn import metrics  
>>> score = metrics.silhouette_score(data, kmeans.labels_,  
metric='euclidean', sample_size=len(data))
```

## 降维

### 降维

- 无监督学习的另一种常见算法—降维:
- 降维: 减少要考虑的随机变量的数量
- 降维分析是指将高维数据到低维数数据空间（二维或三维）的变化过程，其目的是为了降低时间的复杂度和空间复杂度或者是去掉数据集中夹杂的噪音又或者是为了使用较少的特征进行解释，方便我们可以更好地解释数据以及实现数据的可视化

## 降维

### 数据在真实世界中的表示

- 在真实的训练数据中总是存在各种各样的问题：存在噪声或者冗余。在这种情况下，需要一种特征降维的方法来减少特征数，减少噪音和冗余，减少过度拟合的可能性

## 降维

### 数据在真实世界中的例子

- 汽车的样本，里面既有以“千米/每小时”度量的速度特征，又有“英里/小时”的速度特征，显然这两个特征有一个多余
- 在信号传输过程中，由于信道不是理想的，信道另一端收到的信号会有噪音扰动，那么怎么滤去这些噪音呢

## 降维

# 降维的必要性

Fisher's *Iris* Data

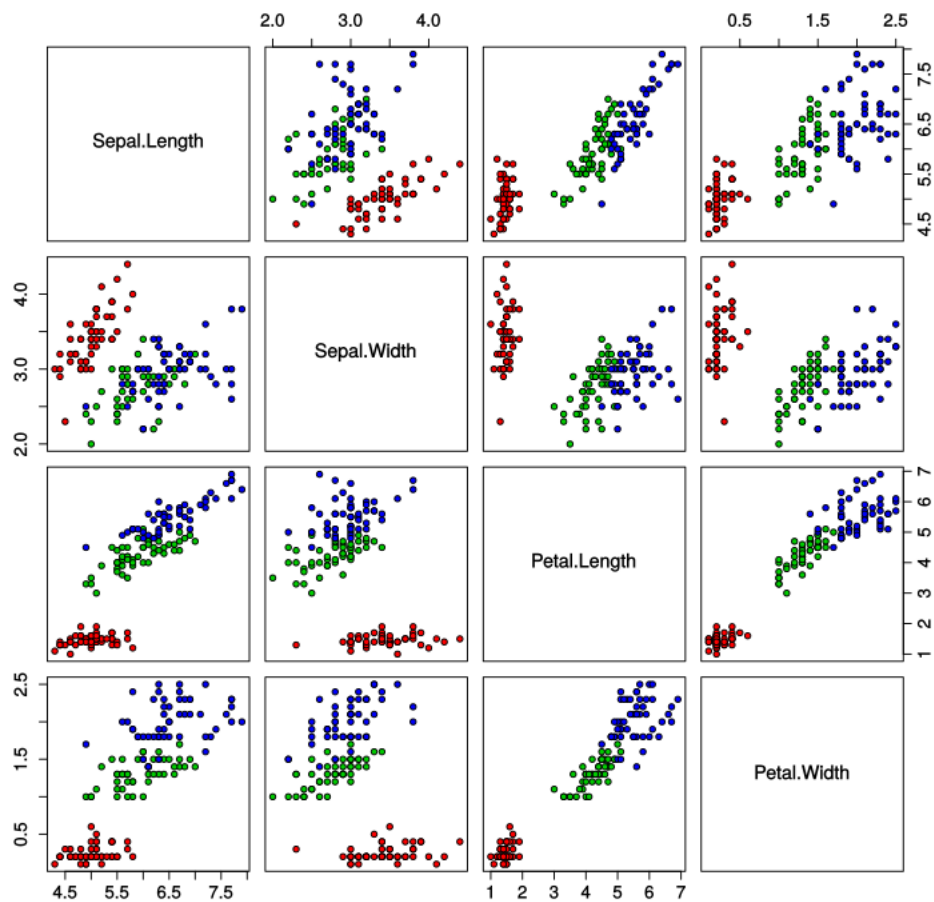
| Sepal length ◆ | Sepal width ◆ | Petal length ◆ | Petal width ◆ | Species ◆        |
|----------------|---------------|----------------|---------------|------------------|
| 5.1            | 3.5           | 1.4            | 0.2           | <i>I. setosa</i> |
| 4.9            | 3.0           | 1.4            | 0.2           | <i>I. setosa</i> |
| 4.7            | 3.2           | 1.3            | 0.2           | <i>I. setosa</i> |
| 4.6            | 3.1           | 1.5            | 0.2           | <i>I. setosa</i> |
| 5.0            | 3.6           | 1.4            | 0.3           | <i>I. setosa</i> |
| 5.4            | 3.9           | 1.7            | 0.4           | <i>I. setosa</i> |
| 4.6            | 3.4           | 1.4            | 0.3           | <i>I. setosa</i> |
| 5.0            | 3.4           | 1.5            | 0.2           | <i>I. setosa</i> |
| 4.4            | 2.9           | 1.4            | 0.2           | <i>I. setosa</i> |
| 4.9            | 3.1           | 1.5            | 0.1           | <i>I. setosa</i> |
| 5.4            | 3.7           | 1.5            | 0.2           | <i>I. setosa</i> |

|     |     |     |     |                     |
|-----|-----|-----|-----|---------------------|
| 7.1 | 3.0 | 5.9 | 2.1 | <i>I. virginica</i> |
| 6.3 | 2.9 | 5.6 | 1.8 | <i>I. virginica</i> |
| 6.5 | 3.0 | 5.8 | 2.2 | <i>I. virginica</i> |
| 7.6 | 3.0 | 6.6 | 2.1 | <i>I. virginica</i> |
| 4.9 | 2.5 | 4.5 | 1.7 | <i>I. virginica</i> |
| 7.3 | 2.9 | 6.3 | 1.8 | <i>I. virginica</i> |
| 6.7 | 2.5 | 5.8 | 1.8 | <i>I. virginica</i> |
| 7.2 | 3.6 | 6.1 | 2.5 | <i>I. virginica</i> |
| 6.5 | 3.2 | 5.1 | 2.0 | <i>I. virginica</i> |
| 6.4 | 2.7 | 5.3 | 1.9 | <i>I. virginica</i> |

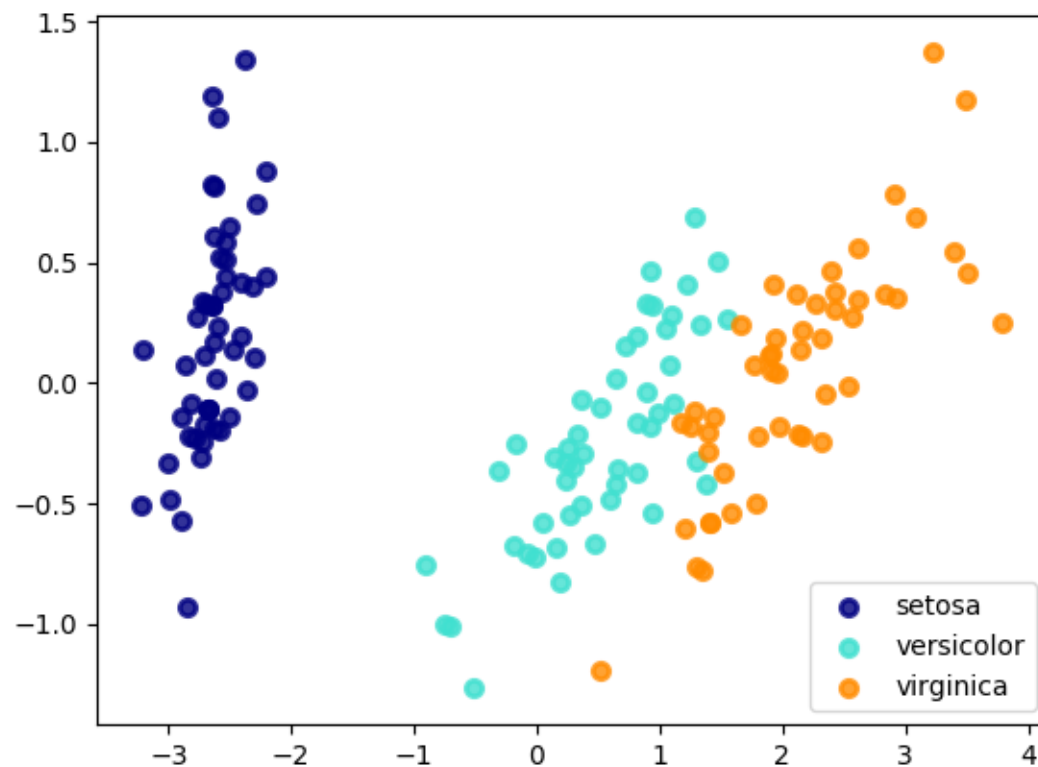
## 降维

# 降维的必要性

Iris Data (red=setosa,green=versicolor,blue=virginica)



PCA of IRIS dataset



## 降维

### 特征维度减少的方法

- 实现维度减少有两种技术，既特征选择和特征提取
- 特征选择指的是选择原始数据集中最具代表性或者统计意义的维度特征
- 特征提取指的是将原始特征转换为一组具有明显物理意义或者统计意义的特征



## 降维

### 特征选择

- 基于统计测试来选择最好的特征，常用的统计指标：
- 评分最高：SelectKBest
- 用户指定的最高得分百分比：SelectPercentile
- 假阳性率 (false positive rate) : SelectFpr
- 伪发现率 (false discovery rate) : SelectFdr
- 族系误差 (family wise error) : SelectFwe

## 降维

### 特征选择的代码实现

```
>>> from sklearn.datasets import load_iris
>>> from sklearn.feature_selection import SelectKBest
>>> from sklearn.feature_selection import chi2
>>> iris = load_iris()
>>> X, y = iris.data, iris.target
>>> X_new = SelectKBest(chi2, k=2).fit_transform(X, y)
```

## 降维

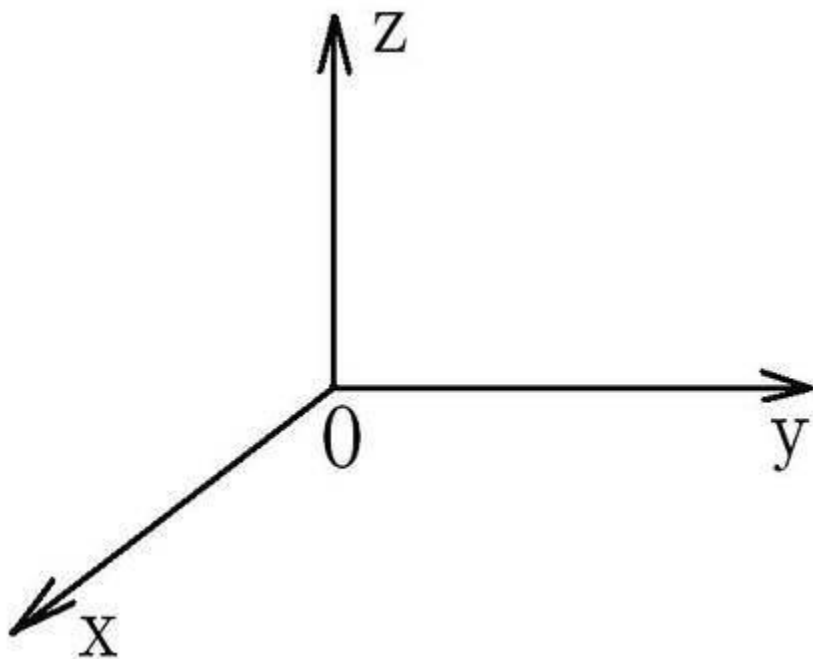
### 特征提取

- 特征提取指的是将原始特征转换为一组具有明显物理意义或者统计意义的特征
- 提取指的是我们并不会因此丢弃某些维度的数据，而转换指的是保留所有的维度数据，只是将高维数据投影低维数空间

## 降维

### 特征提取的转换

■ 思考：考虑三维到二维的转换



## 降维

### 特征提取

- 常用的降维方法：
- 主成分分析 (PCA)
- 因子分析 (Factor Analysis)

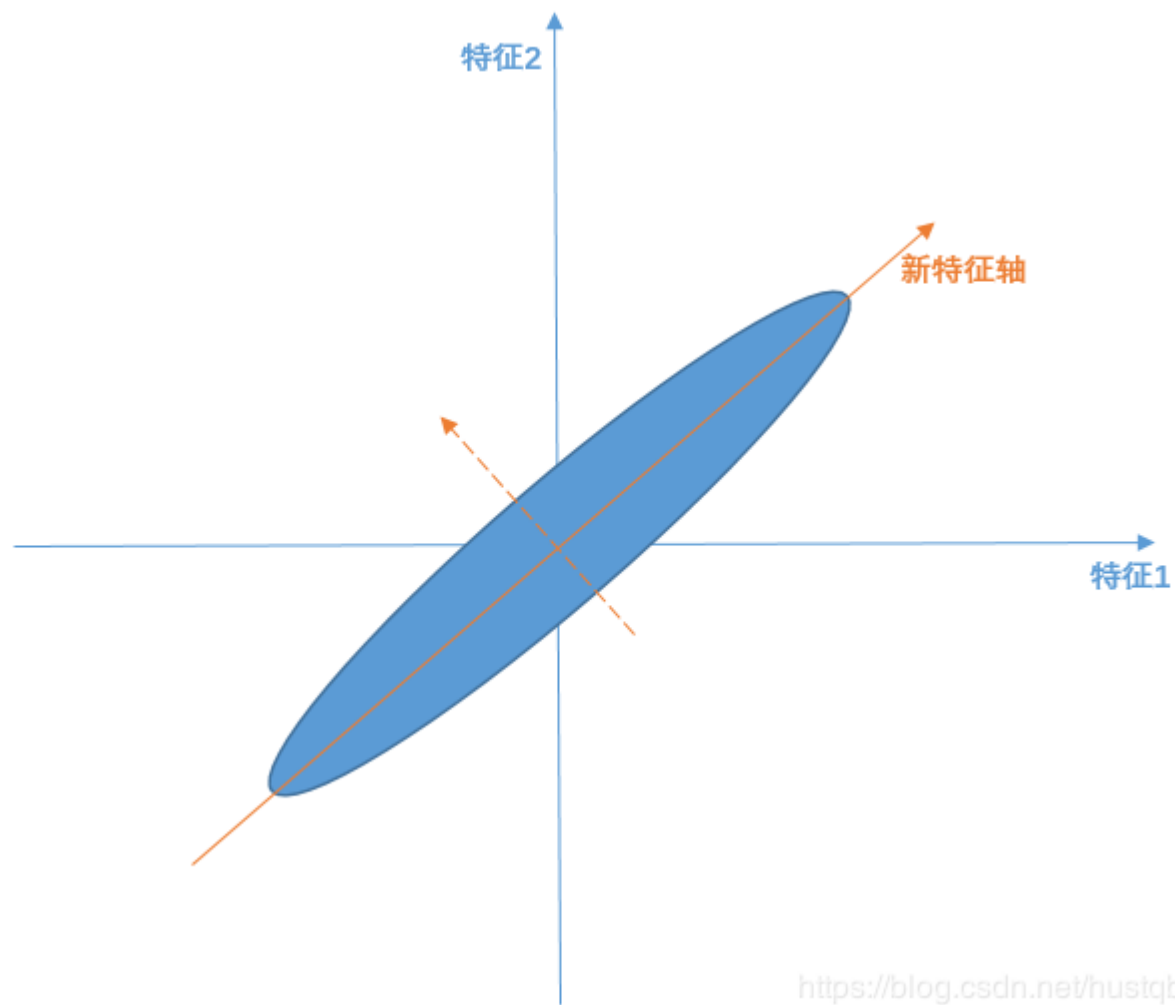
## 降维

### PCA算法

- 主成分分析（PCA）中，数据从原来的坐标系转换到新的坐标系，新坐标系的选择是由数据本身决定的。第一个新坐标轴选择的是原始数据中方差最大的方向，第二个新坐标轴选择和第一个坐标轴正交且具有最大方差的方向。该过程一直重复，重复次数为原始数据中特征数目

## 降维

### PCA图示



<https://blog.csdn.net/hustqb>

## 降维

### PCA算法过程

- 线性变换  $\Rightarrow$  新特征轴可由原始特征轴线性变换表征
- 线性无关  $\Rightarrow$  构建的特征轴是正交的
- 主要线性分量（或者说是主成分）  $\Rightarrow$  方差最大的方向  
(希望投影后投影值尽可能分散，而这种分散程度，可以用数学上的方差来表述)
- PCA算法的求解就是找到主要线性分量及其表征方式的过程



## 降维

### PCA算法的特点

- 相应的，PCA解释方差并对离群点很敏感：少量远离中心的点对方差有很大的影响，从而也对特征向量有很大的影响

## 降维

### PCA的代码实现

```
>>> from sklearn import datasets  
>>> iris = datasets.load_iris()  
>>> data=iris.data  
>>> from sklearn.decomposition import PCA  
>>> pca=PCA(n_components=2)  
>>> newData=pca.fit_transform(data)
```



# 第八章 关联规则

# 目 录

01

关联规则的引言

---

02

apriori算法

---

03

关联规则的度量

---

04

协同过滤（基于item和users）

---

## 引言

# 推荐算法

## ■ 推荐算法是什么？

The screenshot displays a user interface for a recommendation system. At the top, there's a '今日推荐' (Today's Recommendation) section with five promotional banners: '音乐传输变革者' (Music transmission revolution), '时尚大牌不止5折' (Fashion brands more than 50% off), '春季名品特卖' (Spring famous products special sale), and 'NB爆款低至7元/期' (NB hot products as low as 7 yuan/period). Below this is a '猜你喜欢' (Guess you like) section with the subtitle '根据你的喜好精心为你推荐!' (Carefully recommended for you based on your preferences!). On the left of this section is a sidebar titled '发现你的喜欢' (Discover what you like) with buttons for '智能家居' (Smart home), '智能出行' (Smart travel), '图书' (Books), and '影音娱乐' (Audio and video entertainment). The main area shows five product recommendations: '森海塞尔 (Sennheiser)' earbuds for ¥2699.00, 'AKG K450 头戴式耳机' (Headset) for ¥339.00, '惠科 (ENKOR) E50 笔记' (Notebook) for ¥79.90, '拜耳 (Faber) 橡皮' (Eraser) for ¥128.00, and '华为荣耀2' (Huawei Honor 2) for ¥298.00. A vertical sidebar on the right contains icons for shopping cart, heart, share, and other functions. A watermark '腾讯大数据' (Tencent Big Data) is visible in the bottom right of the product area.

今日推荐

音乐传输变革者  
忘路与空响同享  
配了解更热爱

时尚大牌不止5折  
春季换新show  
官方推荐  
团购

春季名品特卖  
大牌低至9元  
时尚先锋

NB爆款低至7元/期  
12期分期免息  
又潮又有范儿  
免息

猜你喜欢 根据你的喜好精心为你推荐!

发现你的喜欢

智能家居 智能出行  
图书 影音娱乐

森海塞尔 (Sennheiser)  
¥2699.00

AKG K450 头戴式耳机 折叠  
¥339.00

惠科 (ENKOR) E50 笔记  
¥79.90

拜耳 (Faber) 橡皮  
¥128.00

华为荣耀2  
¥298.00

腾讯大数据

## 引言

# 推荐算法

## ■ 推荐算法是什么？



### 我的歌单



### 上传本地音乐

绿钻到期，您已无法享有以下特权：绿钻



猜你喜欢



## 引言

### 推荐算法目的

- 帮助用户找到想要的商品（新闻/音乐/.....
- 降低信息过载

## 引言

### 如何进行推荐？

- 关联规则
- 协同过滤



## 引言

### 关联规则挖掘

- 关联规则挖掘：一种发现大量数据中事物（特征）之间有趣的关联的技术。
- 典型应用是购物篮分析：找出顾客购买行为模式、发现交易数据库中不同商品（项）之间的联系

## 引言

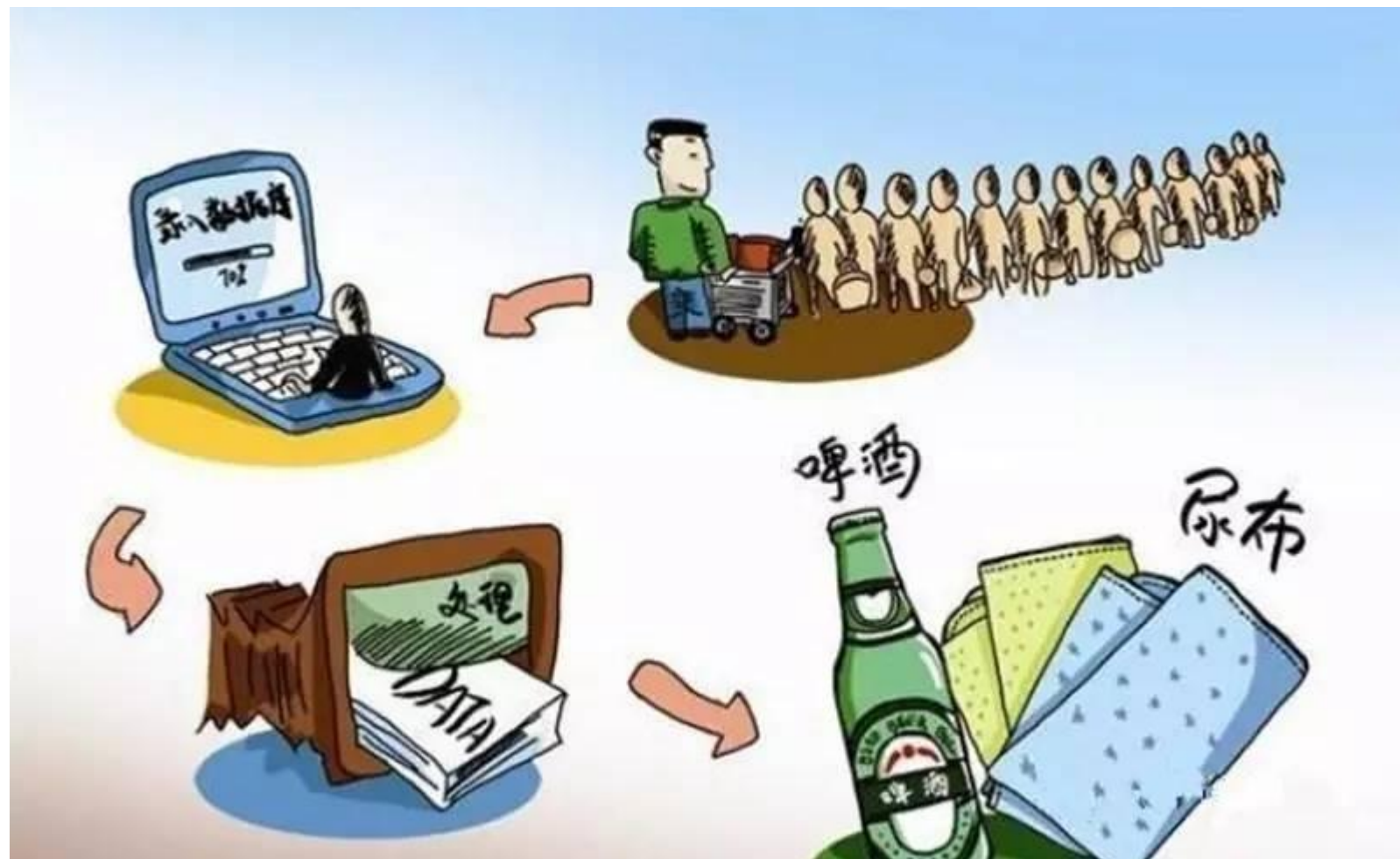
### 关联规则挖掘

- 关联规则挖掘的应用：
- 互联网
- 零售
- 交通事故成因
- 生物医学

## 引言

### 关联规则挖掘

#### ■ 啤酒尿布案例



## 引言

### 关联规则挖掘例子

| <i>TID</i> | <i>Items</i>  |
|------------|---------------|
| T1         | {牛奶,面包}       |
| T2         | {面包,尿布,啤酒,鸡蛋} |
| T3         | {牛奶,尿布,啤酒,可乐} |
| T4         | {面包,牛奶,尿布,啤酒} |
| T5         | {面包,牛奶,尿布,可乐} |

总项集  $I = \{\text{牛奶, 面包, 尿布, 啤酒, 鸡蛋, 可乐}\}$ 。

{啤酒}  $\rightarrow$  {尿布} 是一条表征购买了啤酒也会购买尿布的关联规则。

## 引言

### 关联规则定义

- 假设  $I = \{I_1, I_2, \dots, I_m\}$  是项的集合。给定一个事务数据库  $D$ ，其中每个事务 (Transaction)  $t$  是  $I$  的非空子集
- 关联规则：不相交的非空项集  $X$ 、 $Y$ ，蕴含式  $X \rightarrow Y$ ， $X \rightarrow Y$  是一条关联规则。其中  $X \subset I$ ， $Y \subset I$ ，且  $X \cap Y = \emptyset$

## 引言

### 关联规则的度量：支持度(support)

- 支持度的定义： $\text{support}(X \rightarrow Y) = |X \cap Y| / N$  = 集合X与集合Y中的项在一条记录中同时出现的次数/数据记录的个数。
- 例如： $\text{support}(\{\text{尿布}\} \rightarrow \{\text{啤酒}\}) = \text{啤酒和尿布同时出现的次数} / \text{数据记录数} = 3/5 = 60\%$ 。

## 引言

### ■关联规则的度量：置信度(confidence)来描述

■置信度：  $\text{confidence}(X \rightarrow Y) = |X \cap Y| / |X|$  = 集合X与集合Y中的项在一条记录中同时出现的次数/集合X出现的个数。

■  $\text{confidence}(\{\text{尿布}\} \rightarrow \{\text{啤酒}\})$  = 啤酒和尿布同时出现的次数/尿布出现的次数 =  $3/4 = 75\%$ 。

## 引言

- 支持度和置信度越高，说明规则越强。关联规则挖掘就是要找出数据库中满足支持度  $\text{support} \geq \text{min\_support}$ 、置信度  $\text{confidence} \geq \text{min\_confidence}$  的所有规则



## 引言

### 关联规则挖掘的步骤

- 有一个简单而粗暴的方法
  - 1. 列出所有规则
  - 2. 计算这些规则的支持度和置信度
  - 3. 留下满足支持度置信度阈值的关联规则

## 引言

### 关联规则挖掘的定义与步骤

- 仔细想一下，我们发现对于{啤酒 $\rightarrow$ 尿布}，{尿布 $\rightarrow$ 啤酒}这两个规则的支持度实际上只需要计算{尿布，啤酒}的支持度。
- 于是我们可以把关联规则挖掘拆解成按支持度和置信度两个要求分开计算。

## 引言

### 关联规则挖掘的定义与步骤

- 关联规则挖掘的定义：给定一个交易数据集 $T$ ，找出其中所有支持度 $\text{support} \geq \text{min\_support}$ 、置信度 $\text{confidence} \geq \text{min\_confidence}$ 的关联规则。
- 关联规则分为两步：
  - 找出所有频繁项集
  - 由频繁项集生成满足最小信任度阈值的规则

### 关联规则挖掘的步骤

#### ■ 1) 找出所有频繁项集

这一阶段找出所有满足最小支持度的项集，找出的这些项集称为频繁项集。

#### ■ 2) 由频繁项集生成满足最小信任度阈值的规则，

在上一步产生的频繁项集的基础上生成满足最小自信度的规则，产生的规则称为强规则。

## apriori算法

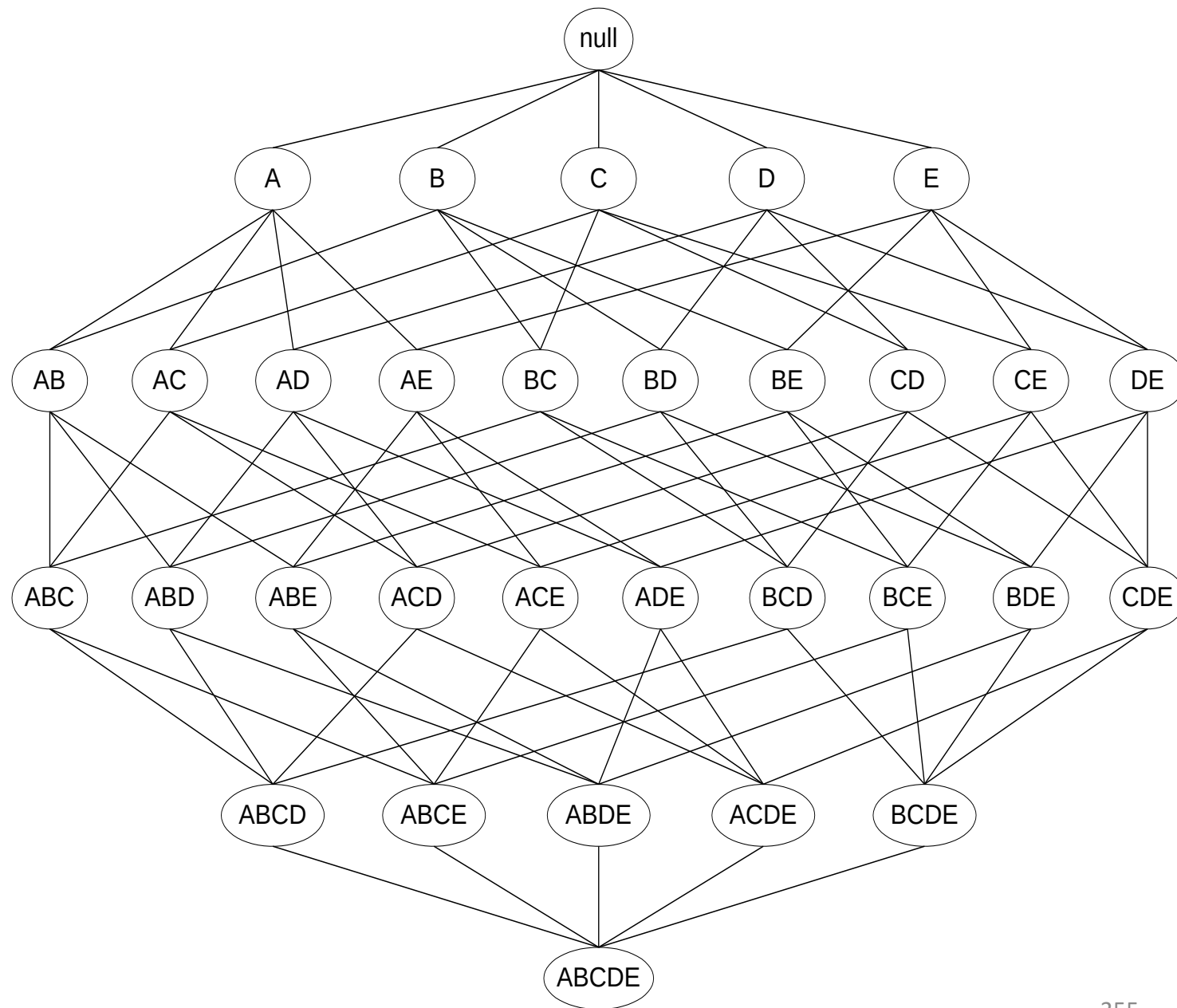
### 频繁项集挖掘

■ 搜索空间依旧

组合爆炸:

■  $n$ 个项数,  $2^d$

个可能



## apriori算法

### Apriori定律

- 为了减少频繁项集的搜索空间，应该尽早的消除一些完全不可能是频繁项集的集合，
- Apriori定律  
频繁项集的子集必为频繁项集。

## apriori算法

### Apriori定律的性质

- Apriori定律性质1): 如果一个集合是频繁项集, 则它的所有子集都是频繁项集。
- 举例: 假设一个集合 $\{A, B\}$ 是频繁项集, 即A、B同时出现在一条记录的次数大于等于最小支持度 $\text{min\_support}$ , 则它的子集 $\{A\}, \{B\}$ 出现次数必定大于等于 $\text{min\_support}$ , 即它的子集都是频繁项集。

## apriori算法

### Apriori算法

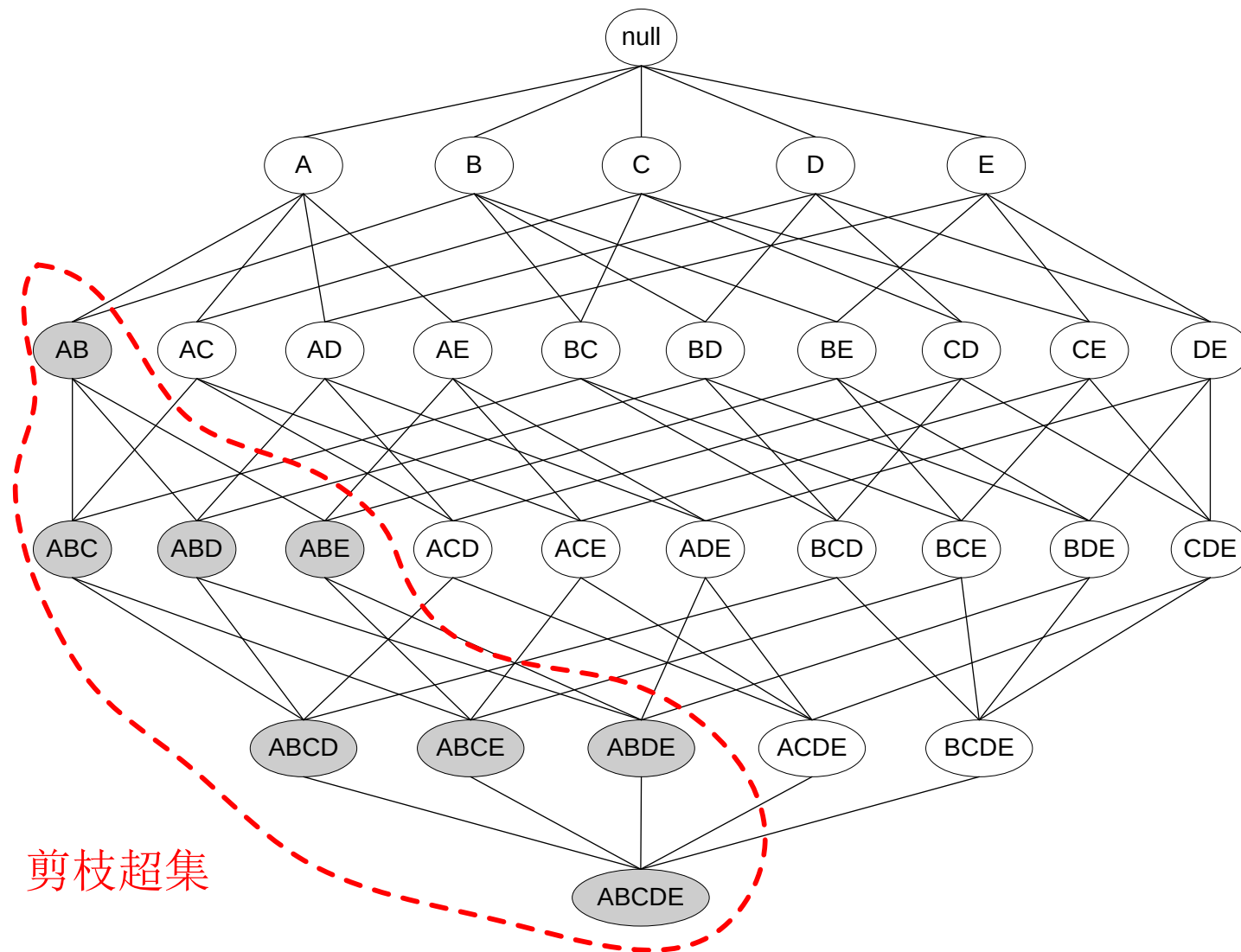
- Apriori定律2): 如果一个集合不是频繁项集, 则它的所有超集都不是频繁项集。
- 举例: 假设集合{A}不是频繁项集, 即A出现的次数小于min\_support, 则它的任何超集如{A,B}出现的次数必定小于min\_support, 因此其超集必定也不是频繁项集。



## apriori算法

### Apriori算法

- 利用这两条定律，  
可以过滤掉很多的  
候选项集

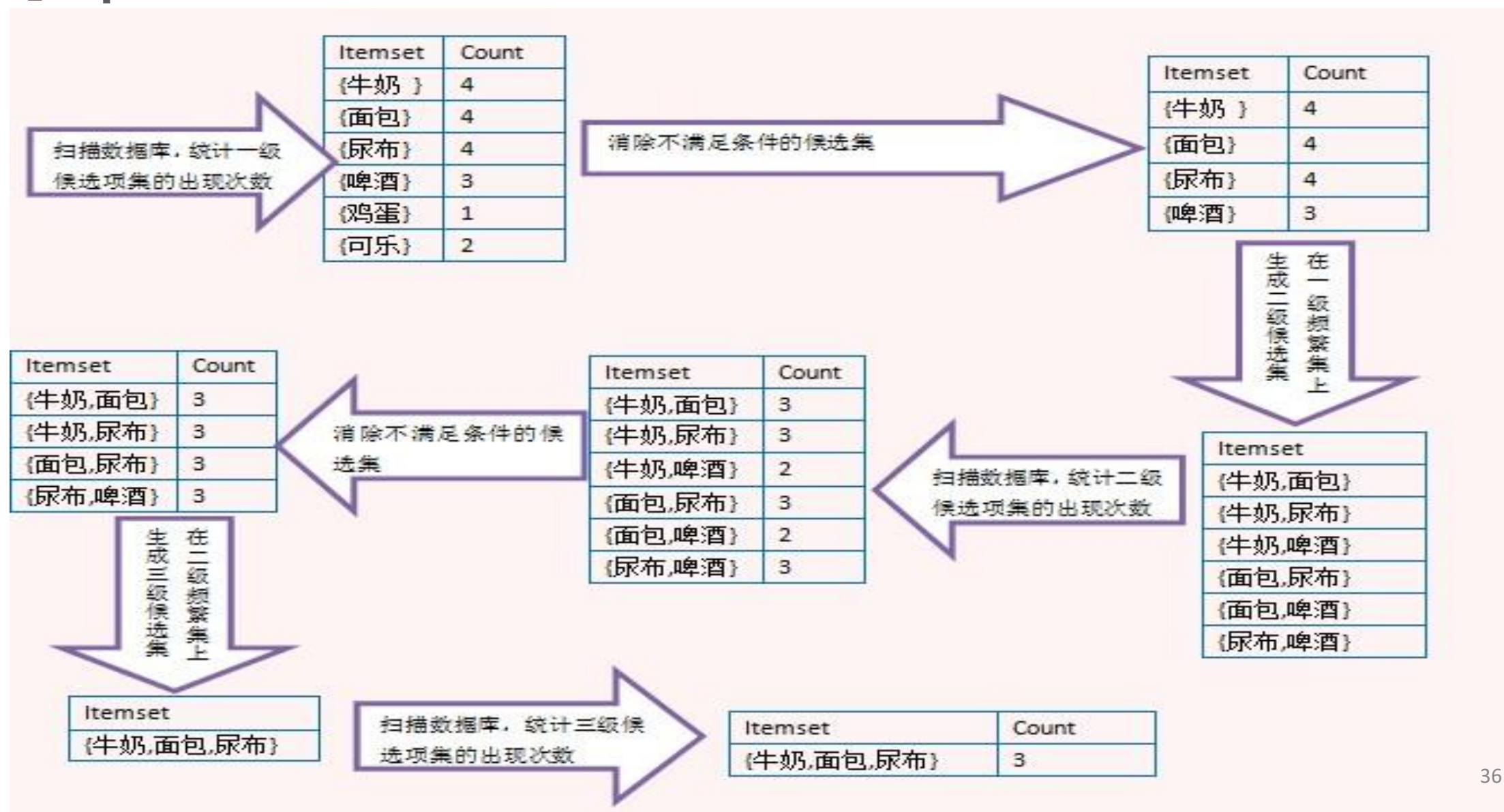


## apriori算法

### Apriori算法

- 利用性质1 通过已知的频繁项集构成长度更大的项集，并将其称为**候选频繁项集**。
- 利用性质2 过滤候选频繁项集中的非频繁项集
- 二项频繁集由在一项频繁集的基础上产生的，三项频繁项集由二项频繁项集的基础上产生的，以此类推。

## apriori算法



## apriori算法

### Apriori算法核心思想

- (1)  $L_1 = \{\text{频繁1项集}\};$
- (2) for( $k=2; L_{k-1} \neq \emptyset; k++$ ) do begin
- (3)    $C_k = \text{apriori\_gen}(L_{k-1});$  //产生新的候选频繁项集
- (4)   for all *transactions*  $t \in D$  do begin
- (5)        $C_t = \text{subset}(C_k, t);$  //找出t中包含的候选频繁项
- (6)       for all *candidates*  $c \in C_t$  do
- (7)            $c.\text{count}++;$
- (8)   end;
- (9)    $L_k = \{c \in C_k \mid c.\text{count} \geq \text{minsup}\}$
- (10) end;
- (11) Answer =  $\bigcup_k L_k$

## apriori算法

### apriori\_gen连接步

假定  $L_{k-1}$  中的项集已排序(按字典序排序)

步骤 1:  $L_{k-1}$  自连接

procedure apriori\_gen( $L_{k-1}$ :frequent  $(k-1)$ -itemsets)

- (1)     for each itemset  $l_1 \in L_{k-1}$
- (2)         for each itemset  $l_2 \in L_{k-1}$
- (3)             if  $(l_1[1] = l_2[1]) \wedge (l_1[2] = l_2[2]) \wedge \dots \wedge (l_1[k-2] = l_2[k-2]) \wedge (l_1[k-1] < l_2[k-1])$  then {
- (4)                  $c = l_1 \bowtie l_2$ ; // join step: generate candidates
- (5)                 if has\_infrequent\_subset( $c, L_{k-1}$ ) then
- (6)                     delete  $c$ ; // prune step: remove unfruitful candidate
- (7)                 else add  $c$  to  $C_k$ ;
- (8)             }
- (9)     return  $C_k$ ;

## apriori算法

### 连接步

$L_2 = \{(\text{牛奶}, \text{面包}), (\text{牛奶}, \text{尿布}), (\text{面包}, \text{尿布}), (\text{啤酒}, \text{尿布})\}$

自连接:  $C3 = L_2 \bowtie L_2$

(牛奶, 面包, 尿布) 由(牛奶, 面包)和(牛奶, 尿布)

$L_3 = \{abc, abd, acd, ace, bcd\}$

自连接:  $C4 = L_3 \bowtie L_3$

Abcd: 由 abc 和 abd

Acde: 由 acd 和 ace

## apriori算法

### apriori\_gen剪枝步

#### ■判断项集是否非频繁

```
procedure has_infrequent_subset( $c$ : candidate  $k$ -itemset;  
                                 $L_{k-1}$ : frequent  $(k-1)$ -itemsets); // use prior knowledge  
(1)   for each  $(k-1)$ -subset  $s$  of  $c$   
(2)       if  $s \notin L_{k-1}$  then  
(3)           return TRUE;  
(4)   return FALSE;
```

## apriori算法

### 剪枝步

- $C3 = (\text{牛奶}, \text{面包}, \text{尿布})$
- $(\text{牛奶}, \text{面包}), (\text{牛奶}, \text{尿布}), (\text{面包}, \text{尿布})$  都出现在  $L2$  中
- $L3 = \{(\text{牛奶}, \text{面包}, \text{尿布})\}$
- $C4 = \{abcd, acde\}$
- $acde$  被删除, 因为  $ade$  不在  $L_3$
- $L4 = \{abcd\}$



## apriori算法

### Apriori算法不足

- 可能产生大量的候选集,以及可能需要重复扫描数据库,是 Apriori 算法的两大缺点。
- 主要改进方向:
  - 减少候选集产生
  - 降低无效的扫描库次数
  - 提高候选集与原数据的比较效率

## 关联规则兴趣度

### 推荐模式评估

- 关联规则挖掘使用支持度-置信度框架
  - 会产生大量冗余规则。
    - 例如  $\{A,B,C\} \rightarrow \{D\}$  等同于  $\{A,B\} \rightarrow \{D\}$
  - 强规则不代表有意义，甚至可能误导。
    - 例如  $\text{conf}(\{\text{怀孕} \rightarrow \text{女性}\}) = 100\%$ 。

## 关联规则兴趣度

### 使用相邻矩阵进行推荐

- 用相邻矩阵计算一个规则  $X \rightarrow Y$  的兴趣度。

Contingency table for  $X \rightarrow Y$

|                | Y        | $\overline{Y}$ |          |
|----------------|----------|----------------|----------|
| X              | $f_{11}$ | $f_{10}$       | $f_{1+}$ |
| $\overline{X}$ | $f_{01}$ | $f_{00}$       | $f_{0+}$ |
|                | $f_{+1}$ | $f_{+0}$       | $ T $    |

$f_{11}$ : support of X and Y

$f_{10}$ : support of  $\underline{X}$  and  $\overline{Y}$

$f_{01}$ : support of  $\overline{X}$  and  $\underline{Y}$

$f_{00}$ : support of  $\overline{X}$  and  $\overline{Y}$

## 关联规则兴趣度

### 相邻矩阵计算置信度

■ 关联规则: Tea  $\rightarrow$  Coffee

Confidence =  $P(\text{Coffee}|\text{Tea}) = 0.75$

but  $P(\text{Coffee}) = 0.9$

$\Rightarrow$  虽然置信度很高, 但从购买影响看, 规则误导了

$\Rightarrow P(\text{Coffee}|\overline{\text{Tea}}) = 0.9375$

|            | Coffee | <u>Coffee</u> |     |
|------------|--------|---------------|-----|
| Tea        | 15     | 5             | 20  |
| <u>Tea</u> | 75     | 5             | 80  |
|            | 90     | 10            | 100 |

## 关联规则兴趣度

### 统计独立性

1000 名学生

600 名会游泳 (S)

700 名会骑车 (B)

420 名同时会游泳和骑车 ( $S \wedge B$ )

$$P(S \wedge B) = 420/1000 = 0.42$$

$$P(S) \times P(B) = 0.6 \times 0.7 = 0.42$$

$$P(S \wedge B) = P(S) \times P(B) \Rightarrow \text{统计独立}$$

$$P(S \wedge B) > P(S) \times P(B) \Rightarrow \text{正相关}$$

$$P(S \wedge B) < P(S) \times P(B) \Rightarrow \text{负相关}$$

## 关联规则兴趣度

### 统计独立性

■ 能表现统计独立性的部分指标

$$Lift = \frac{P(Y | X)}{P(Y)}$$

$$Interest = \frac{P(X, Y)}{P(X)P(Y)}$$

$$PS = P(X, Y) - P(X)P(Y)$$

$$\phi - coefficient = \frac{P(X, Y) - P(X)P(Y)}{\sqrt{P(X)[1 - P(X)]P(Y)[1 - P(Y)]}}$$

## 关联规则兴趣度

### 相邻矩阵

■ 关联规则: Tea  $\rightarrow$  Coffee

x=coffee, y=tea时

$$p(x) = (75 + 15) / 100$$

$$p(x|y) = 15 / 20$$

|            | Coffee | <u>Coffee</u> |     |
|------------|--------|---------------|-----|
| Tea        | 15     | 5             | 20  |
| <u>Tea</u> | 75     | 5             | 80  |
|            | 90     | 10            | 100 |

## 关联规则兴趣度

### 相邻矩阵

■ 关联规则: Tea  $\rightarrow$  Coffee

Confidence =  $P(\text{Coffee}|\text{Tea}) = 0.75$

but  $P(\text{Coffee}) = 0.9$

$\Rightarrow \text{Lift} = 0.75/0.9 = 0.8333 (< 1, \text{负相关})$

$\Rightarrow \text{Interest} = 0.15/(0.2*0.9) = 0.833 (< 1, \text{负相关})$

|            | Coffee | <u>Coffee</u> |     |
|------------|--------|---------------|-----|
| Tea        | 15     | 5             | 20  |
| <u>Tea</u> | 75     | 5             | 80  |
|            | 90     | 10            | 100 |



## 关联规则兴趣度

### 兴趣度

#### ■非常多的兴趣度

#### ■用什么方式判断哪些指标比较好？

| #  | Measure                         | Formula                                                                                                                                                                                                                                                                                              |
|----|---------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1  | $\phi$ -coefficient             | $\frac{P(A,B) - P(A)P(B)}{\sqrt{P(A)P(B)(1-P(A))(1-P(B))}}$                                                                                                                                                                                                                                          |
| 2  | Goodman-Kruskal's ( $\lambda$ ) | $\frac{\sum_j \max_k P(A_j, B_k) + \sum_k \max_j P(A_j, B_k) - \max_j P(A_j) - \max_k P(B_k)}{2 - \max_j P(A_j) - \max_k P(B_k)}$                                                                                                                                                                    |
| 3  | Odds ratio ( $\alpha$ )         | $\frac{P(A,B)P(\bar{A},\bar{B})}{P(A,\bar{B})P(\bar{A},B)}$                                                                                                                                                                                                                                          |
| 4  | Yule's $Q$                      | $\frac{P(A,B)P(\bar{A}\bar{B}) - P(A,\bar{B})P(\bar{A},B)}{P(A,B)P(\bar{A}\bar{B}) + P(A,\bar{B})P(\bar{A},B)} = \frac{\alpha - 1}{\alpha + 1}$                                                                                                                                                      |
| 5  | Yule's $Y$                      | $\frac{\sqrt{P(A,B)P(\bar{A}\bar{B})} - \sqrt{P(A,\bar{B})P(\bar{A},B)}}{\sqrt{P(A,B)P(\bar{A}\bar{B})} + \sqrt{P(A,\bar{B})P(\bar{A},B)}} = \frac{\sqrt{\alpha} - 1}{\sqrt{\alpha} + 1}$                                                                                                            |
| 6  | Kappa ( $\kappa$ )              | $\frac{P(A,B) + P(\bar{A},\bar{B}) - P(A)P(B) - P(\bar{A})P(\bar{B})}{1 - P(A)P(B) - P(\bar{A})P(\bar{B})}$                                                                                                                                                                                          |
| 7  | Mutual Information ( $M$ )      | $\frac{\sum_i \sum_j P(A_i, B_j) \log \frac{P(A_i, B_j)}{P(A_i)P(B_j)}}{\min(-\sum_i P(A_i) \log P(A_i), -\sum_j P(B_j) \log P(B_j))}$                                                                                                                                                               |
| 8  | J-Measure ( $J$ )               | $\max \left( P(A, B) \log \left( \frac{P(B A)}{P(B)} \right) + P(\bar{A}\bar{B}) \log \left( \frac{P(\bar{B} \bar{A})}{P(\bar{B})} \right), \right. \\ \left. P(A, B) \log \left( \frac{P(A B)}{P(A)} \right) + P(\bar{A}\bar{B}) \log \left( \frac{P(\bar{A} \bar{B})}{P(\bar{A})} \right) \right)$ |
| 9  | Gini index ( $G$ )              | $\max \left( P(A)[P(B A)^2 + P(\bar{B} A)^2] + P(\bar{A})[P(B \bar{A})^2 + P(\bar{B} \bar{A})^2] \right. \\ \left. - P(B)^2 - P(\bar{B})^2, \right. \\ \left. P(B)[P(A B)^2 + P(\bar{A} B)^2] + P(\bar{B})[P(A \bar{B})^2 + P(\bar{A} \bar{B})^2] \right. \\ \left. - P(A)^2 - P(\bar{A})^2 \right)$ |
| 10 | Support ( $s$ )                 | $P(A, B)$                                                                                                                                                                                                                                                                                            |
| 11 | Confidence ( $c$ )              | $\max(P(B A), P(A B))$                                                                                                                                                                                                                                                                               |
| 12 | Laplace ( $L$ )                 | $\max \left( \frac{NP(A,B)+1}{NP(A)+2}, \frac{NP(A,B)+1}{NP(B)+2} \right)$                                                                                                                                                                                                                           |
| 13 | Conviction ( $V$ )              | $\max \left( \frac{P(A)P(\bar{B})}{P(\bar{A}B)}, \frac{P(B)P(\bar{A})}{P(\bar{B}A)} \right)$                                                                                                                                                                                                         |
| 14 | Interest ( $I$ )                | $\frac{P(A,B)}{P(A)P(B)}$                                                                                                                                                                                                                                                                            |
| 15 | cosine ( $IS$ )                 | $\frac{P(A,B)}{\sqrt{P(A)P(B)}}$                                                                                                                                                                                                                                                                     |
| 16 | Piatetsky-Shapiro's ( $PS$ )    | $P(A, B) - P(A)P(B)$                                                                                                                                                                                                                                                                                 |
| 17 | Certainty factor ( $F$ )        | $\max \left( \frac{P(B A) - P(B)}{1 - P(B)}, \frac{P(A B) - P(A)}{1 - P(A)} \right)$                                                                                                                                                                                                                 |
| 18 | Added Value ( $AV$ )            | $\max(P(B A) - P(B), P(A B) - P(A))$                                                                                                                                                                                                                                                                 |
| 19 | Collective strength ( $S$ )     | $\frac{P(A,B) + P(\bar{A}\bar{B})}{P(A)P(B) + P(\bar{A})P(\bar{B})} \times \frac{1 - P(A)P(B) - P(\bar{A})P(\bar{B})}{1 - P(A,B) - P(\bar{A}\bar{B})}$                                                                                                                                               |
| 20 | Jaccard ( $\zeta$ )             | $\frac{P(A,B)}{P(A) + P(B) - P(A,B)}$                                                                                                                                                                                                                                                                |
| 21 | Klogsen ( $K$ )                 | $\sqrt{P(A, B) \max(P(B A) - P(B), P(A B) - P(A))}$                                                                                                                                                                                                                                                  |

## 关联规则兴趣度

### 兴趣度的一些性质

- P1:  $M(A,B) = 0$  当A 和 B 是统计独立的
- P2:  $M(A,B)$  随  $P(A,B)$  增大,  $P(A)$  和  $P(B)$ 不变时候单调递增
- O1:  $M(A,B)$  变量交换不影响: 即  $A \rightarrow B$  和  $B \rightarrow A$  值相同
- O5: 值与不包含 A 或 B 的记录无关

## 关联规则兴趣度

- 不同兴趣度的不同性质
- 根据需求选择合适的兴趣度量规则

| Symbol    | Measure             | Range                                                                                                                  | P1   | P2  | P3  | O1    | O2  | O3   | O3' | O4  |
|-----------|---------------------|------------------------------------------------------------------------------------------------------------------------|------|-----|-----|-------|-----|------|-----|-----|
| $\Phi$    | Correlation         | -1 ... 0 ... 1                                                                                                         | Yes  | Yes | Yes | Yes   | No  | Yes  | Yes | No  |
| $\lambda$ | Lambda              | 0 ... 1                                                                                                                | Yes  | No  | No  | Yes   | No  | No*  | Yes | No  |
| $\alpha$  | Odds ratio          | 0 ... 1 ... $\infty$                                                                                                   | Yes* | Yes | Yes | Yes   | Yes | Yes* | Yes | No  |
| Q         | Yule's Q            | -1 ... 0 ... 1                                                                                                         | Yes  | Yes | Yes | Yes   | Yes | Yes  | Yes | No  |
| Y         | Yule's Y            | -1 ... 0 ... 1                                                                                                         | Yes  | Yes | Yes | Yes   | Yes | Yes  | Yes | No  |
| $\kappa$  | Cohen's             | -1 ... 0 ... 1                                                                                                         | Yes  | Yes | Yes | Yes   | No  | No   | Yes | No  |
| M         | Mutual Information  | 0 ... 1                                                                                                                | Yes  | Yes | Yes | Yes   | No  | No*  | Yes | No  |
| J         | J-Measure           | 0 ... 1                                                                                                                | Yes  | No  | No  | No    | No  | No   | No  | No  |
| G         | Gini Index          | 0 ... 1                                                                                                                | Yes  | No  | No  | No    | No  | No*  | Yes | No  |
| s         | Support             | 0 ... 1                                                                                                                | No   | Yes | No  | Yes   | No  | No   | No  | No  |
| c         | Confidence          | 0 ... 1                                                                                                                | No   | Yes | No  | Yes   | No  | No   | No  | Yes |
| L         | Laplace             | 0 ... 1                                                                                                                | No   | Yes | No  | Yes   | No  | No   | No  | No  |
| V         | Conviction          | 0.5 ... 1 ... $\infty$                                                                                                 | No   | Yes | No  | Yes** | No  | No   | Yes | No  |
| I         | Interest            | 0 ... 1 ... $\infty$                                                                                                   | Yes* | Yes | Yes | Yes   | No  | No   | No  | No  |
| IS        | IS (cosine)         | 0 .. 1                                                                                                                 | No   | Yes | Yes | Yes   | No  | No   | No  | Yes |
| PS        | Piatetsky-Shapiro's | -0.25 ... 0 ... 0.25                                                                                                   | Yes  | Yes | Yes | Yes   | No  | Yes  | Yes | No  |
| F         | Certainty factor    | -1 ... 0 ... 1                                                                                                         | Yes  | Yes | Yes | No    | No  | No   | Yes | No  |
| AV        | Added value         | 0.5 ... 1 ... 1                                                                                                        | Yes  | Yes | Yes | No    | No  | No   | No  | No  |
| S         | Collective strength | 0 ... 1 ... $\infty$                                                                                                   | No   | Yes | Yes | Yes   | No  | Yes* | Yes | No  |
| $\zeta$   | Jaccard             | 0 .. 1                                                                                                                 | No   | Yes | Yes | Yes   | No  | No   | No  | Yes |
| K         | Klosgen's           | $\left(\sqrt{\frac{2}{\sqrt{3}}}-1\right)\left(2-\sqrt{3}-\frac{1}{\sqrt{3}}\right) \dots 0 \dots \frac{2}{3\sqrt{3}}$ | Yes  | Yes | Yes | No    | No  | No   | No  | No  |

## 协同过滤（基于用户）

### 兴趣度计算

- 协同过滤简单来说是利用某兴趣相投、拥有共同经验之群体的喜好来推荐用户感兴趣的信息。
- 如果你现在想看个电影，但你不知道具体看哪部，你会怎么做？大部分的人会问问周围的朋友，看看最近有什么好看的电影推荐，而我们一般更倾向于从口味比较类似的朋友那里得到推荐。：

## 关联规则兴趣度

### 兴趣度计算

- 通过相邻矩阵来计算一个规则  $X \rightarrow Y$  的兴趣度
  - 会产生大量冗余规则。
    - 例如  $\{A, B, C\} \rightarrow \{D\}$  等同于  $\{A, B\} \rightarrow \{D\}$
  - 强规则不代表有意义，甚至可能误导。
    - 例如  $\text{conf}(\{\text{怀孕} \rightarrow \text{女性}\}) = 100\%$ 。

## 协同过滤（基于用户）

### 协同过滤算法

- 协同过滤简单来说是利用某兴趣相投、拥有共同经验之群体的喜好来推荐用户感兴趣的信息。
- 如果你现在想看个电影，但你不知道具体看哪部，你会怎么做？大部分的人会问问周围的朋友，看看最近有什么好看的电影推荐，而我们一般更倾向于从口味比较类似的朋友那里得到推荐。：

 协同过滤（基于用户）

|   | 老炮儿 | 唐人街探案 | 星球大战 | 寻龙诀 | 神探夏洛克 | 小门神 |
|---|-----|-------|------|-----|-------|-----|
| A | 3.5 | 1.0   |      |     |       |     |
| B | 2.5 | 3.5   | 3.0  | 3.5 | 2.5   | 3.0 |
| C | 3.0 | 3.5   | 1.5  | 5.0 | 3.0   | 3.5 |
| D | 2.5 | 3.5   |      | 3.5 | 4.0   |     |
| E | 3.5 | 2.0   | 4.5  |     | 3.5   | 2.0 |
| F | 3.0 | 4.0   | 2.0  | 3.0 | 3.0   | 2.0 |
| G | 4.5 | 1.5   | 3.0  | 5.0 | 3.5   |     |

## 协同过滤（基于用户）

### 协同过滤算法

- 协同过滤的整体思路只有两步，非常简单：寻找相似用户邻居，用相似邻居偏好来推荐物品
- 寻找相似用户，有很多种方法可以用来判断相似性，，其实是对于电影品味的相似，也就是说需要将A与其他几位用户做比较，判断是不是品味相似。



## 协同过滤（基于用户）

### 协同过滤算法

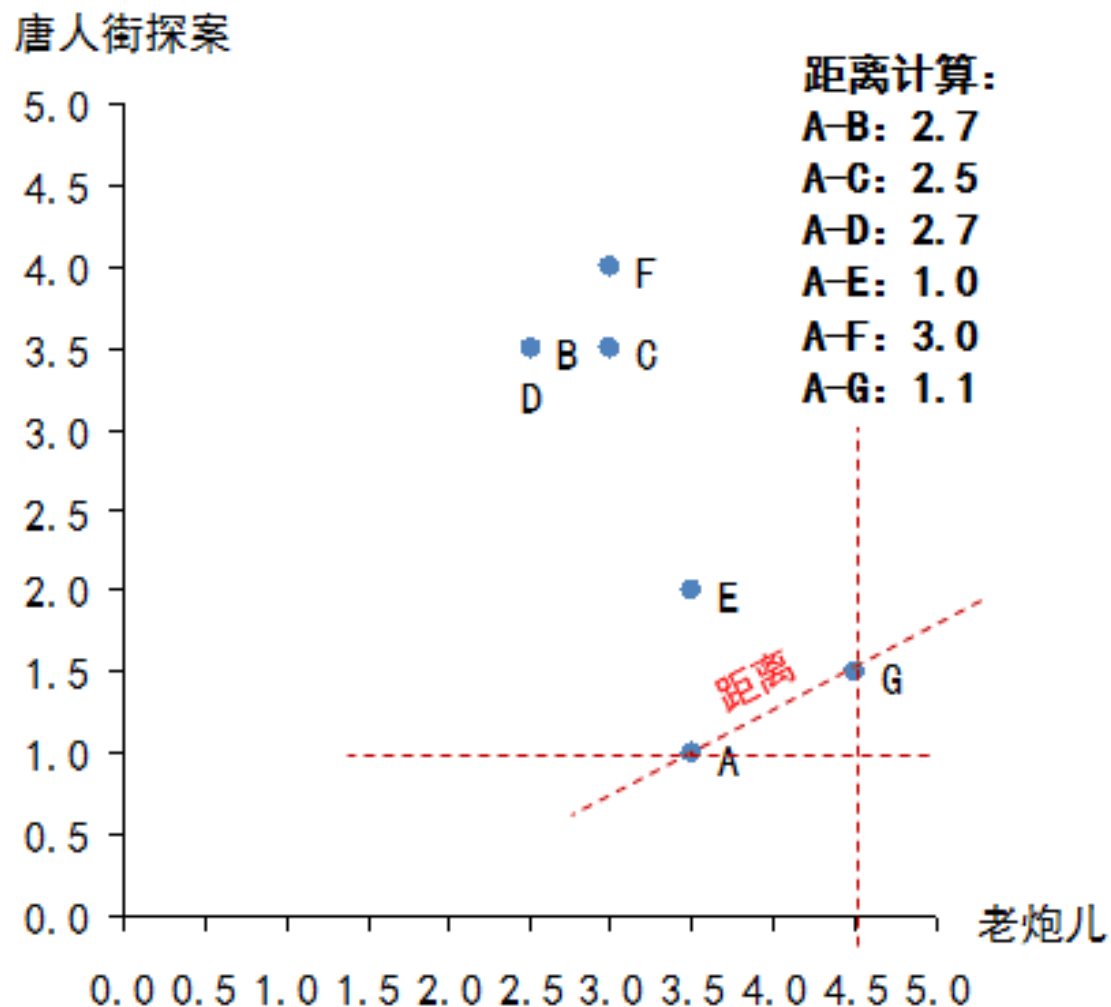
- 用户表示成 $N$ 维向量空间， $N$ 是所有的电影。一部电影看成 $N$ 维空间中的一个维度。用户的对电影的评分相当于就把用户固定在这个 $N$ 维空间的一个点上
- 利用欧几里德距离计算 $N$ 维空间两点的距离

## 协同过滤（基于用户）

### 协同过滤算法

- 本例中A只看过两部电影（《老炮儿》和《唐人街探案》），因此只能通过这两部电影来判断品味了，那么计算A和其他几位的距离：

## 协同过滤（基于用户）



变换方法为：相似性 =  $1/(1 + \text{欧几里德距离})$ ，这个相似性会落在【0, 1】区间内，1表示完全品味一样，0表示完全品味不一样。这时我们就可以找到哪些人的品味和A最为接近了，计算后如下：

相似性：B-0.27，C-0.28，D-0.27，E-0.50，F-0.25，G-0.47

可见，E的口味与A最为接近，其次是G

## 协同过滤（基于用户）

### 协同过滤算法

#### ■ 推荐电影

- 要做电影加权评分推荐。意思是说，品味相近的人对于电影的评价对A选择电影来说更加重要，具体做法可以列一个表，计算加权分：

协同过滤（基于用户）

|   | 相似性  | 星球大战 | 寻龙诀 | 神探夏洛克 | 小门神 |
|---|------|------|-----|-------|-----|
| A | -    | -    | -   | -     | -   |
| B | 0.27 | 3.0  | 3.5 | 2.5   | 3.0 |
| C | 0.28 | 1.5  | 5.0 | 3.0   | 3.5 |
| D | 0.27 | -    | 3.5 | 4.0   | -   |
| E | 0.50 | 4.5  | -   | 3.5   | 2.0 |
| F | 0.25 | 2.0  | 3.0 | 3.0   | 2.0 |
| G | 0.47 | 3.0  | 5.0 | 3.5   | -   |

乘

## 协同过滤（基于用户）

把相似性和对于每个电影的实际评分相乘，就是电影的加权分：

|      | 相似性  | 星球大战 | 寻龙诀   | 神探夏洛克 | 小门神  |
|------|------|------|-------|-------|------|
| A    | -    | -    | -     | -     | -    |
| B    | 0.27 | 0.81 | 0.945 | 0.675 | 0.81 |
| C    | 0.28 | 0.42 | 1.4   | 0.84  | 0.98 |
| D    | 0.27 | -    | 0.945 | 1.08  | -    |
| E    | 0.50 | 2.25 | -     | 1.75  | 1    |
| F    | 0.25 | 0.5  | 0.75  | 0.75  | 0.5  |
| G    | 0.47 | 1.41 | 2.35  | 1.645 | -    |
| 总分   |      | 5.39 | 6.39  | 6.74  | 3.29 |
| 总相似性 |      | 1.77 | 1.54  | 2.04  | 1.3  |
| 推荐度  |      | 3.05 | 4.15  | 3.30  | 2.53 |

## 协同过滤（基于用户）

### 协同过滤算法

- 加权后，还要做少量的计算：总分是每个电影加权分的总和，总相似度是对这个电影有评分的人的相似性综合，推荐度是总分/总相似性，目的是排除看电影人数对于总分的影响

## 协同过滤（基于用户）

### 协同过滤算法

- 结论在最终一行，就是电影的推荐度（因为是根据品味相同的人打分加权算出的分，可以近似认为如果A看了这部电影，预期的评分会是多少）。
- 有了电影的加权得分，通常做法还要设定一个阈值，如果超过了阈值再给用户推荐



## 协同过滤（基于物品）

### 协同过滤算法（基于物品）

- 基于用户的协同过滤，适用于物品较少，用户也不太多的情况。如果用户太多，针对每个用户的购买情况来计算哪些用户和他品味类似，效率很低下。如果商品很多，每个用户购买的商品重合的可能性很小，这样判断品味是否相似也就变得比较困难了。

## 协同过滤（基于物品）

### 协同过滤算法（基于物品）

- 还有一类智能推荐算法，是“基于物品的协同过滤”。  
基于如下考虑：用户购买行为变化大且重合低，物品变化相对较小。
- 原理与基于用户的协同过滤类似，只是在计算邻居时采用物品本身，而不是从用户的角度。

## 协同过滤（基于物品）

### 协同过滤算法（基于物品）

- 由于物品整体变化不大，所以这个相似程度不用每天都算，节省计算资源；同时，可以只给某一样商品只备选5个相似商品，推荐时只做这5个相似物品的加权评分，避免对所有商品都进行加权评分，以避免大量计算

## 协同过滤（基于物品）

针对同一例子：

|   | 老炮儿 | 唐人街探案 | 星球大战 | 寻龙诀 | 神探夏洛克 | 小门神 |
|---|-----|-------|------|-----|-------|-----|
| A | 3.5 | 1.0   |      |     |       |     |
| B | 2.5 | 3.5   | 3.0  | 3.5 | 2.5   | 3.0 |
| C | 3.0 | 3.5   | 1.5  | 5.0 | 3.0   | 3.5 |
| D | 2.5 | 3.5   |      | 3.5 | 4.0   |     |
| E | 3.5 | 2.0   | 4.5  |     | 3.5   | 2.0 |
| F | 3.0 | 4.0   | 2.0  | 3.0 | 3.0   | 2.0 |
| G | 4.5 | 1.5   | 3.0  | 5.0 | 3.5   |     |

## 协同过滤（基于物品）

### 协同过滤算法（基于物品）

- 首先是计算电影之间的相似度，方法还是有很多，这次用Pearson相关系数来做，公式为：

$$\rho_{X,Y} = \frac{\textcircled{1} \sum XY - \frac{\sum X \sum Y}{N} \textcircled{2}}{\sqrt{(\sum X^2 - \frac{(\sum X)^2}{N}) \textcircled{3} (\sum Y^2 - \frac{(\sum Y)^2}{N})} \textcircled{4} \textcircled{5} \textcircled{6}}$$

## 协同过滤（基于物品）

### 协同过滤算法（基于物品）

公式看起来复杂，其实可以分成6个部分分别计算就好了，

我们选《寻龙诀》（X）和《小门神》（Y）作为例子，

来算一下相似度，则

$X = (3.5, 5.0, 3.0)$

$Y = (3.0, 3.5, 2.0)$

数字就是评分，因为只有三个人同时看了这两个电影，所以X，Y两个向量都只有三个元素。按照公式逐步计算：

## 协同过滤（基于物品）

### 协同过滤算法（基于物品）

1. x和y的乘积再求和： $3.5 \times 3.0 + 5.0 \times 3.5 + 3.0 \times 2.0 = 34$
2. x求和乘以y求和，再除以个数： $((3.5 + 5.0 + 3.0) \times (3.0 + 3.5 + 2.0)) / 3 = 32.58$
3. x的平方和： $3.5^2 + 5.0^2 + 3.0^2 = 46.25$
4. x和的平方除以个数： $((3.5 + 5.0 + 3.0)^2) / 3 = 44.08$
5. y的平方和： $3.0^2 + 3.5^2 + 2.0^2 = 25.25$
6. y和的平方除以个数： $((3.0 + 3.5 + 2.0)^2) / 3 = 24.08$

## 协同过滤（基于物品）

### 协同过滤算法（基于物品）

- 最终把这几块的结果带入到整体的公式中：得出相关系数为0.89。
- 按照这种方法，需要两两计算电影的相似性，最终结果如下表：



协同过滤（基于物品）

|       | 老炮儿   | 唐人街探案 | 星球大战  | 寻龙诀  | 神探夏洛克 | 小门神 |
|-------|-------|-------|-------|------|-------|-----|
| 老炮儿   | 1     |       |       |      |       |     |
| 唐人街探案 | -0.77 | 1     |       |      |       |     |
| 星球大战  | 0.3   | -0.67 | 1     |      |       |     |
| 寻龙诀   | 0.65  | -0.68 | -0.08 | 1    |       |     |
| 神探夏洛克 | 0.25  | -0.38 | 0.44  | 0.12 | 1     |     |
| 小门神   | -0.54 | 0.32  | -0.55 | 0.89 | -0.54 | 1   |

## 协同过滤（基于物品）

### 协同过滤算法（基于物品）

- 首相关系数取值为【-1, 1】，1表示完全相似，0表示没关系，-1表示完全相反。结合到电影偏好上，如果相关系数为负数，比如《老炮儿》和《唐人街探案》，意思是说，喜欢《老炮儿》的人，存在厌恶《唐人街探案》的倾向。

## 协同过滤（基于物品）

### 协同过滤算法（基于物品）

- 然后就可以为A推荐电影了，思路是：A只看过两个电影，然后看根据其他电影与这两个电影的相似程度，进行加权评分，得出应该推荐给A的电影，具体方法可以列一个表

协同过滤（基于物品）

| 向A推荐    | A评分 | 星球大战  | “相似分” | 寻龙诀   | “相似分” | 神探夏洛克 | “相似分” | 小门神   | “相似分” |
|---------|-----|-------|-------|-------|-------|-------|-------|-------|-------|
| 老炮儿     | 3.5 | 0.3   | 1.05  | 0.65  | 2.275 | 0.25  | 0.875 | -0.54 | -1.89 |
| 唐人街探案   | 1.0 | -0.67 | -0.67 | -0.68 | -0.68 | -0.38 | -0.38 | 0.32  | 0.32  |
| 求和（推荐度） |     |       | 0.38  |       | 1.595 |       | 0.495 |       | -1.57 |

用A看过的电影的评分，和其他电影的相似度相乘（红框），然后再把相乘后的结果加和（绿框），得出最后的推荐度。这里可以看到，应该向A推荐《寻龙诀》，和上一篇文章用基于用户的协同过滤算法结果是一致的。

## 协同过滤（基于物品）

### 推荐算法的几个基本思想

- 根据和你共同喜好的人来给你推荐（基于用户的）
- 根据你喜欢的物品找出和它相似的来给你推荐（基于物品的）
- 根据你给出的关键字来给你推荐（退化成搜索算法）
- 根据上面的几种条件组合起来给你推荐

## 协同过滤（基于物品）

### 推荐算法的几个基本思想

- 基本思想还是寻找近邻，依据近邻进行推荐。主要差别在于计算相似度的衡量标准上，进而衍生出了各种计算相似度的算法，各种算法的优劣体现在相似度判定的准确度以及算法的计算速度和占用的计算资源：

欧氏距离算法

余弦距离算法

Jaccard距离算法

皮尔逊距离算法